Lecture 6. Unsupervised learning II: probability, variational autoencoders and generation

Deep Learning for Scientific Discovery

2026/2027

1 New samples and hidden causes

A haematology laboratory that studies a rare type of white blood cell has a few hundred microscope images of it and would like a few thousand more, with the same variation of size, colour and shape of the nucleus, to train a classifier or to test an image-analysis pipeline. A statistical physicist who studies the Ising model of a magnet needs thousands of spin configurations at a temperature close to the phase transition, where Monte Carlo sampling is slow, and would like a model that produces them directly once the temperature is given. An astronomer holding tens of thousands of galaxy images would like to know along which few directions these images vary, whether one of them is the size of the galaxy and another its inclination, without labelling a single image. A physicist who records the positions of many oscillators, each with its own spring, its own friction and its own initial state, would like a network to find the frequency and the damping from the records alone.

In all four cases what is observed is a collection of samples, images, spin configurations or

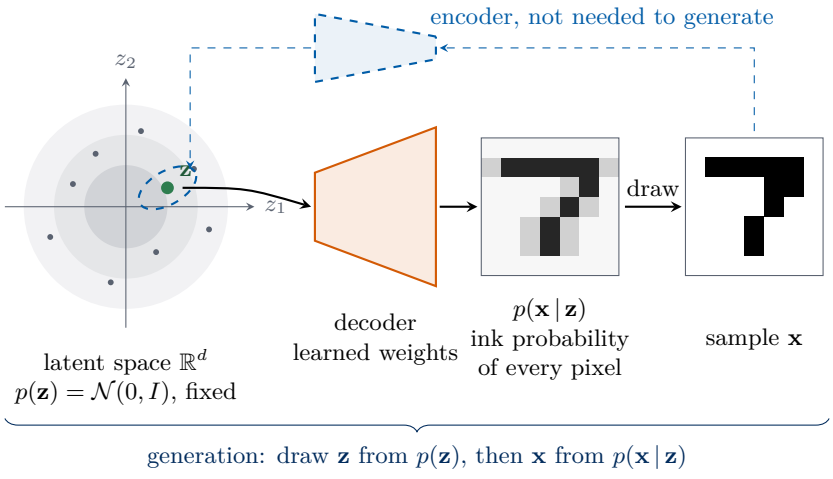


Figure 1: The variational autoencoder as a story of how a sample comes about. A code $\mathbf{z}$ (green) is drawn from the fixed standard Gaussian in the latent space (grey cloud, grey dots are other draws). The learned decoder (orange) turns it into the ink probability of every pixel, shown as shades of grey, and the image $\mathbf{x}$ is drawn from these probabilities. The encoder (blue, dashed) maps a training image back to the small region of codes that could have produced it (dashed ellipse). It is needed to train the decoder, and it later serves to reconstruct and to analyse real images, but drawing new samples does not use it.

trajectories, and what is wanted is a description of the process that produced them. That description has two uses. It must allow new samples to be drawn, samples that resemble the data without copying them. It should also expose a few hidden numbers from which a sample arises, and these numbers should, where possible, coincide with quantities a scientist recognises, such as a temperature, a size or a damping rate. The chapter builds models that attempt both and tests how far they succeed. Some of the answers are partial, and the tests that reveal the limits are part of the method.

Lecture 5 introduced the autoencoder, a pair of networks that compresses a sample $\mathbf{x} \in \mathbb{R}^D$ into a code $\mathbf{z} \in \mathbb{R}^d$ with $d \ll D$ and rebuilds it. Its final section showed why an autoencoder is not a reliable generator. Its encoder maps each training sample to a single point, the training set becomes a cloud of isolated points with large gaps between them, and the decoder is trained only at these points. A code taken from a gap, or drawn at random, still decodes to some image, but to one that the training never constrained, and nothing says from which distribution new codes should be drawn. The codes have no prescribed location or spread either, so there is no distribution from which a new code could be drawn. The same section concluded that a generative model needs two ingredients, codes distributed according to a known density and a decoder trained so that every code drawn from that density decodes to a plausible sample.

These two ingredients describe the data by a story of how a single sample comes about, told as two random draws (Figure 1). First a code $\mathbf{z}$ is drawn in the latent space from the density $p(\mathbf{z})$. This density is not learned. It is chosen before training as the standard Gaussian, a round cloud of codes centred at the origin of $\mathbb{R}^d$, so that drawing a code amounts to drawing d independent Gaussian numbers. Second, the code is passed to the decoder network, and the output of the network fixes the conditional distribution $p(\mathbf{x} \,|\, \mathbf{z})$, the probability of obtaining the sample $\mathbf{x}$ once the code is $\mathbf{z}$. For an image of a digit the decoder gives, for every pixel, the probability that it carries ink, and the image is drawn pixel by pixel from these probabilities. This conditional distribution is the part of the model that is learned, through the weights of the decoder.

A given image $\mathbf{x}$ can come out of the story along many routes, one for every code. The route through a code $\mathbf{z}$ requires that this code is drawn, with probability density $p(\mathbf{z})$, and that the decoder then produces $\mathbf{x}$ from it, with probability $p(\mathbf{x} \,|\, \mathbf{z})$. The probability of $\mathbf{x}$ under the model is the sum of the products of these two factors over all codes, and since the codes form a continuum the sum is an integral,

$$p(\mathbf{x}) = \int p(\mathbf{x} \,|\, \mathbf{z})\, p(\mathbf{z})\, \mathrm{d}\mathbf{z}. \tag{1.1}$$

The density $p(\mathbf{x})$ is never written down as a formula of its own. It follows from the fixed density of codes and the learned decoder, and training adjusts the decoder until this density is high for images that resemble the data and low for everything else. Codes near which the decoder produces a 7 contribute to the probability of a 7, codes near which it produces a 3 contribute to that of a 3, and the integral collects them all.

The figure contains a second network, the encoder, which takes an image and returns the small region of the latent space from which the decoder could have produced it. The encoder is not part of the story of equation (1.1), and drawing new samples does not use it. It is needed to train the decoder, because the integral cannot be computed directly (Section 6), and once trained it also maps real images to their codes, which later sections use to reconstruct, to compare and to modify images. The pair of networks, decoder and encoder, is the variational autoencoder, and it serves all four situations of the opening paragraph.

The sketch and equation (1.1) are a preview. Every object in them, a Gaussian density, a

Table 1: The data sets of the chapter. n counts the training samples, D the numbers per sample. Labels and factors are used only where stated, mostly to evaluate a model after training.

data set	one sample	n	D	labels or factors
Geiger counts	decays in one second	5	1	rate λ, unknown
three groups	a point of the plane	1500	2	group
ring of groups	a point near one of eight centres	fresh draws	2	group
sine curve	a point $(t, \sin 2t)$ plus noise	4000	2	hidden position t
MNIST	28×28 grey image of a digit	60 000	784	digit 0–9
BloodMNIST	28×28 colour image of a cell	11 959	2352	cell type, 8 types
bars	28×28 image of a white bar	20 000	784	angle, shift
Ising model	20×20 spins ± 1	10 200	400	temperature, 51 values
dSprites	32×32 binary image of a sprite	100 000	1024	5 factors
Galaxy10 DECaLS	64×64 colour image of a galaxy	15 076	12 288	morphology, 10 classes
oscillators	100 positions of a noisy oscillator	20 000	100	ω, γ, x_0, v_0
pendulum	50 positions of a damped pendulum	20 000	50	κ, b
coupled oscillators	100 positions of two masses	–	200	k, k_c

conditional distribution, an integral over codes, is defined from the beginning in the sections that follow, and the reader is not expected to understand them yet. The chapter assumes derivatives and integrals of one variable, sums, vectors and matrix products, and the rules of the logarithm. Longer derivations, such as the general multivariate Gaussian and the decomposition of the divergence in Section 11.4, can be skipped on a first reading without losing the main line, which needs their results only. The construction meets a conceptual difficulty, a computational one and one of interpretation. The conceptual difficulty is the meaning of probability in many dimensions. Equation (1.1) is a statement about probability densities in hundreds of dimensions, and the meaning of learning such a density from samples, of evaluating it and of drawing from it must be made concrete before any network is trained. Probability is therefore introduced first, starting from a coin. The computational difficulty is the integral itself. The integral in equation (1.1) runs over all codes and cannot be evaluated for a neural-network decoder, so the quantity that training should maximise is out of reach. The evidence lower bound, the central construction of the chapter, replaces it by a quantity that can be computed and optimised. The difficulty of interpretation arises after training. A model that generates well may still mix the hidden causes of the data in its latent coordinates, and the question when a latent coordinate corresponds to one physical cause leads to the β-weighted variational autoencoder and to the study of disentanglement. When the cause of interest is known for every sample, a label such as the cell type or the temperature, it can be given to the networks directly, which leads to the conditional variational autoencoder.

1.1 Data sets

The data sets of the chapter are listed in Table 1. Each is described in detail where it is first analysed. The handwritten digits of the MNIST collection [1] and the blood-cell images of BloodMNIST [2, 3] are those of Lecture 5. A digit is a vector $\mathbf{x} \in [0,1]^{784}$ of grey values of 28×28 pixels with a label from 0 to 9, and a blood cell is a colour image of 28×28 pixels in three channels, $\mathbf{x} \in [0,1]^{2352}$, with one of eight cell types assigned by clinical pathologists.

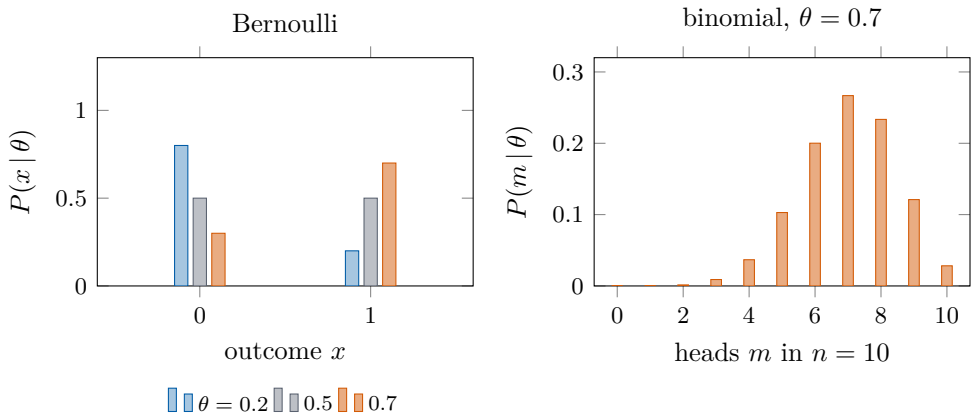


Figure 2: The coin. Left, the Bernoulli distribution (2.1) for three values of θ, two bars that always add up to one. Right, the binomial distribution (2.3) of the number of heads in ten tosses of a coin with $\theta = 0.7$, eleven bars that add up to one.

Two-dimensional points on a sine curve with one hidden number are a test bed on which every quantity of the chapter, including the exact density of the data, can be computed. Spin configurations of the two-dimensional Ising model at 51 temperatures test whether a generative model reproduces physics that is known in closed form. The dSprites images [4], rendered by a program from five known factors, test whether latent coordinates align with those factors, and the Galaxy10 DECaLS images [5, 6] ask the same question for real galaxies whose factors nobody has listed. Simulated trajectories of noisy damped oscillators, and the pendulum of Iten et al. [8], carry known physical parameters against which learned latent coordinates can be checked.

2 Probability from a coin to a Gaussian cloud

A generative model is a probability distribution with adjustable parameters, and everything it does, from training to sampling, is phrased in the language of probability. That language can be built from one assumption only, that a probability is a number between 0 and 1 and that the probabilities of all possible outcomes add up to 1. Each notion is introduced on the smallest example that shows it and is used again later for the variational autoencoder.

2.1 A coin and the Bernoulli distribution

A coin lands heads or tails. We write the outcome of one toss as a number x, with $x = 1$ for heads and $x = 0$ for tails, and call θ the probability of heads. A fair coin has $\theta = \frac{1}{2}$, a bent coin may have $\theta = 0.7$. The two probabilities $P(x = 1) = \theta$ and $P(x = 0) = 1 - \theta$ can be written as one formula,

$$P(x \mid \theta) := \theta^x (1 - \theta)^{1-x}, \qquad x \in \{0, 1\}, \quad 0 \le \theta \le 1. \tag{2.1}$$

For $x = 1$ the second factor has the exponent zero and equals one, which leaves θ. For $x = 0$ the first factor equals one, which leaves $1 - \theta$. Equation (2.1) is the Bernoulli distribution. The symbol $:=$ marks a definition, and it is used throughout the chapter whenever a new symbol is introduced. The left side is a new name for the expression on the right. The vertical bar in $P(x \mid \theta)$ is read "given" and states that the probability of the outcome x is computed for a given value of the parameter θ. The two probabilities add up to $\theta + (1 - \theta) = 1$ for every θ.

Now the coin is tossed n times, and the outcomes $x_1, \ldots, x_n$ are recorded. The tosses are independent, which means that the outcome of one toss does not change the probabilities of the others. The probability of a whole sequence of independent outcomes is then the product of the probabilities of its members,

$$P(x_1, \ldots, x_n \,|\, \theta) = \prod_{i=1}^{n} \theta^{x_i}(1-\theta)^{1-x_i} = \theta^m (1-\theta)^{n-m}, \qquad m = \sum_{i=1}^{n} x_i, \tag{2.2}$$

where m is the number of heads. The product collects one factor θ for every head and one factor $1-\theta$ for every tail. A sequence with m heads can be arranged in $\binom{n}{m}$ different orders, all with the same probability, so the number of heads has the binomial distribution

$$P(m \,|\, \theta) = \binom{n}{m} \theta^m (1-\theta)^{n-m}, \qquad m = 0, 1, \ldots, n. \tag{2.3}$$

Figure 2 shows both distributions. A pixel of a black-and-white image is a variable of the same kind, ink or background, and Section 3.4 uses equation (2.1) with one value of θ for each pixel.

2.2 Continuous quantities and the Gaussian density

A measured intensity, a position or a grey value can take any real value in an interval, and the probability of any single value is zero. Probability is then described by a density $p(x)$, a non-negative function whose area over an interval is the probability that the value falls in it,

$$P(a \le x \le b) = \int_a^b p(x)\,\mathrm{d}x, \qquad \int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1. \tag{2.4}$$

A density is a probability per unit length. Its value at a point is not a probability, and it can exceed one. The most important density is the Gaussian, or normal, density with mean μ and standard deviation $\sigma > 0$,

$$\mathcal{N}(x; \mu, \sigma^2) \coloneqq \frac{1}{\sqrt{2\pi\sigma^2}} \exp\Big(-\frac{(x-\mu)^2}{2\sigma^2} \Big). \tag{2.5}$$

The exponent measures the squared distance of x from the centre μ in units of the width σ, so the density is largest at μ and falls by a factor $e^{-1/2}$ at one standard deviation from it. The prefactor makes the area equal to one, since $\int e^{-u^2/2\sigma^2}\mathrm{d}u = \sqrt{2\pi\sigma^2}$. The fraction of the area within one standard deviation of the mean is 0.683 for every μ and σ. A narrow Gaussian must be tall to keep its area, and for $\sigma = 0.2$ the peak value $1/\sqrt{2\pi \cdot 0.04}$ equals 1.99. Figure 3 shows several Gaussians and a probability as an area.

2.3 Drawing samples and the cumulative distribution

A generative model is useful because new samples can be drawn from it, and the phrase "draw x from p", written $x \sim p$, needs a precise meaning. A draw is the outcome of a random procedure, for example a toss of the coin or a call of a random-number generator, and a distribution describes what happens when the procedure is repeated many times. To draw from p means to run a procedure whose outcomes, collected over many repetitions, fall into every interval $[a, b]$ with the frequency $P(a \le x \le b)$ of equation (2.4). A histogram of many draws, scaled to unit area, then approaches the density, and draws crowd where the density

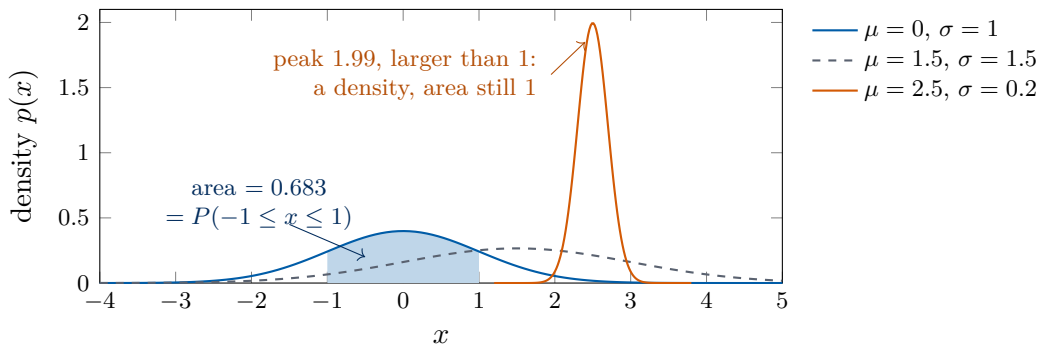


Figure 3: Gaussian densities (2.5). The mean μ moves the curve, the standard deviation σ sets its width. Probabilities are areas, as the shaded area between -1 and 1 under the standard Gaussian shows. The narrow orange curve has a peak value above one, which is allowed for a density because only its area, equal to one, is a probability.

is high and are rare where it is low. A single draw says little, and only the collection of draws reflects the distribution.

Every such procedure on a computer starts from the uniform distribution on the interval from 0 to 1, written $\mathcal{U}(0,1)$, whose density equals 1 on that interval and 0 elsewhere, so that a draw u falls into any subinterval with a frequency equal to its length. A random-number generator produces such draws, and every other distribution is obtained by transforming them. The coin is the simplest case. A draw $u \sim \mathcal{U}(0,1)$ is turned into heads, $x = 1$, if $u < \theta$ and into tails otherwise, and since u falls below θ with frequency θ, the outcomes follow the Bernoulli distribution (2.1).

For a continuous quantity the transformation uses the cumulative distribution function, the probability that a draw does not exceed a given value,

$$F(x) \coloneqq P(X \le x) = \int_{-\infty}^{x} p(x')\,\mathrm{d}x', \tag{2.6}$$

where X denotes the random quantity, called a random variable, and x a number. The function F grows from 0 at $-\infty$ to 1 at $+\infty$, it never decreases, its slope is the density, $F'(x) = p(x)$, and the probability of an interval is the difference $P(a \le X \le b) = F(b) - F(a)$. It is steep where the density is high and flat where the density is low. The inverse of F turns uniform draws into draws from p,

$$x = F^{-1}(u), \qquad u \sim \mathcal{U}(0,1). \tag{2.7}$$

The resulting x does not exceed a value x_0 exactly when u does not exceed $F(x_0)$, which happens with frequency $F(x_0)$, so x has the cumulative distribution F and hence the density p. Figure 4 shows the construction for the standard Gaussian. Ten values of u, evenly spread between 0 and 1, are sent through F^{-1}. Where F is steep, near the centre, they land close together, and where it is flat, in the tails, they spread out, so the resulting values crowd where the density is high, as draws from it should. For the exponential density $p(x) = \lambda e^{-\lambda x}$, $x \ge 0$, the inverse is a formula, $x = -\ln(1-u)/\lambda$. For the Gaussian F has no closed form, and libraries use other transformations of uniform draws that produce standard Gaussian numbers $\varepsilon \sim \mathcal{N}(0,1)$ directly.

A Gaussian with any mean and width is then obtained by shifting and stretching,

$$x = \mu + \sigma\varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,1) \quad \Longrightarrow \quad x \sim \mathcal{N}(\mu, \sigma^2), \tag{2.8}$$

since shifting moves the centre of the bell to μ and stretching widens it by the factor σ. This step, a draw from a fixed simple distribution followed by a deterministic transformation, is

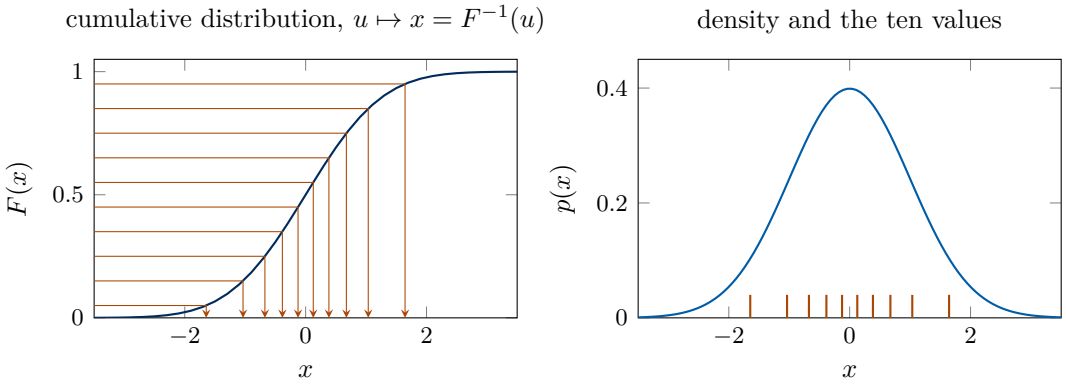


Figure 4: Drawing from the standard Gaussian with the cumulative distribution (2.6). Left, ten values $u = 0.05, 0.15, \ldots, 0.95$, evenly spread on the vertical axis, are carried horizontally to the curve $F(x)$ and down to the x axis, which is the inverse (2.7). Right, the ten resulting values (orange ticks) crowd near the centre, where the density is high, and spread out in the tails. Random uniform values instead of evenly spread ones give random draws from the Gaussian.

the basic pattern of all sampling in the chapter. The variational autoencoder draws its codes in exactly this way (Section 7.3), and its decoder is a learned transformation of such draws (Section 3.7).

2.4 Expectation, variance and covariance

The expectation of a function $f(x)$ under a density p is its average weighted by the density,

$$\mathbb{E}_{x\sim p}[f(x)] \coloneqq \int p(x)\, f(x)\, \mathrm{d}x \;\approx\; \frac{1}{n}\sum_{i=1}^{n} f(x_i), \qquad x_i \sim p, \tag{2.9}$$

where $x \sim p$ means that x is drawn from p. The right-hand side estimates the expectation by an average over n samples. This replacement is used for every loss of the chapter, since a loss is an expectation over the data and is computed as an average over a mini-batch. The estimate scatters around the true value with the standard deviation $\sigma_f/\sqrt{n}$, where σ_f is the standard deviation of $f(x)$ under p. A few samples can therefore give an estimate far from the true value, and many samples give one close to it.

The mean $\mu = \mathbb{E}[x]$ locates a distribution, and the variance

$$\mathrm{Var}[x] \coloneqq \mathbb{E}\big[(x-\mu)^2\big] = \sigma^2 \tag{2.10}$$

measures its spread. For the Gaussian (2.5) the parameters μ and σ^2 are the mean and the variance. A sample $\mathbf{x} = (x_1, \ldots, x_d)$ of several quantities, such as the pixels of an image or the coordinates of a latent code, has a mean vector $\boldsymbol{\mu}$ with components $\mu_j = \mathbb{E}[x_j]$, and two of its components x_j and x_l have the covariance

$$\Sigma_{jl} \coloneqq \mathrm{Cov}[x_j, x_l] \coloneqq \mathbb{E}\big[(x_j-\mu_j)(x_l-\mu_l)\big]. \tag{2.11}$$

The covariance is positive when x_j and x_l tend to lie on the same side of their means and negative when they tend to lie on opposite sides. The numbers Σ_{jl} form the covariance matrix Σ, which has the variances on its diagonal, $\Sigma_{jj} = \mathrm{Var}[x_j]$, and is symmetric. The covariance

carries the units of the two quantities. Dividing it by the two standard deviations gives the dimensionless correlation coefficient

$$\rho_{jl} \coloneqq \frac{\Sigma_{jl}}{\sqrt{\Sigma_{jj}\,\Sigma_{ll}}} \in [-1, 1], \tag{2.12}$$

which equals ± 1 only when x_l is an exact linear function of x_j. From n samples the covariance matrix is estimated by the sample covariance matrix C of Lecture 5, $C_{jl} = \frac{1}{n}\sum_i (x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l)$, whose eigenvectors are the principal directions.

2.5 The multivariate Gaussian

The Gaussian density of a vector $\mathbf{x} \in \mathbb{R}^d$ with mean $\boldsymbol{\mu}$ and covariance matrix Σ is

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma) \coloneqq \frac{1}{(2\pi)^{d/2}\,(\det \Sigma)^{1/2}} \exp\Big(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \Big). \tag{2.13}$$

Each factor has the meaning of its one-dimensional counterpart. The quadratic form in the exponent is a squared distance of $\mathbf{x}$ from the centre measured in units of the spread, with the spread along each direction taken into account through Σ^{-1}. Points with the same value of this form have the same density, and they lie on an ellipsoid around $\boldsymbol{\mu}$. The prefactor normalises the density. The determinant $\det \Sigma$ is the product of the variances along the principal axes, so $(\det \Sigma)^{1/2}$ is proportional to the volume of the ellipsoid, and a wider cloud must have a lower density to keep its total probability equal to one.

The shape of the ellipsoids follows from the eigendecomposition $\Sigma = V \Lambda V^\top$ of Lecture 5, with orthonormal eigenvectors $\mathbf{v}_l$ in the columns of V and eigenvalues λ_l on the diagonal of Λ. In the rotated coordinates $\mathbf{w} = V^\top(\mathbf{x} - \boldsymbol{\mu})$ the quadratic form becomes $\sum_l w_l^2 / \lambda_l$, so the axes of the ellipses point along the eigenvectors and have half-lengths $\sqrt{\lambda_l}$ at unit distance. The principal directions of PCA are the axes of the Gaussian that has the same covariance as the data.

Three cases of a two-dimensional Gaussian with zero mean recur in the chapter, and Figure 5 shows each as a surface and as a map with sample points,

$$\text{(a)}\ \ \Sigma = I, \qquad \text{(b)}\ \ \Sigma = \operatorname{diag}(\sigma_1^2, \sigma_2^2), \qquad \text{(c)}\ \ \Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix},\ \rho \neq 0. \tag{2.14}$$

In case (a) the contours are circles, both coordinates have the same spread and the density looks the same in every direction. In case (b) the contours are ellipses aligned with the coordinate axes. In case (c) the ellipses are tilted, and for $\rho = 0.8$ the eigenvectors are $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$ with eigenvalues $1 + \rho = 1.8$ and $1 - \rho = 0.2$.

When Σ is diagonal the density (2.13) splits into a product. The inverse of a diagonal matrix is diagonal with the entries $1/\sigma_j^2$, its determinant is the product $\prod_j \sigma_j^2$, and the quadratic form becomes a sum of squares, so the exponential of the sum becomes a product of exponentials,

$$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2) = \prod_{j=1}^{d} \frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\Big(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2} \Big) = \prod_{j=1}^{d} \mathcal{N}(x_j; \mu_j, \sigma_j^2). \tag{2.15}$$

Two quantities are called independent when their joint density is the product of their separate densities. Knowing the value of one then tells nothing about the other. Equation (2.15) shows that the coordinates of a Gaussian with diagonal covariance are independent. In case (c) the cross term $-2\rho x_1 x_2$ in the exponent prevents the product form, and knowing x_1 shifts the likely values of x_2.

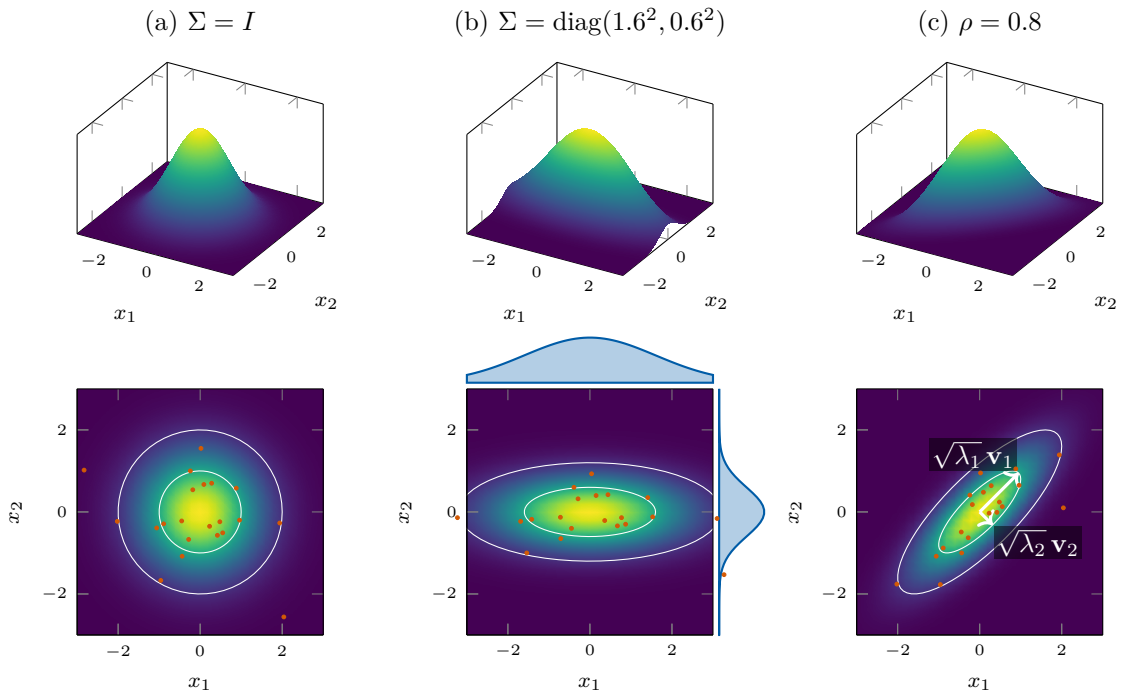


Figure 5: The three Gaussians of equation (2.14), as surfaces (top) and as maps of the density seen from above (bottom), with the contours at one and two units of the distance in the exponent of (2.13) in white and twenty sample points in orange. (a) Identity covariance, circular contours, independent coordinates of equal spread. (b) Diagonal covariance, ellipses aligned with the axes. The blue curves outside the top and right edges are the two one-dimensional Gaussians whose product is the density, equation (2.15). (c) Correlation $\rho = 0.8$, tilted ellipses whose axes are the eigenvectors of Σ (arrows). Here a large x_1 makes a large x_2 likely. The sample points are illustrative draws.

For a Gaussian, zero covariance and independence are the same property, since a zero off-diagonal entry removes the cross term. For other distributions they differ. Let x be uniform on $[-1, 1]$ and $y = x^2$. The covariance

$$\mathrm{Cov}[x, y] = \mathbb{E}[x \cdot x^2] - \mathbb{E}[x]\,\mathbb{E}[x^2] = \mathbb{E}[x^3] - 0 = 0 \tag{2.16}$$

vanishes because x^3 is odd, yet y is a function of x and the two are as dependent as two quantities can be. Figure 6 shows the points. Zero correlation excludes only a linear trend. Independence, the property that the joint density factorises, is the stronger requirement, and it will matter in Section 11.4.

A correlated Gaussian is drawn by the same pattern of shifting and stretching. With the eigenvectors V and eigenvalues Λ of the covariance matrix, $\Sigma = V\Lambda V^\top$, the vector $\mathbf{x} = \boldsymbol{\mu} + V\Lambda^{1/2}\boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ has the covariance $V\Lambda^{1/2}\, I\, \Lambda^{1/2}V^\top = \Sigma$. Independent standard Gaussian numbers are stretched along the principal directions and rotated into place.

2.6 Joint, marginal and conditional distributions

A latent-variable model involves two quantities, an observed x and a hidden z, and their joint density $p(x, z)$ assigns a density to every pair. Three further distributions follow from it. The marginal density of x is obtained by summing the joint density over all values of the hidden

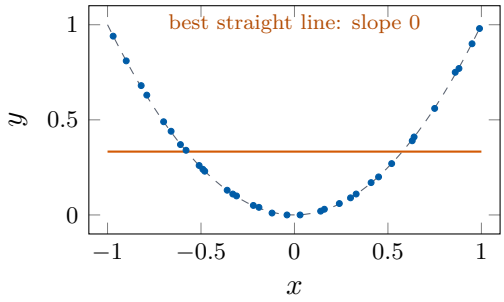


Figure 6: Uncorrelated but dependent. The points (x, x^2) with x uniform on $[-1, 1]$ have zero covariance, equation (2.16), and the best straight-line fit is flat, yet y is determined by x. The points are illustrative draws.

Table 2: Joint probabilities $p(x, z)$ of a screening test. Row sums are the marginal probabilities of the test result, column sums the prior probabilities of the state.

	ill	healthy	$p(x)$
positive	0.09	0.18	0.27
negative	0.01	0.72	0.73
$p(z)$	0.10	0.90	1

quantity,

$$p(x) = \int p(x, z)\, \mathrm{d}z. \tag{2.17}$$

The conditional density of x given z is the joint density restricted to a fixed value of z and renormalised, $p(x \mid z) = p(x, z)/p(z)$, which gives the product rule

$$p(x, z) = p(x \mid z)\, p(z) = p(z \mid x)\, p(x). \tag{2.18}$$

The two factorisations describe the same joint density in two orders. The first is the generative order, in which z is drawn first and x is then drawn given z. The second is the inferential order, which asks for the hidden z once x has been observed. Equating them gives the relation that turns one order into the other.

Bayes' theorem. *For a joint density $p(x, z) = p(x \mid z)\, p(z)$ and an observed x with $p(x) > 0$,*

$$p(z \mid x) = \frac{p(x \mid z)\, p(z)}{p(x)}, \qquad p(x) = \int p(x \mid z')\, p(z')\, \mathrm{d}z'. \tag{2.19}$$

A table with two hidden and two observed values shows all of these with numbers that can be checked on paper. A screening test for a disease is positive for 90% of the ill and for 20% of the healthy, and 10% of the screened people are ill. The hidden quantity z is the state, ill or healthy, with the prior $p(\text{ill}) = 0.1$, and the observed quantity x is the test result. The product rule (2.18) gives the joint probabilities of Table 2, for example $p(\text{positive}, \text{ill}) = 0.9 \cdot 0.1 = 0.09$. The marginal probability of a positive test is the sum of its row, $p(\text{positive}) = 0.09 + 0.18 = 0.27$, and Bayes' theorem gives the posterior of a person who tested positive, $p(\text{ill} \mid \text{positive}) = 0.09/0.27 = 1/3$. A positive test raises the probability of the disease from 0.1 to 0.33, and it does not make the disease certain, because the healthy are nine times more numerous and produce two thirds of the positive tests.

Figure 7 computes the same quantities for continuous variables, in an example in which $z \sim \mathcal{N}(0, 1)$ and, given z, x is Gaussian around $2 \tanh(2z)$ with standard deviation 0.35. The

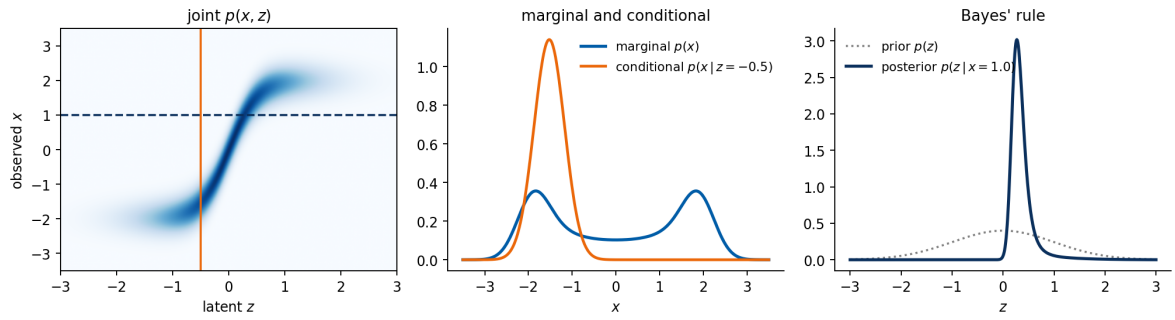


Figure 7: Joint, marginal and conditional densities for $z \sim \mathcal{N}(0, 1)$ and $x \mid z \sim \mathcal{N}(2\tanh(2z), 0.35^2)$, evaluated on a grid. Left, the joint density $p(x, z)$, with the vertical line $z = -0.5$ and the horizontal line $x = 1$. Middle, the marginal $p(x)$ with two peaks and the conditional $p(x \mid z = -0.5)$, a slice of the joint density along the vertical line, renormalised. Right, the prior $p(z)$ and the posterior $p(z \mid x = 1)$ from Bayes' theorem, a slice along the horizontal line.

joint density is concentrated along the curve $x = 2\tanh(2z)$. The marginal $p(x)$ has two peaks near ± 2, because the tanh maps most values of z close to ± 2. Observing $x = 1$ narrows the broad prior of z, of standard deviation 1, to a posterior with standard deviation 0.245 whose peak lies close to the value $z \approx 0.27$ at which $2\tanh(2z) = 1$. The posterior is skewed, with a longer tail towards large z, where $2\tanh(2z)$ flattens and many values of z give an x near one, so its mean, 0.360, lies to the right of the peak. The same structure, a broad distribution of a hidden quantity that an observation narrows to a small region, is the structure of the variational autoencoder.

2.7 Probability and likelihood

A model $p(x \mid \theta)$ gives the probability, or the density, of data x when its parameters are θ. The same formula can be read in two ways. With θ fixed and x varied it is a probability distribution over outcomes, and summed or integrated over all outcomes it gives one. With the data fixed at the values actually observed and θ varied it becomes a function of the parameters.

Definition 2.1 (Likelihood)**.** Let $p(x \mid \theta)$ be a model with parameters θ, and let x_{obs} be the observed data. The likelihood of the parameters is the function

$$L(\theta) := p(x_{\text{obs}} \mid \theta),$$

read as a function of θ with the data held fixed. For n independent observations $x_1, \ldots, x_n$ it is the product $L(\theta) := \prod_{i=1}^{n} p(x_i \mid \theta)$. The maximum-likelihood estimate is the value of the parameters that makes the observed data most probable, $\hat{\theta} := \arg\max_\theta L(\theta)$.

The likelihood is not a probability distribution over θ. It ranks parameter values by how probable they make the observed data, and its integral over θ need not equal one. A distribution over θ is obtained only by multiplying the likelihood by a prior distribution of θ and normalising, by Bayes' theorem (2.19), as Section 2.8 shows for a coin.

A counting experiment shows both readings on a quantity measured in every natural science. A Geiger counter next to a weak radioactive source records the number of decays k in one second (Figure 8). The source holds a very large number N of nuclei, each of which decays within the second independently and with a tiny probability θ, so the count is binomial (2.3)

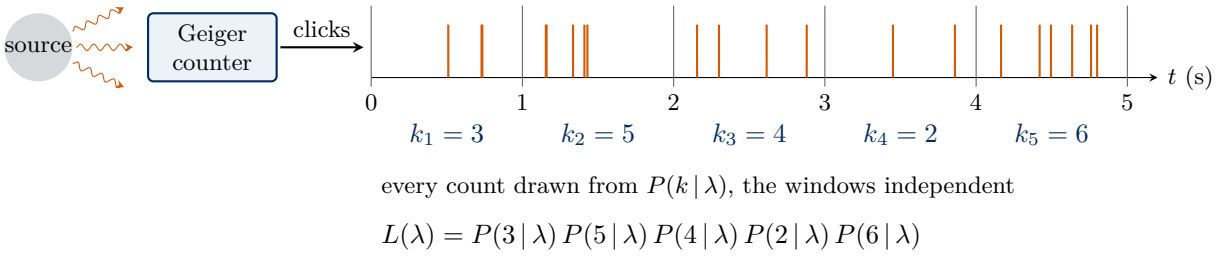


Figure 8: The counting experiment behind the Poisson example. Decays of a radioactive source reach a Geiger counter at random times (orange clicks). The time axis is cut into five windows of one second, and the counts $k_1, \ldots, k_5$ in the windows are independent draws from the Poisson distribution (2.20) with the unknown rate λ. The likelihood of the rate is the product of the five probabilities, equation (2.21).

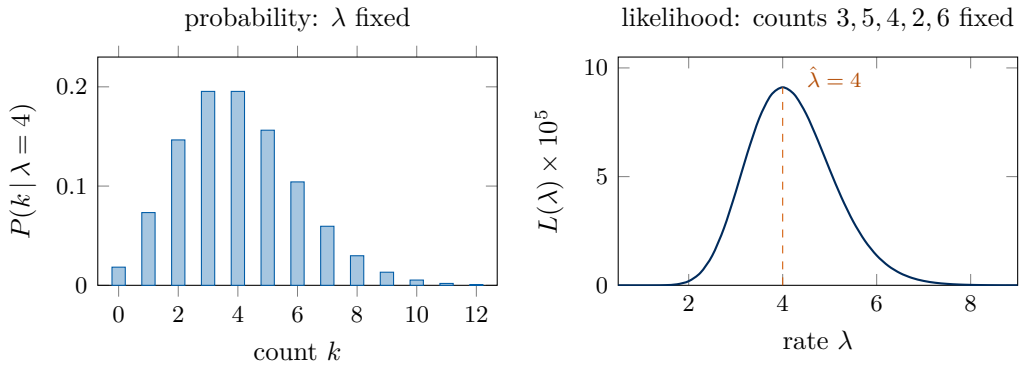


Figure 9: The two readings of the Poisson formula (2.20). Left, the distribution of a single count for the rate $\lambda = 4$, bars that add up to one. Right, the likelihood (2.21) of the rate for the five observed counts, largest at the mean count $\hat{\lambda} = 4$. Its peak value is $9.1 \cdot 10^{-5}$ and its area $2.1 \cdot 10^{-4}$, neither of them one.

with $n = N$. In the limit of large N and small θ with a fixed mean $N\theta = \lambda$ the binomial distribution becomes the Poisson distribution

$$P(k \mid \lambda) := \frac{\lambda^k e^{-\lambda}}{k!}, \qquad k = 0, 1, 2, \ldots, \tag{2.20}$$

whose single parameter $\lambda > 0$ is the mean number of counts per second, the rate of the source. The same distribution describes photons from a faint star in a telescope pixel, cells in a field of a microscope or mutations in a stretch of a genome. With λ fixed and k varied, equation (2.20) is a probability distribution over counts. For $\lambda = 4$ the probability of exactly three counts is $4^3 e^{-4}/3! = 0.195$, and the probabilities of all counts add up to one (Figure 9, left).

Suppose now that five one-second intervals gave the counts $3, 5, 4, 2, 6$ and that the rate is unknown. The intervals are independent, so the likelihood is the product of five Poisson probabilities,

$$L(\lambda) = \prod_{i=1}^{5} \frac{\lambda^{k_i} e^{-\lambda}}{k_i!} = \frac{\lambda^{20}\, e^{-5\lambda}}{3!\, 5!\, 4!\, 2!\, 6!}, \tag{2.21}$$

where 20 is the total number of counts. Its maximum follows from the derivative of its logarithm,

$$\frac{\mathrm{d}}{\mathrm{d}\lambda} \ln L(\lambda) = \frac{20}{\lambda} - 5 = 0 \quad \Longrightarrow \quad \hat{\lambda} = \frac{20}{5} = 4, \tag{2.22}$$

count k	0	1	2	3	4	5	6	7	8	9	10	11	12	
$\lambda = 1$	0.37	0.37	0.18	0.06	0.02	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	$\sum_k \approx 1$
$\lambda = 2$	0.14	0.27	0.27	0.18	0.09	0.04	0.01	0.00	0.00	0.00	0.00	0.00	0.00	$\sum_k \approx 1$
$\lambda = 3$	0.05	0.15	0.22	0.22	0.17	0.10	0.05	0.02	0.01	0.00	0.00	0.00	0.00	$\sum_k \approx 1$
$\lambda = 4$	0.02	0.07	0.15	0.20	0.20	0.16	0.10	0.06	0.03	0.01	0.01	0.00	0.00	$\sum_k \approx 1$
$\lambda = 6$	0.00	0.01	0.04	0.09	0.13	0.16	0.16	0.14	0.10	0.07	0.04	0.02	0.01	$\sum_k \approx 1$

Figure 10: The Poisson probabilities $P(k \,|\, \lambda)$ of a single count, with the count k across and five values of the rate λ down, shaded by size. Read along a row (orange frame, $\lambda = 4$) the numbers are a probability distribution over counts and add up to one, up to the counts above 12 that are not shown. Read down a column (blue frame, $k = 4$) they are the likelihood of λ for one observed count of four, values of a function of λ that is not normalised over λ.

the mean count, which is what an experimenter would have taken as the estimate of the rate without any theory. Figure 9 (right) draws $L(\lambda)$. Its largest value, $L(4) = 9.1 \cdot 10^{-5}$, is far below one, although $\lambda = 4$ is the best possible rate. The five counts are only one of very many outcomes that the source could have produced, and each of them is improbable.

The likelihood is not a probability distribution over λ, and Figure 10 shows why. The table of the values $P(k \,|\, \lambda)$ has one row for each rate and one column for each count. Every row is a probability distribution and adds up to one. A column is the likelihood of one observed count, and nothing makes it add up to one. For the five counts the area under the curve of Figure 9 follows from the integral $\int_0^\infty \lambda^m e^{-a\lambda} \, \mathrm{d}\lambda = m!/a^{m+1}$, with $m = 20$ and $a = 5$,

$$\int_0^\infty L(\lambda) \, \mathrm{d}\lambda = \frac{20!}{5^{21} \; 3! \, 5! \, 4! \, 2! \, 6!} = 2.1 \cdot 10^{-4}. \tag{2.23}$$

The statement that $\lambda = 4$ has the largest likelihood compares rates. A probability for the rate itself requires a prior distribution of λ and Bayes' theorem.

2.8 The vocabulary of Bayesian inference

The variational autoencoder borrows its vocabulary from Bayesian inference, and its words apply first to the coin and then to a latent code. A belief about the coin before any toss is expressed by a prior density $p(\theta)$. If the coin is believed to be roughly fair, a reasonable choice is a density peaked at one half, for example $p(\theta) \propto \theta^4(1-\theta)^4$, normalised to area one. Suppose that ten tosses gave seven heads. By the binomial formula (2.3) the likelihood is $L(\theta) = 120\,\theta^7(1-\theta)^3$, largest at the observed fraction of heads, $\hat{\theta} = 0.7$. Bayes' theorem (2.19) with z replaced by θ gives the posterior

$$p(\theta \,|\, m) = \frac{P(m \,|\, \theta)\, p(\theta)}{P(m)} \;\propto\; \theta^7(1-\theta)^3 \cdot \theta^4(1-\theta)^4 = \theta^{11}(1-\theta)^7, \tag{2.24}$$

normalised to area one. The posterior peaks at $\theta = 11/18 = 0.61$, between the prior peak 0.5 and the likelihood peak 0.7, and it is narrower than the prior because ten tosses have added information. The denominator $P(m) = \int P(m \,|\, \theta)\, p(\theta) \, \mathrm{d}\theta$ is the probability of the observed data averaged over all values of θ that the prior allows. It is called the evidence, or the marginal likelihood, and it does not depend on θ. Figure 11 shows the three curves.

The same words apply when the unknown is a latent code $\mathbf{z}$ of a sample $\mathbf{x}$. The prior $p(\mathbf{z})$ describes the codes before any sample is seen. The likelihood $p(\mathbf{x} \,|\, \mathbf{z})$ gives the probability of

Table 3: The vocabulary of Bayesian inference for the coin and for the latent code of a variational autoencoder.

term	symbol	meaning	coin	variational autoencoder
prior	$p(\theta)$, $p(\mathbf{z})$	belief before the data are seen	roughly fair coin	$\mathcal{N}(0, I)$, the codes before any $\mathbf{x}$
likelihood	$p(x \mid \theta)$, $p(\mathbf{x} \mid \mathbf{z})$	probability of the observed data for a given parameter or code, read as a function of θ or $\mathbf{z}$ it is not normalised	$120\,\theta^7(1-\theta)^3$	the decoder $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$
posterior	$p(\theta \mid x)$, $p(\mathbf{z} \mid \mathbf{x})$	updated belief after the data	peak at 0.61	codes that could have produced $\mathbf{x}$, intractable
evidence, marginal likelihood	$p(x)$, $p(\mathbf{x})$	probability of the data averaged over the prior, the normaliser of Bayes' theorem	$P(m{=}7)$	$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x}\vert\mathbf{z})p(\mathbf{z})\mathrm{d}\mathbf{z}$
joint	$p(x, \theta)$, $p(\mathbf{x}, \mathbf{z})$	probability of data and parameter together	prior × likelihood	decoder × prior
marginal	$p(x) = \int p(x, z)\mathrm{d}z$	distribution of one quantity, the other summed out	–	$p_{\boldsymbol{\theta}}(\mathbf{x})$, $p(\mathbf{z})$
conditional	$p(x \mid z)$	distribution of one quantity at a fixed value of the other	binomial	decoder, encoder
approximate posterior	$q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$	a simple distribution chosen to resemble the posterior	–	encoder cloud, Section 6
variational	–	choosing the best member of a family of approximating distributions	–	best Gaussian cloud
amortised	–	one network gives the approximation for every input	–	one encoder for all $\mathbf{x}$

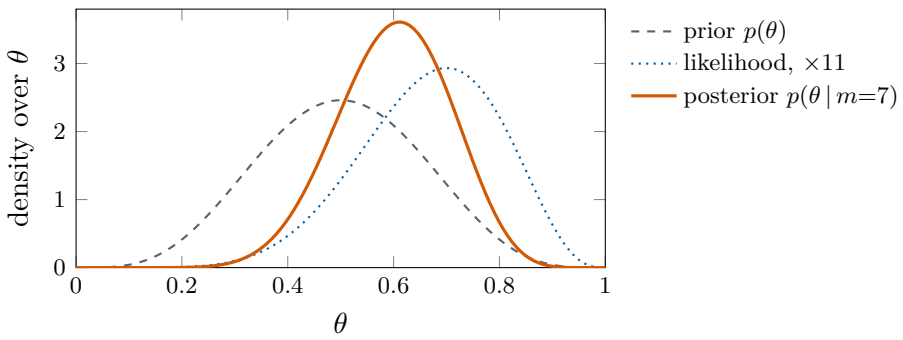


Figure 11: Bayes' theorem for the coin with seven heads in ten tosses. The prior (grey) expresses the belief that the coin is roughly fair. The likelihood (blue, multiplied by 11 so that its area is one for display) peaks at 0.7. Their normalised product, the posterior (orange), peaks at 0.61 and is narrower than the prior.

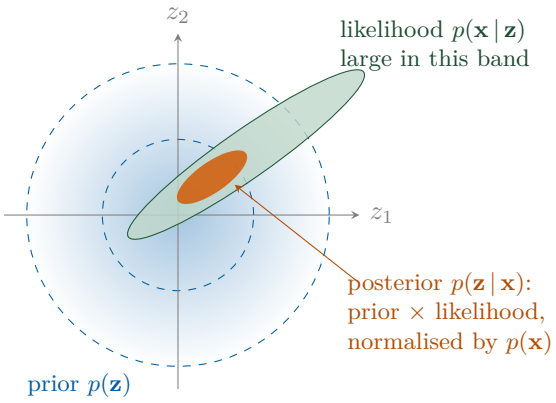


Figure 12: Bayesian vocabulary in the latent space. The prior $p(\mathbf{z})$ spreads over a disc. One observed sample $\mathbf{x}$ is explained well only by codes in a narrow band (green), where the likelihood $p(\mathbf{x}\,|\,\mathbf{z})$ is large. The region is a band because moving the code along one direction changes the decoded image little, so that many codes along that direction decode to almost the same image. The posterior (orange) is the part of the prior inside that band, renormalised by the evidence $p(\mathbf{x})$. Drawings in this style, without numbers on the axes, are schematic.

the sample for a given code, and as a function of $\mathbf{z}$ it marks the region of codes that explain the sample well. The posterior $p(\mathbf{z}\,|\,\mathbf{x})$ is the prior restricted to that region and normalised, the set of codes that could have produced $\mathbf{x}$. The evidence $p(\mathbf{x}) = \int p(\mathbf{x}\,|\,\mathbf{z})\,p(\mathbf{z})\,\mathrm{d}\mathbf{z}$ is the model of the data itself, equation (1.1). Figure 12 draws the picture, and Table 3 collects the vocabulary together with three further terms that Section 6 introduces.

The three cases of Figure 5 return in the latent space with these words. The prior $\mathcal{N}(0, I)$ is case (a). The variational autoencoder approximates the posterior of each sample by a Gaussian with diagonal covariance, case (b). The exact posterior, and the distribution of the codes of all samples taken together, can look like case (c) or be non-Gaussian altogether. Section 8 returns to this distinction, which decides what disentanglement can mean.

3 Learning a probability distribution from samples

3.1 What is available and what is wanted

The phrase "learning a probability distribution" needs a definition, because the distribution that produced the data is never seen. What is available is a finite set of samples $\mathbf{x}_1, \ldots, \mathbf{x}_n$, assumed to be independent draws from an unknown density $p_{\text{data}}(\mathbf{x})$. For the coin, p_{data} is the Bernoulli distribution with the unknown bias of the real coin. For the digits it is the distribution of all images that people produce when they write a digit, a density over $\mathbb{R}^{784}$ that no formula describes. For the Ising model at temperature T it is the Boltzmann distribution, known in closed form but too complicated to sample quickly near the phase transition.

Modelling p_{data} means three choices. The first is a family of densities $p_\theta(\mathbf{x})$, labelled by parameters θ, chosen so that each member can be evaluated at any $\mathbf{x}$, or sampled from, or both. The family of the coin is the Bernoulli distribution with one parameter, the family of a single Gaussian has a mean and a covariance, and the family of a variational autoencoder has the weights of a neural network. The second choice is a criterion of closeness between p_θ and p_{data} that can be computed from the samples alone. The third is an optimisation of θ under that criterion. The fitted model is judged afterwards in two ways, by the log-likelihood

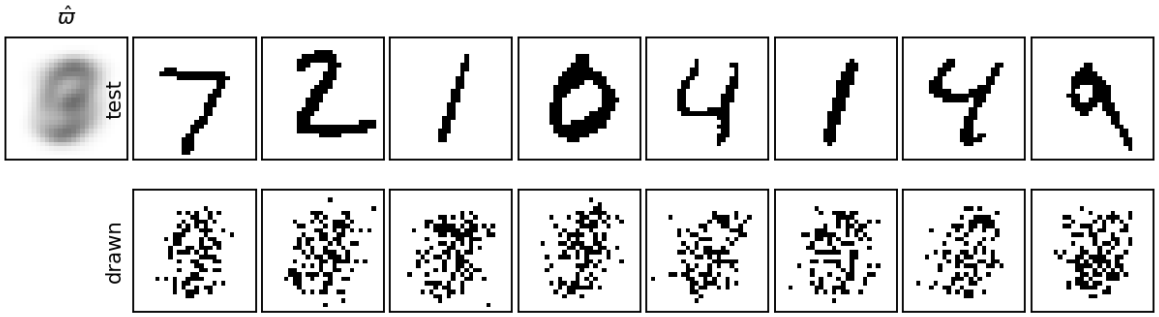


Figure 13: The model of independent pixels (3.2) fitted to binarised MNIST digits. Left, the maximum-likelihood probabilities of ink $\hat{\varpi}_j$, the mean image. Top row, eight test digits. Bottom row, eight digits drawn from the fitted model, every pixel inked with its own probability. The drawn images have the right amount of ink in the right region, but no strokes, because the model cannot express that neighbouring pixels are inked together.

$\ln p_\theta(\mathbf{x})$ it assigns to samples held out from the training, and by whether new samples drawn from p_θ look like the data. Neither test returns p_{data} itself. A learned distribution is always a member of the chosen family, and the best that training can achieve is the member closest to p_{data}.

3.2 Maximum likelihood and the negative log-likelihood

Every member p_θ of the chosen family is a candidate description of the data, an ansatz, and $p_\theta(\mathbf{x})$ is the probability, or the density, of obtaining the outcome $\mathbf{x}$ if the data were produced by this candidate. For a fixed θ one can ask how probable the candidate makes exactly the data at hand. The samples are independent, so the answer is the product of the probabilities of the single samples, the likelihood of Section 2.7,

$$L(\theta) = \prod_{i=1}^{n} p_\theta(\mathbf{x}_i), \qquad \hat{\theta} = \arg\max_\theta L(\theta). \tag{3.1}$$

Maximum likelihood chooses the candidate under which the observed data are most probable. The value of L is not one, even for a candidate that coincides with p_{data}. For discrete data every factor is the probability of one particular outcome, below one, and even the true distribution spreads its probability over very many outcomes. For continuous data the factors are densities, which can exceed one, and the likelihood can then be any positive number. In both cases its absolute value carries no meaning.

The handwritten digits show how far from one the likelihood is. The simplest candidate family treats every pixel of a binarised digit, ink $x_j = 1$ or background $x_j = 0$, as an independent Bernoulli variable with its own probability of ink ϖ_j, after the grey values are turned into ink where they exceed 0.5,

$$p_{\boldsymbol{\varpi}}(\mathbf{x}) = \prod_{j=1}^{784} \varpi_j^{x_j} (1 - \varpi_j)^{1 - x_j}, \tag{3.2}$$

a product of 784 Bernoulli distributions (2.1). Maximising the likelihood of the 60 000 training digits pixel by pixel, as for the Poisson rate (2.22), gives $\hat{\varpi}_j$ equal to the fraction of training digits with ink at pixel j, the mean image of Figure 13. At 108 pixels near the border no training digit has ink, and $\hat{\varpi}_j = 0$ would give a test digit with ink there the probability zero and the log-likelihood $-\infty$. The probabilities are therefore kept between 10^{-3} and $1 - 10^{-3}$.

Under this fitted model the first test digit, a 7, has the probability $p_{\hat{\theta}}(\mathbf{x}) = e^{-177.9} \approx 10^{-77}$, and the probability of all 60 000 training digits together is about $e^{-1.2\cdot 10^7}$. Only the comparison of the likelihoods of different candidates carries meaning.

In practice one minimises the negative log-likelihood per sample,

$$\ell(\theta) \coloneqq -\frac{1}{n} \ln L(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \ln p_\theta(\mathbf{x}_i), \tag{3.3}$$

for three reasons. The logarithm turns the product into a sum, and a sum over samples is what a mini-batch estimates. The product itself cannot be computed. A thousand samples, each of probability 0.01, give a likelihood of 10^{-2000}, far smaller than the smallest positive number of about 10^{-308} that double-precision arithmetic represents, so the product rounds to zero, while the logarithm -4605 is an ordinary number. The logarithm is strictly increasing, so the maximum of $\ln L$ lies at the same θ as the maximum of L. The minus sign turns the maximisation into the minimisation of a loss, and the factor $1/n$ makes its value independent of the size of the data set.

For a one-dimensional Gaussian model $p_\theta(x) = \mathcal{N}(x; \mu, \sigma^2)$ with $\theta = (\mu, \sigma^2)$ the minimisation can be carried out in closed form. Inserting equation (2.5) gives

$$\ell(\mu, \sigma^2) = \frac{1}{2n\sigma^2} \sum_{i=1}^{n} (x_i - \mu)^2 + \frac{1}{2} \ln\left(2\pi\sigma^2\right), \tag{3.4}$$

and setting the two derivatives to zero,

$$\frac{\partial \ell}{\partial \mu} = -\frac{1}{n\sigma^2} \sum_i (x_i - \mu) = 0, \qquad \frac{\partial \ell}{\partial \sigma^2} = -\frac{1}{2n\sigma^4} \sum_i (x_i - \mu)^2 + \frac{1}{2\sigma^2} = 0, \tag{3.5}$$

yields the sample mean and the sample variance,

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^{n} x_i, \qquad \hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^{n} (x_i - \hat{\mu})^2. \tag{3.6}$$

Fitting a Gaussian to measurements by computing their mean and variance is therefore maximum likelihood. The same holds in d dimensions, where the fitted covariance matrix is the sample covariance matrix C of Lecture 5.

3.3 Cross-entropy and the Kullback–Leibler divergence

The negative log-likelihood measures how well p_θ describes the samples. Its meaning becomes clear when the number of samples grows. By equation (2.9) the average over samples tends to an expectation under p_{data}, the cross-entropy of the model with respect to the data,

$$\ell(\theta) \xrightarrow{n\to\infty} -\mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\left[\ln p_\theta(\mathbf{x})\right] \equiv H(p_{\text{data}}, p_\theta). \tag{3.7}$$

Adding and subtracting $\mathbb{E}_{p_{\text{data}}}[\ln p_{\text{data}}]$ splits it into two parts,

$$H(p_{\text{data}}, p_\theta) = \underbrace{-\mathbb{E}_{p_{\text{data}}}\left[\ln p_{\text{data}}(\mathbf{x})\right]}_{H(p_{\text{data}})} + \underbrace{\mathbb{E}_{p_{\text{data}}}\left[\ln \frac{p_{\text{data}}(\mathbf{x})}{p_\theta(\mathbf{x})}\right]}_{\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)}. \tag{3.8}$$

The first part is the entropy of the data distribution, $H(p_{\text{data}}) = -\mathbb{E}_{p_{\text{data}}}[\ln p_{\text{data}}(\mathbf{x})]$, the average surprise of the data under their own distribution. It is small when the data are

concentrated on a few outcomes and large when they are spread over many, and with the natural logarithm it is measured in nats, one nat being $1/\ln 2 = 1.44$ bits. An event of probability one half has the surprise $-\ln \frac{1}{2} = \ln 2 = 0.693$ nats, which is one bit. A fair coin has the entropy $\ln 2 = 0.693$ nats, one bit per toss, and a coin with $\theta = 0.9$, whose outcome is easier to guess, has the lower entropy 0.325 nats. It does not depend on θ. The second part is the Kullback–Leibler divergence.

Definition 3.1 (Kullback–Leibler divergence)**.** For two densities p and q over the same space,

$$\mathrm{KL}(p \,\|\, q) \coloneqq \mathbb{E}_{\mathbf{x}\sim p}\Big[\ln \frac{p(\mathbf{x})}{q(\mathbf{x})}\Big] = \int p(\mathbf{x}) \ln \frac{p(\mathbf{x})}{q(\mathbf{x})}\, \mathrm{d}\mathbf{x}. \tag{3.9}$$

The quantity $-\ln q(\mathbf{x})$ is the surprise of observing $\mathbf{x}$ under the model q, large for an outcome that q considers rare. The divergence is the average extra surprise when data drawn from p are described by q instead of by p itself. With the natural logarithm it is measured in nats. For two outcomes the definition can be evaluated directly. A fair coin, $p = (0.5, 0.5)$, described by a model of a bent coin, $q = (0.9, 0.1)$, gives $\mathrm{KL}(p \,\|\, q) = 0.5 \ln \frac{0.5}{0.9} + 0.5 \ln \frac{0.5}{0.1} = -0.294 + 0.805 = 0.511$ nats. The rare outcome of the model, tails with $q = 0.1$, occurs half of the time, and its large extra surprise dominates. The opposite order gives $\mathrm{KL}(q \,\|\, p) = 0.368$ nats, a different number. It is never negative. The proof uses a known fact about functions whose graph bends downwards, Jensen's inequality. The chapter needs this fact twice. Here it shows that a divergence cannot be negative, which makes maximum likelihood a minimisation of the divergence from the data. In Section 6.2 it replaces the logarithm of an average that cannot be computed by an average of logarithms that can, and this replacement, the evidence lower bound, is the loss on which the variational autoencoder is trained.

Definition 3.2 (Concave function)**.** A function f on an interval is concave if every chord of its graph lies on or under the graph,

$$f\big(t\,u_1 + (1-t)\,u_2\big) \;\geq\; t\,f(u_1) + (1-t)\,f(u_2) \qquad \text{for all } u_1, u_2 \text{ in the interval and } 0 \leq t \leq 1.$$

A twice differentiable function is concave when $f''(u) \leq 0$ everywhere, and strictly concave when $f''(u) < 0$. The logarithm is strictly concave on $u > 0$, since $(\ln u)'' = -1/u^2$.

Theorem 3.3 (Jensen's inequality [9, 10])**.** Let f be a concave function and u a random variable with values in its interval. Then

$$\mathbb{E}\big[f(u)\big] \;\leq\; f\big(\mathbb{E}[u]\big).$$

If f is strictly concave, equality holds only when u takes a single value with probability one.

For two equally probable values u_1 and u_2 the theorem is the chord condition of the definition with $t = \frac{1}{2}$. For $u_1 = 1$ and $u_2 = 100$ the logarithm of the average is $\ln 50.5 = 3.92$, and the average of the logarithms is $\frac{1}{2}(0 + \ln 100) = 2.30$. The average of the two logarithms, the midpoint of the chord, lies under the logarithm of their average (Figure 14). For a distribution with many values the same holds for every weighted average, and the proof for a general distribution is given in [10, Theorem 2.6.2].

Applied with $u = q(\mathbf{x})/p(\mathbf{x})$ and $\mathbf{x} \sim p$,

$$-\,\mathrm{KL}(p \,\|\, q) = \mathbb{E}_p\Big[\ln \frac{q}{p}\Big] \leq \ln \mathbb{E}_p\Big[\frac{q}{p}\Big] = \ln \int q(\mathbf{x})\, \mathrm{d}\mathbf{x} = \ln 1 = 0, \tag{3.10}$$

with equality only when $q = p$. Equations (3.7)–(3.10) together state that minimising the negative log-likelihood minimises $\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$, the divergence of the model from the data,

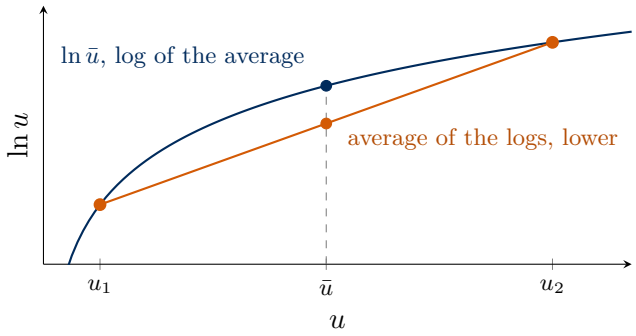


Figure 14: Jensen's inequality for the logarithm, a concave function. For two equally probable values u_1 and u_2, the average of their logarithms is the midpoint of the chord (orange) and lies under the curve at the average $\bar{u} = \frac{1}{2}(u_1 + u_2)$, which is the logarithm of the average (blue). The more the values differ, the larger the difference.

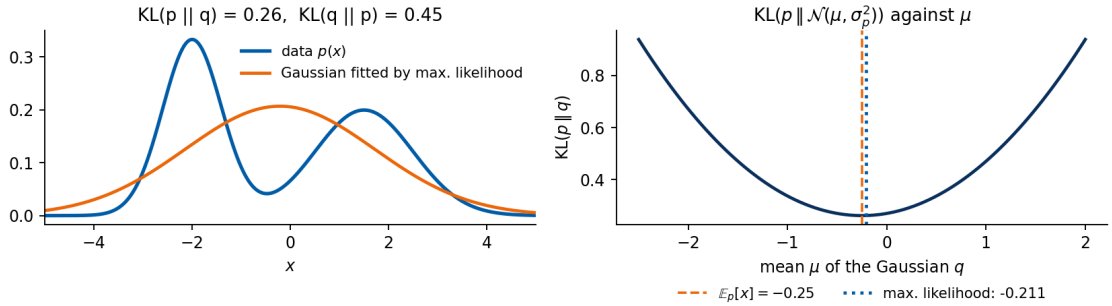


Figure 15: Maximum likelihood minimises the Kullback–Leibler divergence. Left, a density p with two peaks (blue) and the Gaussian q^* fitted to 10^4 of its samples by maximum likelihood (orange). The title gives the divergences in both orders. Right, $\mathrm{KL}(p \,\|\, q)$ for Gaussians q whose width is fixed at the standard deviation σ_p of p, as a function of their mean. The minimum lies at the mean of p (dashed), and the maximum-likelihood mean from the samples (dotted) is close to it.

and that the smallest value the negative log-likelihood can reach is the entropy of the data. The cross-entropy loss used for classification in the earlier lectures is the same construction applied to the distribution of labels. In the ideal case, a family that contains p_{data} and a very large data set, the minimum is reached at $p_\theta = p_{\text{data}}$, and the negative log-likelihood per sample then equals the entropy of the data, not zero. For the Geiger counter with the true rate $\lambda = 4$ this floor is the entropy of the Poisson distribution, 2.09 nats per count. A small data set can be fitted below the floor. The five counts of Section 2.7 give 1.86 nats per count at $\hat{\lambda} = 4$, because counts close to their own mean happen to be probable, and new counts from the same source would give more. For this reason the quality of a model is measured by the negative log-likelihood of samples held out from the training.

The model of independent pixels (3.2) shows the other side. With 784 parameters and 60 000 training digits it cannot adapt to the particular sample, and its negative log-likelihood is 206.2 nats per digit on the training digits and 206.0 on the test digits. That is far better than the 543.4 nats of the candidate that inks every pixel with probability one half, but the drawn digits of Figure 13 show that it is far above the entropy of real digits. The divergence that remains is the price of the independence of the pixels. In a real digit the ink of one pixel makes ink in its neighbours likely, because the pixels belong to one stroke, and a product of independent factors cannot express this. The latent-variable model of Section 4 keeps the

product form of equation (3.2), now as the decoder distribution $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$ with probabilities computed from a code $\mathbf{z}$, and lets the code carry what the pixels share.

The divergence is not symmetric. Figure 15 fits a single Gaussian by maximum likelihood to a density with two peaks. The fitted Gaussian lies between the peaks and covers both, and the divergence that remains is $\mathrm{KL}(p \,\|\, q^*) = 0.26$ nats, while the opposite order gives $\mathrm{KL}(q^* \,\|\, p) = 0.45$ nats. The training of a variational autoencoder uses the divergence in the order $\mathrm{KL}(q \,\|\, p)$, from the approximation to the target (Section 6).

3.4 Squared error and binary cross-entropy as negative log-likelihoods

The two reconstruction losses of Lecture 5 are negative log-likelihoods. Three objects must be kept apart. The network returns numbers, for example 784 values between 0 and 1. These numbers set the parameters of a distribution, for example the probability of ink of every pixel. A sample is drawn from that distribution, for example an image in which every pixel is ink or background. The loss compares the data with the distribution, and a displayed generated image usually shows the numbers, not a sample. Let a network map some input to an output vector $\hat{\mathbf{x}}$ of the same size D as the sample $\mathbf{x}$, and let the output define a distribution of the sample in which the components are independent.

If the components are real numbers with Gaussian noise of a fixed width σ_x around the output, $p(\mathbf{x} \mid \hat{\mathbf{x}}) = \prod_j \mathcal{N}(x_j; \hat{x}_j, \sigma_x^2)$, then

$$-\ln p(\mathbf{x} \mid \hat{\mathbf{x}}) = \frac{1}{2\sigma_x^2} \sum_{j=1}^{D} (x_j - \hat{x}_j)^2 + \frac{D}{2} \ln\left(2\pi\sigma_x^2\right), \tag{3.11}$$

the squared error divided by $2\sigma_x^2$, plus a constant that does not affect training as long as σ_x is fixed. The width σ_x sets the weight of the squared error against every other term of a loss, which matters in Section 7.4. If the components are pixels that are either ink, $x_j = 1$, or background, $x_j = 0$, and the output $\hat{x}_j \in (0, 1)$ is the probability of ink, each pixel is a Bernoulli variable (2.1) with $\theta = \hat{x}_j$, and

$$-\ln p(\mathbf{x} \mid \hat{\mathbf{x}}) = -\sum_{j=1}^{D} \Big[x_j \ln \hat{x}_j + (1 - x_j) \ln\left(1 - \hat{x}_j\right) \Big], \tag{3.12}$$

the binary cross-entropy. Each term is $-\ln \hat{x}_j$ for an ink pixel and $-\ln(1-\hat{x}_j)$ for a background pixel, zero only when the network is certain and right and unbounded when it is certain and wrong. The grey values of MNIST lie between 0 and 1 and are not binary, and equation (3.12) is still used as the loss, which is then a cross-entropy between the grey value and the predicted probability. For a grey value such as 0.4 the expression is no longer the probability of an observation, since a Bernoulli variable takes only the values 0 and 1, and the loss summed over pixels is no longer a normalised likelihood. It is used as a practical loss that is smallest when every predicted probability equals its grey value, and the bounds and the numbers in nats computed with it are then statements about this loss. Choosing a reconstruction loss is therefore choosing a likelihood. The squared error assumes Gaussian noise and the binary cross-entropy assumes independent binary pixels.

3.5 Densities in low dimension

In one or two dimensions a density can be learned directly. A histogram divides the axis into bins and counts samples, which is a density with one parameter per bin. A Gaussian has two

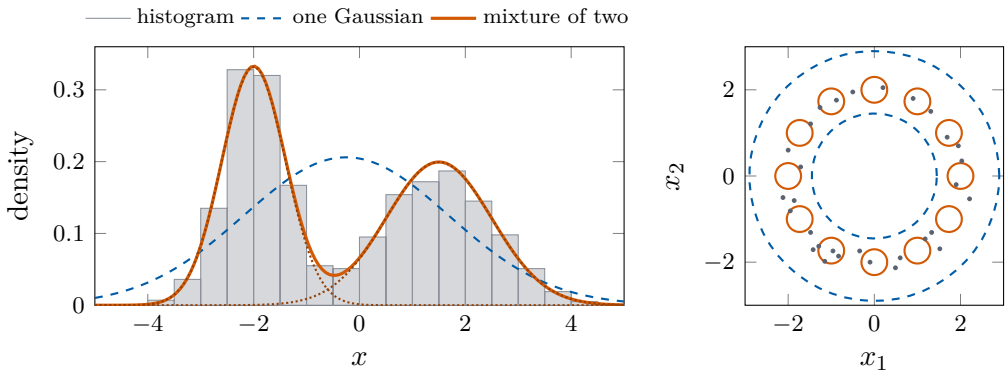


Figure 16: Densities in low dimension. Left, one set of values described by a histogram (grey), by one Gaussian fitted by maximum likelihood (blue, dashed), which puts its peak where few values lie, and by a mixture of two Gaussians (orange), the sum of the two weighted components (dotted). Right, points on a ring. The contours of the best single Gaussian (blue, dashed) are circles centred on the empty middle, while a mixture of twelve small Gaussians (orange contours) follows the ring. Both panels are schematic.

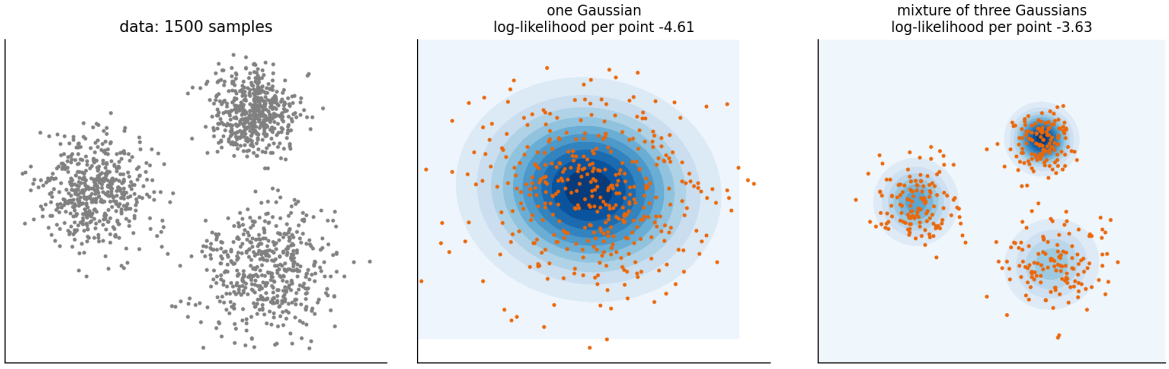


Figure 17: Maximum likelihood in two dimensions. Left, 1500 points drawn from three groups. Middle, a single Gaussian fitted by maximum likelihood (blue density) and 400 new points drawn from it (orange). Right, a mixture of three Gaussians fitted to the same points and 400 new points drawn from it. The titles give the mean log-likelihood per point.

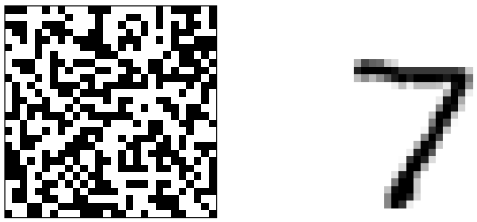
parameters and gives a smooth curve, but it has a single peak. A data set with two groups of values calls for a mixture of Gaussians,

$$p_\theta(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k), \qquad \pi_k \geq 0, \quad \sum_k \pi_k = 1, \tag{3.13}$$

a weighted sum of K Gaussians, which is again a density because each component has area one and the weights add up to one. Figure 16 shows the three descriptions of the same one-dimensional data, and a two-dimensional case in which the data lie on a ring. One Gaussian puts its highest density in the empty centre of the ring, while a mixture of many small Gaussians placed along the ring follows it.

Figure 17 makes the same comparison with fitted models. For 1500 points in the plane drawn from three groups, a mixture of three Gaussians reaches a mean log-likelihood of -3.63 per point against -4.61 for a single Gaussian, and its new samples look like the data, while those of the single Gaussian fill the space between the groups.

A mixture is the first latent-variable model of the chapter. Drawing from equation (3.13)



independent random pixels a handwritten digit

Figure 18: Two points of $\mathbb{R}^{784}$. Left, an image whose pixels are set independently, here by pseudo-random coin tosses. Almost all binary images look like this. Right, a test image of MNIST. Images of digits form a thin, structured subset of all images.

takes two steps. A component label k is drawn with probability π_k, and then $\mathbf{x}$ is drawn from the Gaussian of that component. The label is hidden, since only $\mathbf{x}$ is recorded, and the density of $\mathbf{x}$ is the sum over the hidden label of the joint probability, which is the discrete form of the marginal (2.17).

3.6 Why images defeat the direct approach

A binary image of 28×28 pixels can take $2^{784} \approx 10^{236}$ different values. A histogram with one bin for every possible image cannot be stored, and even a data set of 10^5 images would leave all bins but a negligible fraction empty. Almost every one of these possible images is noise. Figure 18 places an image with independent random pixels next to a handwritten digit. Images that look like digits occupy a vanishing fraction of all images, and they are not scattered at random among them. Neighbouring pixels are strongly correlated, and a digit can be changed into another acceptable digit only along a few directions, such as the thickness, the slant or the size of the stroke. Lecture 5 called this the manifold hypothesis. The data lie close to a thin, curved set of low dimension inside $\mathbb{R}^{784}$.

A single Gaussian over the 784 pixels fails in another way. Its maximum-likelihood mean, equation (3.6), is the average of all training images, a grey blur in which all digits overlap, and its full covariance matrix has $784 \cdot 785/2 = 307\,720$ independent entries. A sample from it is the average image plus correlated noise, which captures the correlations of neighbouring pixels but none of the shape of a stroke, because a Gaussian has a single peak and the data have one ridge for every way of writing every digit. A mixture would need an impractical number of components to follow the curved, low-dimensional set. The direct approach, a density written explicitly over the space of the data, therefore fails for images, and the structure of the data must enter the model.

3.7 Simple randomness pushed through a learned map

The remedy uses the observation that the data are produced by a few hidden causes. A few random numbers are drawn from a simple known distribution and passed through a flexible function that places them on the data manifold. The one-dimensional case shows that this loses no generality. Let $z \sim \mathcal{N}(0,1)$ and $x = g(z)$ with a strictly increasing function g. A small interval $\mathrm{d}z$ around z is mapped to an interval of length $g'(z)\,\mathrm{d}z$ around x, and the probability $p(z)\,\mathrm{d}z$ that it carries is conserved, so the density of x is

$$p_x(x) = \frac{p(z)}{g'(z)}\Big|_{z=g^{-1}(x)}. \tag{3.14}$$

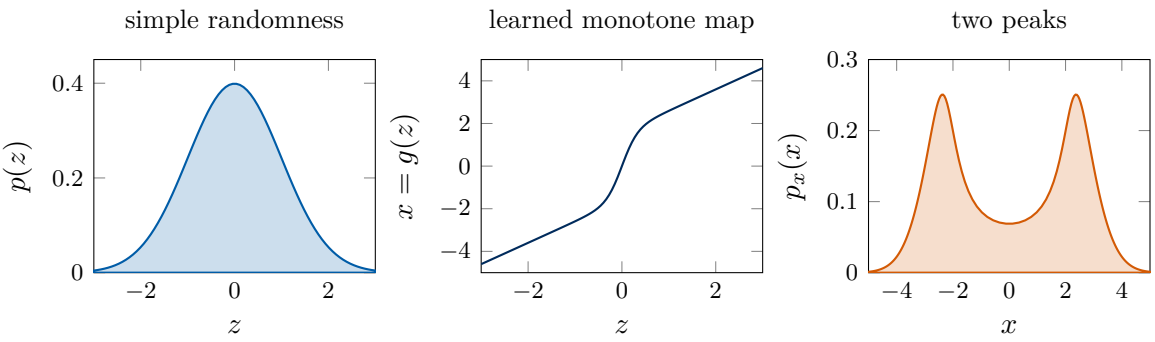


Figure 19: Change of variables, equation (3.14). A standard Gaussian z (left) is passed through the increasing function $g(z) = z + 1.6\tanh(3z)$ (middle), which is steep near $z = 0$. The steep part spreads the central probability thinly, and the density of $x = g(z)$ (right) has two peaks. The functions are chosen for illustration.

Where g is steep, the probability is spread thin and p_x is low. Where g is flat, it is compressed and p_x is high. Figure 19 uses a function that is steep near $z = 0$, and the single peak of the Gaussian becomes two peaks. Any target density with cumulative distribution F, equation (2.6), can be reached in this way. The choice

$$g(z) = F^{-1}(\Phi(z)), \qquad \Phi(z) \coloneqq \int_{-\infty}^{z} \mathcal{N}(z'; 0, 1)\, \mathrm{d}z', \tag{3.15}$$

maps the Gaussian onto the target without error, because $\Phi(z)$ is uniformly distributed on $(0, 1)$ and F^{-1} turns a uniform number into a sample with cumulative distribution F, as in equation (2.7). A flexible enough g can therefore turn a standard Gaussian into any distribution.

In many dimensions the same idea reads as follows. A latent vector $\mathbf{z} \in \mathbb{R}^d$ is drawn from $\mathcal{N}(0, I)$ and a network $g_{\boldsymbol{\theta}}$ maps it to a point $g_{\boldsymbol{\theta}}(\mathbf{z})$ in data space. The points $g_{\boldsymbol{\theta}}(\mathbf{z})$ form a d-dimensional surface in $\mathbb{R}^D$, and the network bends this surface onto the data manifold. A surface has zero volume in $\mathbb{R}^D$, and a density concentrated on it would assign zero density to every real sample that lies slightly off it. The network output is therefore taken as the centre of a narrow distribution, for example a Gaussian of width σ_x or a product of Bernoulli pixels with probabilities $g_{\boldsymbol{\theta}}(\mathbf{z})$. This conditional distribution $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$ gives every $\mathbf{x}$ a nonzero density, and the model of the data is the marginal

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, \mathrm{d}\mathbf{z}, \tag{3.16}$$

the form announced in equation (1.1).

3.8 Learning a distribution, from the coin to images

Equation (3.16) is one of several ways to give a neural network a density over high-dimensional data. Autoregressive models [13], normalising flows [14], generative adversarial networks [15] and diffusion models [16] are others, and this chapter follows the latent-variable route only.

The meaning of "learning a distribution" is the same for all these models and for the coin. A family of distributions is chosen, its parameters are fitted by maximum likelihood or by a bound on it, and the fitted model is used to evaluate probabilities or to draw samples. Table 4 collects the cases of this chapter. What changes from row to row is the family, from one number θ for the coin to a million weights of a network.

Table 4: Learning a distribution, from the coin to the variational autoencoder.

model	family	parameters	sampling
coin	Bernoulli (2.1)	θ, one number	heads with probability θ
measurements	Gaussian (2.5)	μ, σ^2	$x = \mu + \sigma\varepsilon$, $\varepsilon \sim \mathcal{N}(0,1)$
groups of points	mixture (3.13)	π_k, $\boldsymbol{\mu}_k$, Σ_k	label k, then its Gaussian
images	latent-variable model (3.16)	network weights $\boldsymbol{\theta}$	$\mathbf{z} \sim \mathcal{N}(0, I)$, then $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$

Table 5: The pieces of a latent-variable model and of the variational autoencoder.

piece	what it is	learned or fixed	parameters
prior $p(\mathbf{z})$	distribution of codes, $\mathcal{N}(0, I)$	chosen before training	none
decoder $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$	network output $g_{\boldsymbol{\theta}}(\mathbf{z})$ sets a Gaussian mean or Bernoulli probabilities	learned	decoder weights $\boldsymbol{\theta}$
model $p_{\boldsymbol{\theta}}(\mathbf{x})$	$\int p_{\boldsymbol{\theta}}(\mathbf{x}\|\mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$, the decoder applied to the prior	implied, never written down	through $\boldsymbol{\theta}$
posterior $p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})$	codes that could have produced $\mathbf{x}$, from Bayes' theorem	implied, intractable	through $\boldsymbol{\theta}$
encoder $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$	Gaussian cloud approximating the posterior, used only in training	learned	encoder weights $\boldsymbol{\phi}$

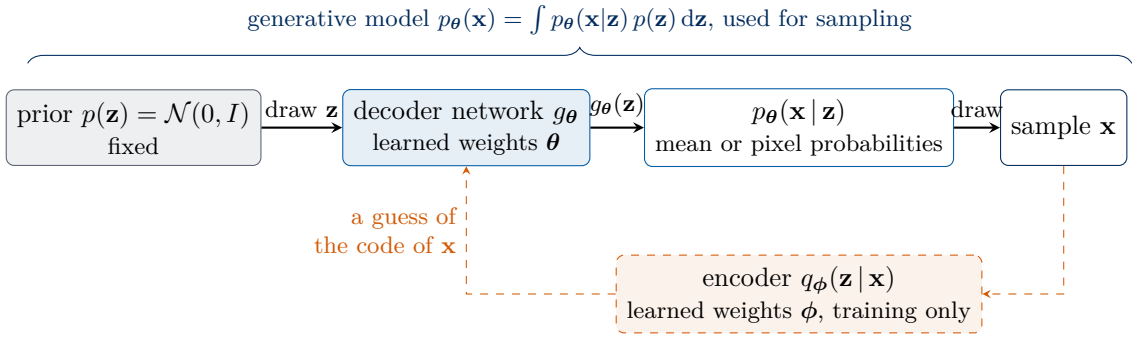


Figure 20: The generative model and its training aid. Sampling runs along the top row, from a fixed prior through the learned decoder to a sample. The encoder (dashed) maps a training sample to the codes that could have produced it and is used only during training.

4 Latent-variable models

4.1 What is modelled and what is fixed

Equation (3.16) contains several distributions, and only some of them are learned. Table 5 lists them, and Figure 20 shows how they connect. The prior $p(\mathbf{z}) = \mathcal{N}(0, I)$ is chosen before training and never changes. The decoder is a network with weights $\boldsymbol{\theta}$, and its output is a set of parameters of a distribution over $\mathbf{x}$, the Gaussian means of equation (3.11) or the pixel probabilities of equation (3.12). The density $p_{\boldsymbol{\theta}}(\mathbf{x})$ of the data is never written down. It is implied by the prior and the decoder through the integral (3.16), and sampling from it takes two steps, $\mathbf{z}$ from the prior and then $\mathbf{x}$ from the decoder distribution. A fourth distribution, the encoder $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$ with weights $\boldsymbol{\phi}$, will be introduced in Section 6. It approximates the posterior $p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})$, the codes that could have produced $\mathbf{x}$, and it is needed only to train the

decoder. It is not part of the generative model, and drawing new samples does not use it, although it serves to encode real images after training.

Fixing the prior costs little in principle. By the change of variables of Section 3.7, a sufficiently flexible decoder can bend a standard Gaussian into whatever shape the data require, so a learned prior would only shift work from the decoder to the prior. The argument is exact in one dimension and for an ideal decoder. A network of finite size, trained for a finite time, approximates such a map, and learned priors can then improve a model in practice.

4.2 A curve with one hidden number

The sine curve of Table 1 is a test bed for a latent-variable model in which every quantity can be checked. One sample is a point of the plane,

$$\mathbf{x} = (t,\ \sin 2t) + \boldsymbol{\eta}, \qquad t \sim \mathcal{U}(-2, 2), \qquad \boldsymbol{\eta} \sim \mathcal{N}(0, s^2 I), \quad s = 0.05, \tag{4.1}$$

where $\mathcal{U}(-2, 2)$ is the uniform distribution on the interval and $\boldsymbol{\eta}$ is a small Gaussian noise. The data set has 4000 such points, and the hidden position t of each is stored for evaluation only. A model with a single latent number, $d = 1$, has the right number of hidden causes. Its decoder is Gaussian, $p_{\boldsymbol{\theta}}(\mathbf{x} \mid z) = \mathcal{N}(\mathbf{x}; g_{\boldsymbol{\theta}}(z), s^2 I)$, with the width fixed to the true noise level. The model is trained and compared with the exact density in Section 7.5.

4.3 A continuous mixture

Equation (3.16) is a mixture model with a continuous label. Replacing the integral over z by a sum over a fine grid of latent values z_r with spacing Δz gives

$$p_{\boldsymbol{\theta}}(\mathbf{x}) \approx \sum_r \underbrace{p(z_r)\,\Delta z}_{\text{weight } \pi_r}\ \mathcal{N}(\mathbf{x}; g_{\boldsymbol{\theta}}(z_r), s^2 I), \tag{4.2}$$

a mixture (3.13) of small Gaussian bumps of width s centred on points of the decoded curve, weighted by the prior probability of their latent interval.

4.4 Why the integral is intractable

Training should maximise the log-likelihood $\ln p_{\boldsymbol{\theta}}(\mathbf{x}_i)$ of every sample, and the integral (3.16) must be evaluated to do so. For one latent dimension a grid suffices, as in equation (4.2). For $d = 16$ a grid with only ten values per coordinate has 10^{16} points, each requiring a pass through the decoder. The remaining option is to estimate the integral as an expectation over the prior,

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim p(\mathbf{z})}\left[p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})\right] \approx \frac{1}{K}\sum_{k=1}^{K} p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z}_k), \qquad \mathbf{z}_k \sim \mathcal{N}(0, I), \tag{4.3}$$

and this estimate fails for a different reason. A digit is explained by a small region of codes, the region where the posterior of Figure 12 lives. Outside it the decoder produces other digits, and $p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$, a product of 784 pixel probabilities, is smaller by many orders of magnitude. Almost every code drawn from the prior misses the region, its term in the sum is negligible, and the estimate is dominated by the rare draw that happens to hit it (Figure 21). If the region holds a fraction 10^{-6} of the prior probability, an illustrative value, about a million draws are needed for a single useful term, and this for every sample and every training step. The posterior is intractable for the same reason, since Bayes' theorem (2.19) divides by $p_{\boldsymbol{\theta}}(\mathbf{x})$.

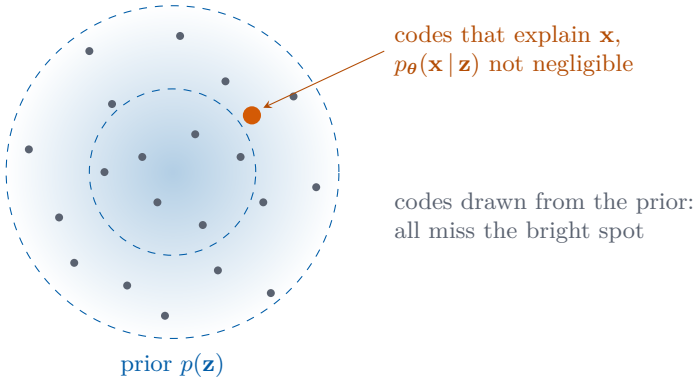


Figure 21: Why sampling the prior does not estimate $p_{\boldsymbol{\theta}}(\mathbf{x})$. Only codes in a small region (orange) decode to something close to the observed sample $\mathbf{x}$. Codes drawn from the prior (grey) almost never fall into it, so the average (4.3) is dominated by rare hits.

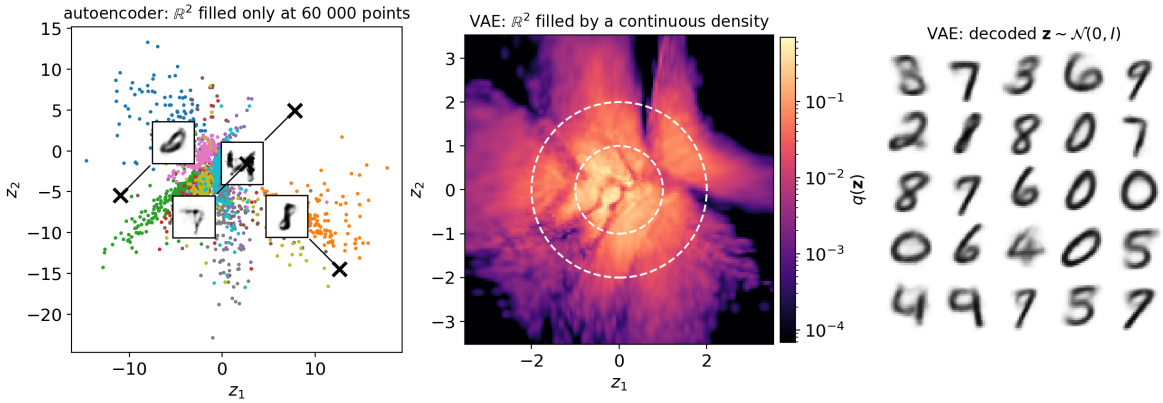


Figure 22: Codes of an autoencoder and of a variational autoencoder, both with $d = 2$, trained on the MNIST training digits without labels. Left, the codes of 2000 test digits of the autoencoder (the panel title counts the 60 000 training codes at which the decoder was trained), coloured by digit, with four latent points far from all training codes (crosses) and the images decoded there. Middle, the density of all encoder clouds of the variational autoencoder, on the logarithmic colour scale at its side, with the circles of radius 1 and 2 of the prior $\mathcal{N}(0, I)$. The density is continuous, with a few dark wedges where it falls well under the prior. Right, 25 images decoded from codes drawn from the prior.

5 From autoencoder points to variational clouds

5.1 Two requirements for generation

Generation requires (a) a known density $p(\mathbf{z})$ to draw codes from, and (b) a decoder that produces a plausible sample for every code this density can produce, including codes between those of the training samples. The autoencoder of Lecture 5 meets neither. Its codes lie wherever the training placed them, and its decoder was trained only at the codes of the training samples.

The left panel of Figure 22 shows the codes of an autoencoder with $d = 2$ trained on MNIST. They spread to values of about 20, the digits form arms with empty wedges between them, and latent points chosen in the wedges (crosses) decode to images that are not digits. The right panel shows what the variational autoencoder of the next sections produces with the

same data and the same latent dimension.

5.2 Clouds instead of points

The variational autoencoder meets both requirements with two changes to the autoencoder. The first change concerns the encoder. For an input $\mathbf{x}$ it returns a cloud of codes instead of one code, a Gaussian

$$q_{\boldsymbol{\phi}}(\mathbf{z}\,|\,\mathbf{x}) \coloneqq \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x})) \tag{5.1}$$

with a centre $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x})$ and a width $\sigma_{\boldsymbol{\phi},k}(\mathbf{x})$ along each latent axis k, both computed by the encoder network. A Gaussian with diagonal covariance is the simplest cloud that has a centre and a width, it is described by $2d$ numbers, and its divergence from the prior has a closed form. During training the decoder receives a random code from this cloud, a different one at each step. Every code near $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x})$ must therefore decode to something close to $\mathbf{x}$, and the decoder is trained on whole neighbourhoods. Where the clouds of neighbouring samples overlap, the decoder learns to produce intermediate images, and the gaps between training codes are filled.

Clouds alone do not meet requirement (a). The reconstruction loss prefers narrow clouds far apart from each other, since then the code identifies the sample without ambiguity, and the clouds would shrink back to points. The second change adds a penalty on the difference between each cloud and the prior $\mathcal{N}(0, I)$, measured by the divergence $\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(0, I)\right)$. It vanishes only for a cloud with centre zero and unit widths. It grows when a centre moves away from the origin, which pulls the clouds towards the origin, and when a width shrinks under one, which keeps them wide. Under the two pressures the clouds overlap and together fill the region where the prior has its probability. Codes drawn from the prior then land where the decoder has been trained. Figure 23 draws the two situations side by side, and the middle panel of Figure 22 shows the result for the digits. The sum of the reconstruction loss and this penalty turns out to be the negative of a lower bound on $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$.

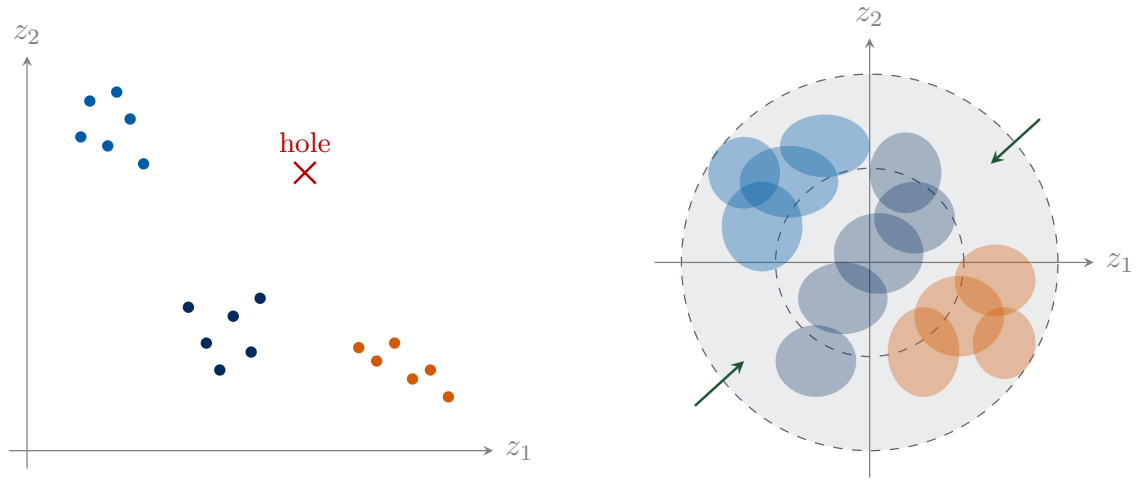
6 The evidence lower bound

6.1 A spotlight on the relevant codes

The aim of training is a large log-likelihood $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$ for every training sample, so that the model makes the observed data probable. Section 4.4 showed that this quantity cannot be computed by averaging the decoder likelihood over codes drawn from the prior, because almost all such codes contribute nothing. The codes that matter for a given $\mathbf{x}$ lie in a small region, and if that region were known in advance, the codes could be drawn there. Suppose a second distribution $q(\mathbf{z}\,|\,\mathbf{x})$ is available that points to this region, a guess of which codes could have produced $\mathbf{x}$. In the variational autoencoder this distribution is the encoder cloud of Section 5.2. Multiplying and dividing by it inside the integral (3.16) changes nothing,

$$p_{\boldsymbol{\theta}}(\mathbf{x}) = \int q(\mathbf{z}\,|\,\mathbf{x})\, \frac{p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}\, \mathrm{d}\mathbf{z} = \mathbb{E}_{\mathbf{z}\sim q(\mathbf{z}|\mathbf{x})}\big[w(\mathbf{z})\big], \qquad w(\mathbf{z}) \coloneqq \frac{p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}, \tag{6.1}$$

but the expectation is now taken over codes drawn from q. Every draw falls into the region that explains $\mathbf{x}$, and the weight $w(\mathbf{z})$ corrects for having drawn from q instead of from the prior. Figure 24 draws the idea as a spotlight. This construction is known as importance sampling. Its precision depends on how well q covers the region of the posterior, and the best possible guess is the posterior itself, for which $w(\mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x})$ is the same number for every draw.



autoencoder: one point per sample, no known density, holes between

variational autoencoder: one cloud per sample, KL pulls clouds into $\mathcal{N}(0, I)$ (grey disc)

Figure 23: The change from the autoencoder to the variational autoencoder. Left, the autoencoder maps each sample to one point, the points of different digits (colours) form separate groups, and a code in a hole between them was never decoded during training. Right, the variational autoencoder maps each sample to a Gaussian cloud $q_\phi(\mathbf{z} \mid \mathbf{x})$ with diagonal covariance. The decoder is trained on random points of every cloud, and the KL penalty (green arrows) pulls the clouds towards the prior and keeps them wide, so that together they fill the disc where the prior puts its probability.

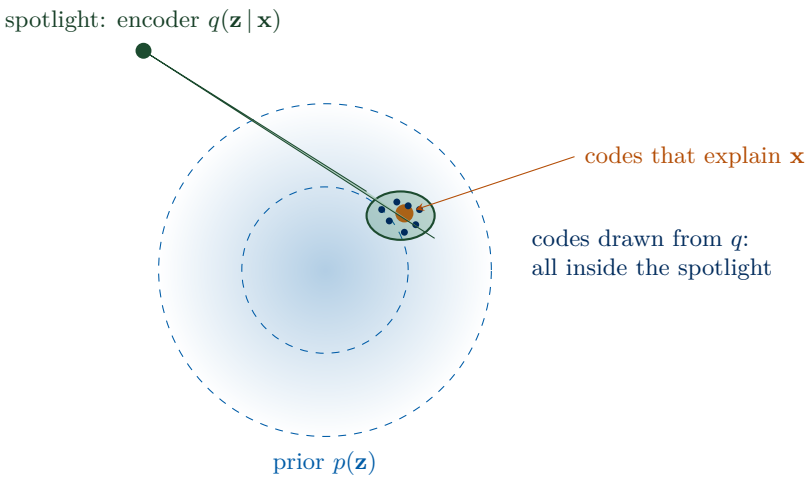


Figure 24: Importance sampling, equation (6.1). Codes drawn from a distribution $q(\mathbf{z} \mid \mathbf{x})$ that points to the region explaining $\mathbf{x}$ (green spotlight) all contribute, in contrast with the codes drawn from the prior in Figure 21. The weights $w(\mathbf{z})$ correct for the change of distribution.

6.2 A bound that is never too high

The logarithm of the expectation (6.1) is still hard to compute, because the weights $w(\mathbf{z})$ are products of hundreds of pixel probabilities and vary over many orders of magnitude, so the expectation is dominated by rare draws with enormous weights. An example shows the consequence. If $\ln w$ is Gaussian with standard deviation 4 around zero, the expectation is $\mathbb{E}[w] = e^8$, but half of this value comes from draws with $\ln w$ above 16, which occur with probability $3 \cdot 10^{-5}$. An average of a few draws almost never contains such a draw, and the logarithm of the average falls systematically below $\ln \mathbb{E}[w] = 8$, by an amount that changes from one set of draws to the next. Such an estimate is called biased, because its average over many repetitions differs from the quantity it estimates. The average of the logarithms, $\mathbb{E}_q[\ln w(\mathbf{z})]$, behaves differently. In the example it equals zero, and the average of $\ln w$ over a few draws scatters around zero with standard deviation $4/\sqrt{K}$ for K draws, without any systematic error, so it is an unbiased estimate. It is a sum of ordinary numbers, the log-probabilities of the pixels, and its gradient with respect to the weights of both networks is equally unbiased. The price is that it lies below $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$, and Section 6.3 shows that the distance is a divergence that training can shrink. Averaging the logarithms gives a bound, and the size of its error has a meaning of its own, the gap of Section 6.3. Jensen's inequality, Theorem 3.3, compares the two quantities. Since the logarithm is concave, the average of the logarithms is smaller than the logarithm of the average (Figure 14). Applied to equation (6.1),

$$\ln p_{\boldsymbol{\theta}}(\mathbf{x}) = \ln \mathbb{E}_q\big[w(\mathbf{z})\big] \;\geq\; \mathbb{E}_q\big[\ln w(\mathbf{z})\big] = \mathbb{E}_q\Big[\ln \frac{p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}\Big]. \tag{6.2}$$

The right-hand side is an average of logarithms, which can be estimated from a few codes drawn from q, and it is never larger than the log-likelihood. It is called the evidence lower bound, after the evidence $p_{\boldsymbol{\theta}}(\mathbf{x})$ of Table 3.

Splitting the logarithm of the product inside the expectation separates the bound into two terms,

$$\mathrm{ELBO}(\mathbf{x}) = \mathbb{E}_q\Big[\ln p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\Big] + \mathbb{E}_q\Big[\ln \frac{p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}\Big] = \underbrace{\mathbb{E}_{\mathbf{z}\sim q(\mathbf{z}|\mathbf{x})}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\big]}_{\text{reconstruction}} - \underbrace{\mathrm{KL}\left(q(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p(\mathbf{z})\right)}_{\text{distance from the prior}}. \tag{6.3}$$

The first term is the expected log-likelihood of $\mathbf{x}$ when its code is drawn from the spotlight, the reconstruction term. The second term is the divergence (3.9) of the spotlight from the prior. These are the two terms of Section 5.2, the reconstruction loss and the penalty that pulls the clouds towards $\mathcal{N}(0, I)$.

6.3 The size of the gap

Jensen's inequality states that the bound cannot exceed the log-likelihood, and an exact identity states by how much. For every code $\mathbf{z}$, Bayes' theorem gives $p_{\boldsymbol{\theta}}(\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z}) / p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x})$. The left-hand side does not depend on $\mathbf{z}$, so its logarithm equals its own average over q, and multiplying and dividing by q inside the logarithm gives

$$\ln p_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_q\Big[\ln \frac{p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z})}{p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x})}\Big] = \mathbb{E}_q\Big[\ln \frac{p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\, p(\mathbf{z})}{q(\mathbf{z}\,|\,\mathbf{x})}\Big] + \mathbb{E}_q\Big[\ln \frac{q(\mathbf{z}\,|\,\mathbf{x})}{p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x})}\Big]. \tag{6.4}$$

The first expectation is the bound (6.2), and the second is a Kullback–Leibler divergence,

$$\ln p_{\boldsymbol{\theta}}(\mathbf{x}) = \mathrm{ELBO}(\mathbf{x}) + \mathrm{KL}\left(q(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x})\right). \tag{6.5}$$

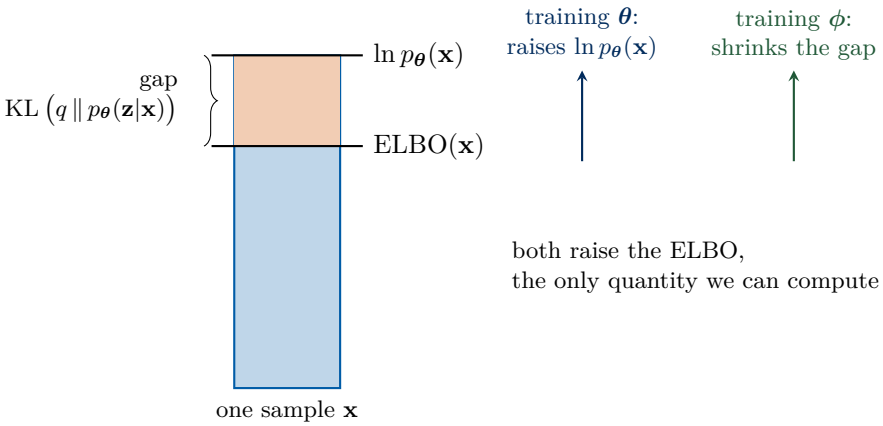


Figure 25: Equation (6.5) as a bar. The log-likelihood of a sample (top line) is the evidence lower bound plus a gap equal to the divergence of the encoder cloud from the true posterior. Training raises the bound, by improving the model through $\boldsymbol{\theta}$ and by tightening the gap through $\boldsymbol{\phi}$.

The amount by which the bound falls short is the divergence of the spotlight from the true posterior, how badly the guess misses the region of codes that explain $\mathbf{x}$. Since a divergence is never negative, equation (6.5) reproduces the inequality (6.2), and it adds that the bound is reached when q equals the posterior.

Definition 6.1 (Evidence lower bound)**.** For a latent-variable model $p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) = p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, \mathbf{z})\, p(\mathbf{z})$ and any distribution $q(\mathbf{z} \,|\, \mathbf{x})$, the evidence lower bound is

$$\mathrm{ELBO}(\mathbf{x}) \coloneqq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, \mathbf{z})\big] - \mathrm{KL}\left(q(\mathbf{z} \,|\, \mathbf{x}) \,\|\, p(\mathbf{z})\right) \;\leq\; \ln p_{\boldsymbol{\theta}}(\mathbf{x}), \tag{6.6}$$

with the gap $\mathrm{KL}\left(q(\mathbf{z}|\mathbf{x}) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{z}|\mathbf{x})\right)$.

Figure 25 draws equation (6.5) as a bar. The log-likelihood depends only on the decoder parameters $\boldsymbol{\theta}$. The gap depends on both the decoder, through the true posterior, and on the guess q. In the variational autoencoder q is the encoder cloud (5.1) with parameters $\boldsymbol{\phi}$, and the bound is raised with respect to both sets of parameters. Raising it with respect to $\boldsymbol{\phi}$ leaves the log-likelihood unchanged, so it can only shrink the gap, and it makes the encoder a better approximation of the posterior. Raising it with respect to $\boldsymbol{\theta}$ changes both the log-likelihood and the gap, since the true posterior depends on the decoder, and a step in $\boldsymbol{\theta}$ that raises the bound is not guaranteed to raise the log-likelihood. The bound is therefore a surrogate objective. It is the quantity that can be computed and optimised, and the log-likelihood lies above it by the gap, which the encoder keeps small. One objective trains both networks.

A physicist meets the same structure in the variational principle of quantum mechanics, which serves here as an analogy. For any normalised trial state $|\psi\rangle$ the expectation value of the Hamiltonian is never smaller than the ground-state energy, $\langle\psi|\hat{H}|\psi\rangle \geq E_0$. The trial state is chosen from a family with adjustable parameters, the best member gives the lowest bound, and the difference measures how far the trial state is from the ground state. In the variational autoencoder the family of trial distributions is the family of Gaussian clouds, the bound is smaller than $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$ and is maximised, and the gap is the divergence of the trial distribution from the posterior. The word variational in the name refers to this choice of the best member of a family of approximations.

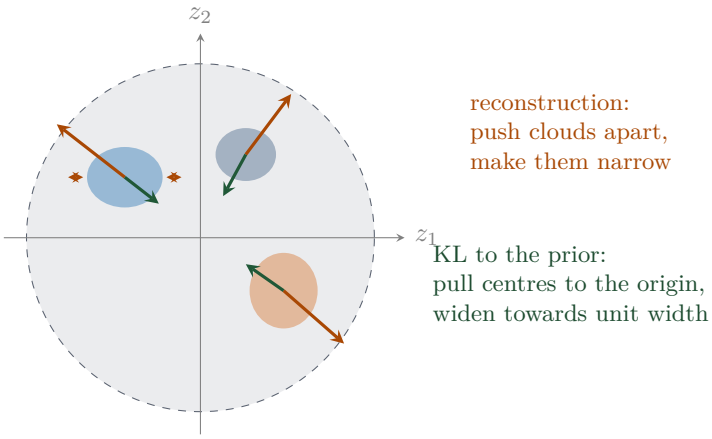


Figure 26: The two terms of the evidence lower bound as forces on the encoder clouds. The reconstruction term (orange) pushes the clouds of different samples apart and shrinks them. The divergence from the prior (green) pulls them towards the origin and widens them. The maximum of the bound balances the two.

Table 6: The evidence lower bound of a model with two codes, for three guesses $q(z)$ of the posterior. The evidence is $\ln p(\mathbf{x}) = -0.799$.

guess $q(z=1)$, $q(z=2)$	weights $w(1)$, $w(2)$	bound $\mathbb{E}_q[\ln w]$	gap	$\mathbb{E}_q[\ln p(\mathbf{x}\|z)]$	$\mathrm{KL}(q \,\|\, p(z))$
0.5, 0.5 (the prior)	0.80, 0.10	-1.263	0.464	-1.263	0.000
0.9, 0.1	0.44, 0.50	-0.799	0.0006	-0.431	0.368
0.889, 0.111 (true posterior)	0.45, 0.45	-0.799	0	-0.454	0.344

6.4 Two codes with numbers

A model with only two codes makes every quantity of the last three subsections a number. Let z take the values 1 and 2 with the prior probabilities $p(z) = \frac{1}{2}$ each, and let the decoder give an observed image $\mathbf{x}$ the probabilities $p(\mathbf{x} \,|\, z = 1) = 0.8$ and $p(\mathbf{x} \,|\, z = 2) = 0.1$. The integral (3.16) becomes a sum of two terms, and the evidence is

$$p(\mathbf{x}) = 0.8 \cdot 0.5 + 0.1 \cdot 0.5 = 0.45, \qquad \ln p(\mathbf{x}) = -0.799.$$

Bayes' theorem gives the true posterior $p(z = 1 \,|\, \mathbf{x}) = 0.40/0.45 = 0.889$ and $p(z = 2 \,|\, \mathbf{x}) = 0.111$. Code 1 explains the image far better. Table 6 evaluates the bound for three guesses q. For every guess the weights $w(z) = p(\mathbf{x}|z)\, p(z)/q(z)$ average to the evidence, $\mathbb{E}_q[w] = 0.45$, as equation (6.1) requires. The average of their logarithms is the bound, and it lies below $\ln p(\mathbf{x})$ by exactly the divergence of the guess from the true posterior. A guess that ignores the image, equal to the prior, misses by 0.464 nats. A guess that puts most weight on code 1 misses by 0.0006 nats, and the true posterior itself makes all weights equal, so that averaging and logarithm commute and the bound is exact. The last two columns show the same bound as reconstruction minus divergence from the prior, equation (6.6). The guess that follows the image pays a divergence of 0.37 nats, because it differs from the prior, and gains more in reconstruction.

6.5 Two terms as a bargain

Figure 26 draws the two terms of equation (6.3) as the two pressures of Section 5.2. Acting alone, the reconstruction term shrinks the clouds into separated points, as in an autoencoder,

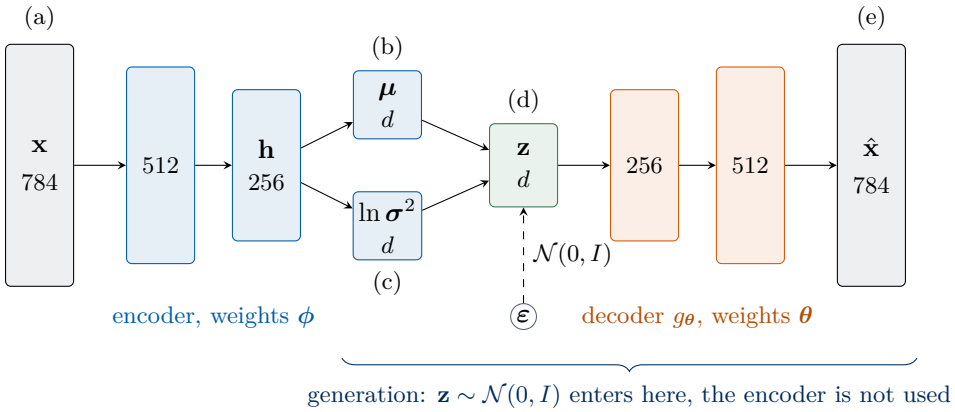


Figure 27: The variational autoencoder for the digits, equations (7.1). The encoder (blue) maps an image to the centre $\boldsymbol{\mu}$ and the log-variances $\ln \boldsymbol{\sigma}^2$ of its cloud. A code $\mathbf{z}$ is formed from them and an external noise $\boldsymbol{\varepsilon}$ (green), and the decoder (orange) maps it to the pixel probabilities $\hat{\mathbf{x}}$. Labels (a) to (e) refer to the lines of equation (7.1), and the numbers give the layer widths.

and the divergence makes every cloud equal to the prior. The bound is maximised by clouds only as narrow as the reconstruction needs, packed together inside the prior. The bargain reads, explain $\mathbf{x}$ well from the codes the encoder picks, and do not pick codes that the prior considers implausible.

7 The variational autoencoder

7.1 Architecture

The variational autoencoder [11, 12] turns the evidence lower bound into a network that can be trained by gradient descent. Figure 27 shows the network used for the digits, and equations (7.1) follow the flow of information through it,

$$\mathbf{h} = \mathrm{ReLU}\left(W_2\ \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2\right) \in \mathbb{R}^{256}, \tag{7.1a}$$

$$\boldsymbol{\mu} = W_\mu \mathbf{h} + \mathbf{b}_\mu \in \mathbb{R}^d, \tag{7.1b}$$

$$\ln \boldsymbol{\sigma}^2 = W_\sigma \mathbf{h} + \mathbf{b}_\sigma \in \mathbb{R}^d, \tag{7.1c}$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I_d), \tag{7.1d}$$

$$\hat{\mathbf{x}} = \mathrm{sigm}\left(g_{\boldsymbol{\theta}}(\mathbf{z})\right) \in (0,1)^{784}, \tag{7.1e}$$

where $\mathrm{ReLU}(u) = \max(0, u)$ acts on every component, $\odot$ is the product component by component, $\mathrm{sigm}(u) = 1/(1+e^{-u})$ is the logistic function, and $g_{\boldsymbol{\theta}}$ is the decoder network with layers of 256 and 512 units and 784 outputs.

The encoder (7.1a) passes the 784 pixel values through two dense layers to a hidden representation $\mathbf{h}$ of 256 numbers. Two separate linear heads read it. Head (7.1b) gives the centre $\boldsymbol{\mu}$ of the cloud, and head (7.1c) gives the logarithms of the variances. The logarithm can take any real value while the variance $\sigma_k^2 = \exp(\ln \sigma_k^2)$ stays positive, so the network needs no constraint on its output. Together the two heads define the encoder cloud (5.1), a Gaussian with diagonal covariance, described by $2d$ numbers. The two dense layers and the two heads together are called the encoder network f_ϕ, so that $(\boldsymbol{\mu}, \ln \boldsymbol{\sigma}^2) = f_\phi(\mathbf{x})$. Line (7.1d) draws a code from this cloud by shifting and stretching a standard Gaussian noise vector, as explained

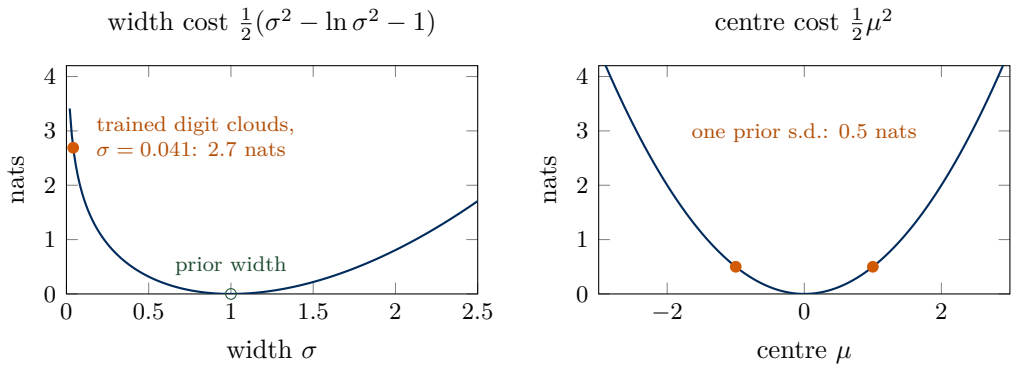


Figure 28: The two parts of the divergence (7.3) of one latent coordinate. Left, the cost of the width, zero at the prior width 1 and growing without bound for narrow clouds. The orange point marks the mean width of the trained digit clouds of the model with $d = 2$. Right, the cost of moving the centre away from the origin.

in Section 7.3. The decoder (7.1e) maps the code through layers of 256 and 512 units to 784 unbounded numbers, called logits, and the logistic function turns them into the probabilities of ink $\hat{x}_j$ that define the Bernoulli decoder $p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, \mathbf{z}) = \prod_j \hat{x}_j^{x_j}(1 - \hat{x}_j)^{1-x_j}$.

The trained quantities are the encoder weights $\boldsymbol{\phi} = (W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_\mu, \mathbf{b}_\mu, W_\sigma, \mathbf{b}_\sigma)$ and the decoder weights $\boldsymbol{\theta}$, for $d = 16$ in total 1 079 600 numbers. Nothing is stored for the individual training samples. The cloud of a sample is computed from the sample whenever it is needed, for training samples and for new ones alike, and one network serves all inputs. This sharing is called amortised inference, since the cost of finding a good approximate posterior is paid once, during training, instead of by a separate optimisation for each sample.

7.2 The divergence of a Gaussian cloud from the prior

The second term of the bound (6.3) has a closed form for a Gaussian cloud and a Gaussian prior. In one dimension, with $q(z) = \mathcal{N}(z; \mu, \sigma^2)$ and $p(z) = \mathcal{N}(z; 0, 1)$, the logarithms of the two densities are

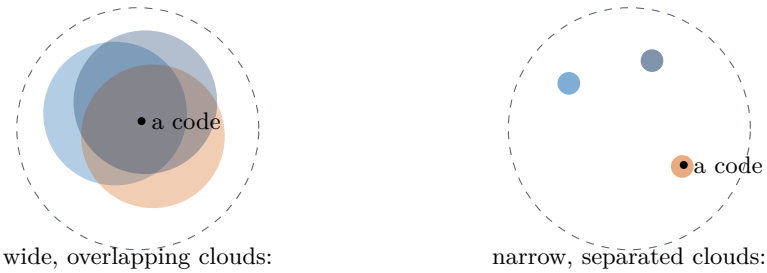
$$\ln q(z) = -\tfrac{1}{2}\ln(2\pi\sigma^2) - \frac{(z-\mu)^2}{2\sigma^2}, \qquad \ln p(z) = -\tfrac{1}{2}\ln(2\pi) - \frac{z^2}{2}, \tag{7.2}$$

and the expectations under q of the two squares are $\mathbb{E}_q[(z-\mu)^2] = \sigma^2$ and $\mathbb{E}_q[z^2] = \mu^2 + \sigma^2$. The divergence is therefore

$$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_q[\ln q - \ln p] = -\tfrac{1}{2}\ln\sigma^2 - \tfrac{1}{2} + \tfrac{1}{2}(\mu^2 + \sigma^2) = \tfrac{1}{2}(\mu^2 + \sigma^2 - \ln\sigma^2 - 1). \tag{7.3}$$

It vanishes only for $\mu = 0$ and $\sigma = 1$. The term μ^2 is the cost of moving the centre away from the origin, and the term $\sigma^2 - \ln\sigma^2 - 1$, which is zero at $\sigma = 1$ and grows without bound as $\sigma \to 0$, is the cost of a sharp cloud. Figure 28 plots the two parts. Moving a centre by one prior standard deviation costs half a nat. For $\sigma \ll 1$ the width term is close to $-\ln\sigma - \frac{1}{2}$, so every narrowing of a cloud by a factor e costs one more nat. The reconstruction pulls the other way, and the trained width is where one more factor e of narrowing gains less than one nat of reconstruction. For the digits with $d = 2$ the mean width is 0.041, which costs about 2.7 nats per coordinate.

For a cloud with diagonal covariance in d dimensions both densities factorise, equation (2.15),



wide, overlapping clouds:
the code does not tell the sample,
little information, small KL

narrow, separated clouds:
the code identifies the sample,
much information, large KL

Figure 29: The divergence as the price of information, equation (7.5). Clouds of three samples (colours) inside the prior (dashed). Left, wide clouds overlap, a code is compatible with all three samples, and the divergence from the prior is small. Right, narrow clouds identify their sample and cost more.

the logarithm of a product is a sum, and the divergence is a sum over the latent coordinates,

$$\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}) \,\|\, \mathcal{N}(0, I)\right) = \sum_{k=1}^{d} \mathrm{KL}_k, \qquad \mathrm{KL}_k := \tfrac{1}{2}(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1). \tag{7.4}$$

Each coordinate pays separately.

The divergence is called the price of information, and a short argument justifies the name. Let the encoder clouds of the n training samples be averaged into the distribution of all codes, $q(\mathbf{z}) = \frac{1}{n}\sum_i q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i)$. Multiplying and dividing by $q(\mathbf{z})$ inside the logarithm of every divergence splits it into $\int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \ln[q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i)/q(\mathbf{z})]\,\mathrm{d}\mathbf{z} + \int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \ln[q(\mathbf{z})/p(\mathbf{z})]\,\mathrm{d}\mathbf{z}$. In the average over the samples the second integral becomes $\int q(\mathbf{z}) \ln[q(\mathbf{z})/p(\mathbf{z})]\,\mathrm{d}\mathbf{z}$, because the average of the clouds is $q(\mathbf{z})$, and the divergence averaged over the samples takes the form

$$\frac{1}{n}\sum_{i=1}^{n} \mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \,\|\, p(\mathbf{z})\right) = \underbrace{\frac{1}{n}\sum_{i=1}^{n} \int q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i) \ln \frac{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_i)}{q(\mathbf{z})}\,\mathrm{d}\mathbf{z}}_{I_q(\mathbf{x};\mathbf{z})} + \mathrm{KL}\left(q(\mathbf{z}) \,\|\, p(\mathbf{z})\right) \;\geq\; I_q(\mathbf{x};\mathbf{z}). \tag{7.5}$$

The first part is the mutual information between a sample and its code, the average of $\ln[q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})/q(\mathbf{z})]$. The mutual information measures by how many nats, on average, knowing the code reduces the uncertainty about which sample produced it. It is zero when the clouds of all samples coincide, so that a code says nothing about the sample, and it grows as the clouds separate. Section 11.5 uses the same quantity for two discrete variables, equation (11.7). The second part is a divergence and cannot be negative. The average divergence is therefore an upper bound on the information that the codes carry about the samples. Figure 29 shows the two extremes. Wide clouds that overlap leave a code ambiguous and carry little information at a low price. Narrow clouds that do not overlap identify the sample and cost much.

7.3 The reparametrisation trick

The reconstruction term $\mathbb{E}_{q_{\boldsymbol{\phi}}}[\ln p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})]$ is estimated by drawing a code from the cloud and evaluating the decoder there. Its gradient with respect to the decoder weights $\boldsymbol{\theta}$ poses no problem. Its gradient with respect to the encoder weights $\boldsymbol{\phi}$ does, because $\boldsymbol{\phi}$ enters only through the distribution from which $\mathbf{z}$ is drawn, and a sampling routine that returns a number drawn from $\mathcal{N}(\mu, \sigma^2)$ does not expose how its output would change if μ or σ changed slightly.

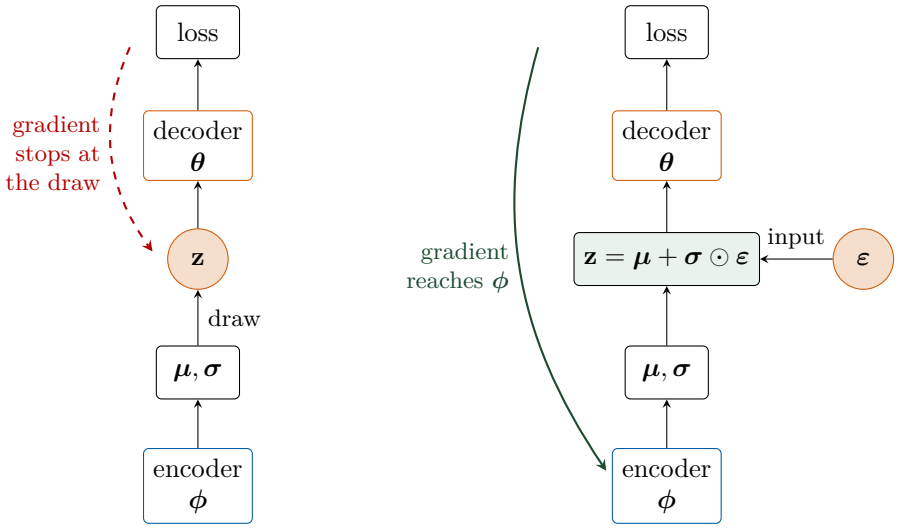


Figure 30: The reparametrisation trick as a computation graph. Left, a code drawn inside the network blocks the gradient of the loss on its way to the encoder. Right, the noise $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, I)$ enters as an external input, the code is a differentiable function of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$, and the gradient flows from the loss through the decoder and the code into the encoder.

Backpropagation needs exactly this dependence, and a sampling node written in this way blocks it (left part of Figure 30).

Line (7.1d) removes the obstacle. A sample of $\mathcal{N}(\mu_k, \sigma_k^2)$ is obtained by drawing ε_k from the standard Gaussian, which does not depend on any weight, and setting $z_k = \mu_k + \sigma_k \varepsilon_k$. The randomness becomes an input that enters from the side, and $\mathbf{z}$ becomes an ordinary differentiable function of the encoder outputs,

$$\frac{\partial z_k}{\partial \mu_k} = 1, \qquad \frac{\partial z_k}{\partial \sigma_k} = \varepsilon_k. \tag{7.6}$$

In terms of expectations, an average over a distribution that depends on $\boldsymbol{\phi}$ is rewritten as an average over a fixed distribution of a function that depends on $\boldsymbol{\phi}$,

$$\mathbb{E}_{\mathbf{z} \sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\big[f(\mathbf{z})\big] = \mathbb{E}_{\boldsymbol{\varepsilon} \sim \mathcal{N}(0,I)}\big[f\big(\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}) + \boldsymbol{\sigma}_{\boldsymbol{\phi}}(\mathbf{x}) \odot \boldsymbol{\varepsilon}\big)\big], \tag{7.7}$$

and the gradient can be taken inside the second expectation.

Figure 31 shows what the decoder must learn for one input. The encoder places the cloud of a test digit 0 in the latent plane of a model with $d = 2$, eight noise vectors give eight codes inside it, and all eight decode to versions of the same 0. During training one such code per input and step is decoded and compared with the input.

7.4 The loss

The loss of the variational autoencoder is the negative evidence lower bound, averaged over a mini-batch of B samples $\mathbf{x}_b$, $b = 1, \ldots, B$. The expectation over the cloud is estimated with a single code per sample and step, $\mathbf{z}_b = \boldsymbol{\mu}_b + \boldsymbol{\sigma}_b \odot \boldsymbol{\varepsilon}_b$, and the reconstruction term of the Bernoulli decoder is the binary cross-entropy (3.12). Written out as the sums that the

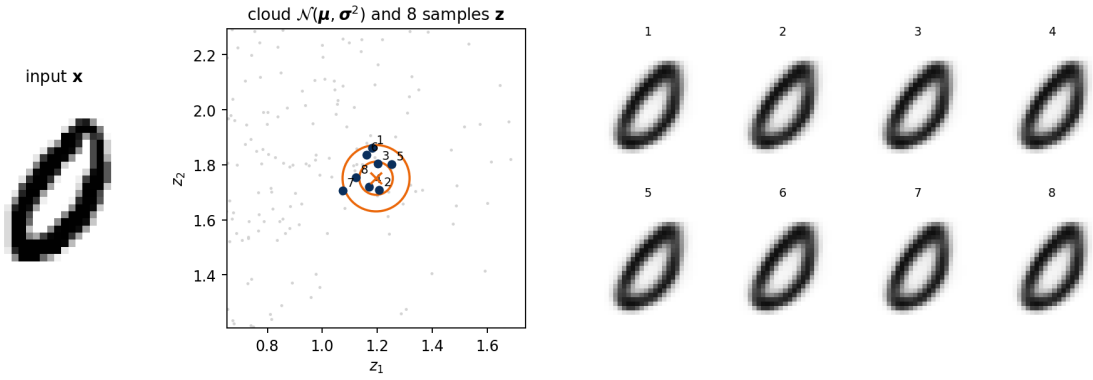


Figure 31: Samples from the cloud of one input, for a variational autoencoder with $d = 2$ trained on MNIST. Left, a test digit. Middle, its cloud $\mathcal{N}(\boldsymbol{\mu}, \operatorname{diag} \boldsymbol{\sigma}^2)$ with the centre (cross), the one- and two-standard-deviation ellipses (orange), the centres of other test digits (grey) and eight codes $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}$ (dots). Right, the images decoded from the eight codes, all versions of the input.

program computes, over the batch, the pixels j and the latent coordinates k,

$$\mathcal{L}(\boldsymbol{\phi}, \boldsymbol{\theta}) = \frac{1}{B} \sum_{b=1}^{B} \Bigg[\underbrace{- \sum_{j=1}^{D} \Big(x_{bj} \ln \hat{x}_{bj} + (1 - x_{bj}) \ln \big(1 - \hat{x}_{bj}\big) \Big)}_{\text{reconstruction, } -\ln p_{\boldsymbol{\theta}}(\mathbf{x}_b | \mathbf{z}_b)} + \underbrace{\frac{1}{2} \sum_{k=1}^{d} \Big(\mu_{bk}^2 + \sigma_{bk}^2 - \ln \sigma_{bk}^2 - 1 \Big)}_{\mathrm{KL}(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x}_b) \,\|\, \mathcal{N}(0,I))} \Bigg], \tag{7.8}$$

with $\hat{\mathbf{x}}_b = \operatorname{sigm}\big(g_{\boldsymbol{\theta}}(\mathbf{z}_b)\big)$.

The two networks learn together, and neither needs to be told the correct code of any image. At the start both have random weights. The encoder sends a mini-batch of images to clouds at arbitrary places, a code is drawn from every cloud, and the decoder turns the codes into nearly uniform grey images. The loss (7.8) is computed, and one backward pass gives the gradient with respect to all weights of both networks, through the reparametrised codes. The decoder weights move so that the codes it receives rebuild their images better, and the encoder weights move so that each image gets a code from which the current decoder rebuilds it better and a cloud closer to the prior. After many steps the encoder places similar images at nearby codes, because the decoder can then serve them with similar outputs, and the decoder has learned what the codes mean. The meaning of the code is not given in advance. It is whatever arrangement lets the two networks lower the loss together.

For a Gaussian decoder of width σ_x the reconstruction term is replaced by $\sum_j (x_{bj} - \hat{x}_{bj})^2 / 2\sigma_x^2$, equation (3.11). The width then fixes the exchange rate between the two terms. Halving σ_x makes every reconstruction error four times more expensive relative to the divergence, and σ_x therefore acts like an inverse weight of the divergence term. A single code per sample gives a noisy estimate of the expected reconstruction, but the noise averages out over the many steps of training. The inequality $\mathrm{ELBO}(\mathbf{x}) \leq \ln p_{\boldsymbol{\theta}}(\mathbf{x})$ holds for the expectation over the cloud. A single-code estimate of the loss is a random number whose average is the negative bound, and an individual estimate can lie on either side of it. One gradient step evaluates the encoder, draws B noise vectors, evaluates the decoder, computes equation (7.8), and lets backpropagation update $\boldsymbol{\phi}$ and $\boldsymbol{\theta}$ together.

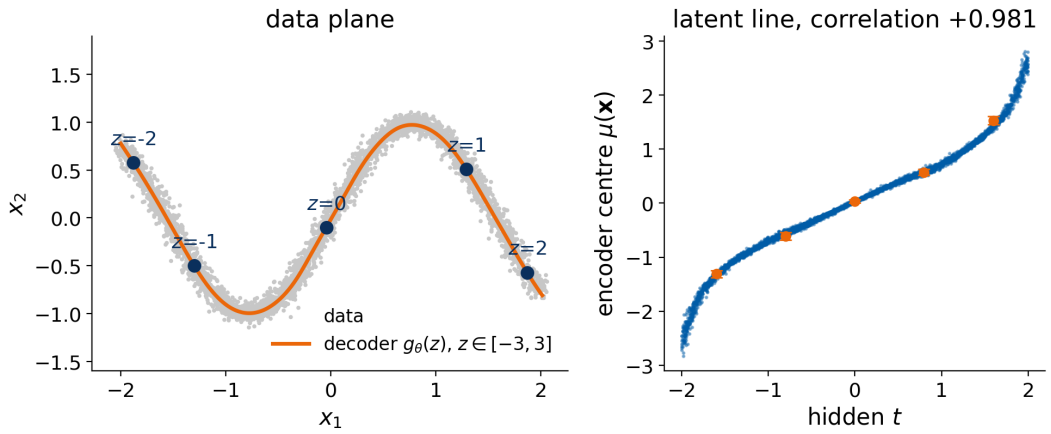


Figure 32: The sine curve with one latent number. Left, the 4000 data points (grey) and the decoder mean $g_{\boldsymbol{\theta}}(z)$ for z from -3 to 3 (orange), with five latent values marked. Right, the centre $\mu(\mathbf{x})$ of the encoder cloud of every point against the hidden position t. The relation is monotone and S-shaped.

Table 7: The trained model with one latent number on the sine curve. Correlations and widths refer to the 4000 training points, likelihoods to 2000 fresh points.

quantity	value
rank correlation of the encoder centre with t	0.9996
linear correlation of the encoder centre with t	0.981
range of the encoder centres	-2.83 to 2.81
median encoder width	0.028
largest deviation of $T(z)$ from $4\Phi(z) - 2$, $\lvert z\rvert \le 2.5$	0.12
generated points with $x_1 \in [-2, 2]$	99.2%
mean evidence lower bound (nats per point)	-0.552
mean log-likelihood (nats per point)	-0.401
average gap (nats per point)	0.151

Table 8: Mean log-likelihood of 2000 fresh points of the sine curve under four models, in nats per point. The integral over z of the latent-variable model is computed numerically, equation (4.2). The mixture of 30 Gaussians works here in two dimensions, the route that Section 3.6 ruled out for 784.

model	mean $\ln p(\mathbf{x})$
mixture of 3 Gaussians	-0.850
mixture of 30 Gaussians	-0.388
latent-variable model, $d = 1$	-0.401
true process	-0.360

7.5 The sine curve with a trained model

The model with one latent number of Section 4.2 is trained with the loss (7.8) and a Gaussian decoder on the 4000 points of the curve, with the settings of Table 27. The encoder centres of the training points are ordered like t, along an S-shaped curve (Figure 32 and Table 7).

New points are generated by drawing $z \sim \mathcal{N}(0, 1)$ and decoding it. Their horizontal positions are uniform like those of the data only if the decoder solves the problem of Section 3.7. Its

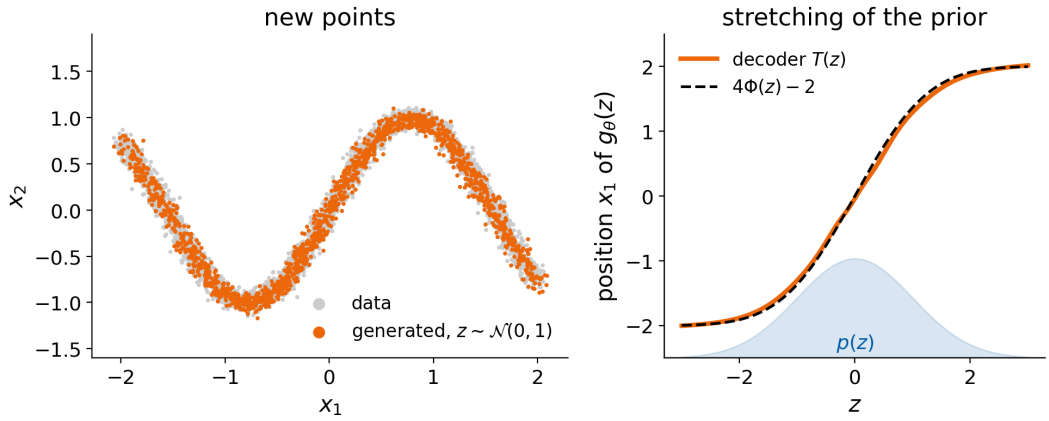


Figure 33: Generation on the sine curve. Left, data (grey) and points generated from $z \sim \mathcal{N}(0, 1)$ (orange). Right, the horizontal position $T(z)$ of the decoder mean (orange) and the function $4\Phi(z) - 2$ of equation (7.9) (dashed), with the prior density shaded. The decoder stretches the centre of the prior and compresses its tails so that the generated positions are uniform.

horizontal coordinate $T(z)$ must satisfy the inverse-distribution relation (3.15) for the uniform target on $[-2, 2]$,

$$T(z) = 4\,\Phi(z) - 2. \tag{7.9}$$

Nothing in the training states that t is uniform. The decoder nevertheless finds this function closely (Figure 33). The fixed Gaussian prior is turned into the uniform distribution of the data by the decoder, as the change-of-variables argument of Section 3.7 predicts. The encoder and the decoder are inverse maps along the curve, $T(\mu(t)) \approx t$, as the two panels of Figures 32 and 33 show from both sides.

For $d = 1$ the sum (4.2) can be evaluated, and the log-likelihood of fresh points measures how well each model describes the curve (Table 8). The true process of equation (4.1) gives the best value any model can reach on average. Three Gaussians cannot follow the curve. A mixture of 30 Gaussians and the trained model come close to the true value. The difference between the last two lies in the meaning of the hidden variable. The 30 component labels are an unordered list, while the latent z is a coordinate along the curve that is defined also between the training points.

The same sum gives both sides of equation (6.5), the log-likelihood and the bound computed with the trained encoder. On fresh points the bound is smaller than the log-likelihood for every point, and the average gap is 0.151 nats (Table 7). The gap measures how far the Gaussian clouds of the encoder are from the exact posteriors, which for a curved decoder are not Gaussian.

7.6 Each term alone

The sum of the two terms is what makes a generator, and removing either term shows why. Without the divergence term, the loss (7.8) rewards ever narrower clouds and allows their centres to move anywhere. The clouds collapse to points, the codes spread beyond the region of the prior, and the model returns to the autoencoder with holes. Codes drawn from $\mathcal{N}(0, I)$ then fall into regions where no sample was encoded. Without the reconstruction term, the loss is minimised by making every cloud equal to the prior, $\boldsymbol{\mu} = 0$ and $\boldsymbol{\sigma} = 1$ for every input. The code then carries no information about $\mathbf{x}$, and the decoder, which appears only in the

Table 9: The weight of the divergence term on the sine curve. Median encoder width, range of the encoder centres over the data, and Pearson correlation of the centre with the hidden position t.

β	median σ	range of μ	correlation of μ with t
0.02	0.0065	-3.99 to 4.04	0.989
1	0.028	-2.83 to 2.81	0.981
20	0.13	-2.20 to 2.19	0.985

Table 10: The variational autoencoder on MNIST. Losses are per training image in nats at the end of training, other quantities refer to the test digits or to generated digits.

quantity	value
loss, $d = 2$: reconstruction + KL	$138.9 = 132.1 + 6.8$
loss, $d = 16$: reconstruction + KL	$99.9 = 80.3 + 19.6$
parameters, $d = 16$	1 079 600
active coordinates of 16, KL per coordinate	11, from 0.99 to 2.58 nats
collapsed coordinates, KL per coordinate	5, 0.00 nats
sum of KL over coordinates, test digits	20.1 nats
Euclidean distance of grey values to the nearest training image, generated	4.73
same distance, test digits	4.03
generated pixel probabilities between 0.1 and 0.9	19.7%
latent plane, $d = 2$, in units of the median spacing of neighbouring codes	
draws from a Gaussian fitted to the codes, to the nearest training code, autoencoder	1.6
prior draws to the nearest training code, VAE	1.32
median width of the VAE clouds	5.6
prior draws where the cloud density is under a tenth of the prior	3.6%

reconstruction term, receives no training signal at all and stays at its random initial weights. A related failure occurs with both terms present. When the decoder can rebuild the data well without the code, the divergence term pushes the clouds onto the prior, the code carries nothing, and the decoder learns to ignore it and to produce typical images. This failure is called posterior collapse [18]. It can also affect individual coordinates of a trained model.

The sine curve shows the two limits in a controlled way. Multiplying the divergence term by a weight β and training with $\beta = 0.02$, 1 and 20 gives the results of Table 9. With $\beta = 0.02$ the divergence is almost switched off. The clouds are four times narrower than with $\beta = 1$, their centres spread beyond the region where the prior has appreciable probability, and codes drawn from the prior almost never reach the ends of the curve. With $\beta = 20$ the divergence dominates, the clouds widen to a median of 0.13, and a single output must serve a whole stretch of the curve, so the decoder cuts the corners of the sine. The order of the codes along the curve survives in all three cases.

7.7 New digits

We train the network of Figure 27 with the loss (7.8) on the 60 000 MNIST training images, without their labels, once with $d = 2$, which can be drawn, and once with $d = 16$. New digits are produced without the encoder, by drawing $\mathbf{z} \sim \mathcal{N}(0, I)$ and computing the pixel probabilities $\mathrm{sigm}(g_{\boldsymbol{\theta}}(\mathbf{z}))$. These are shown as grey images, the usual display of a variational autoencoder. A literal sample of the model would draw every pixel as ink with its probability,

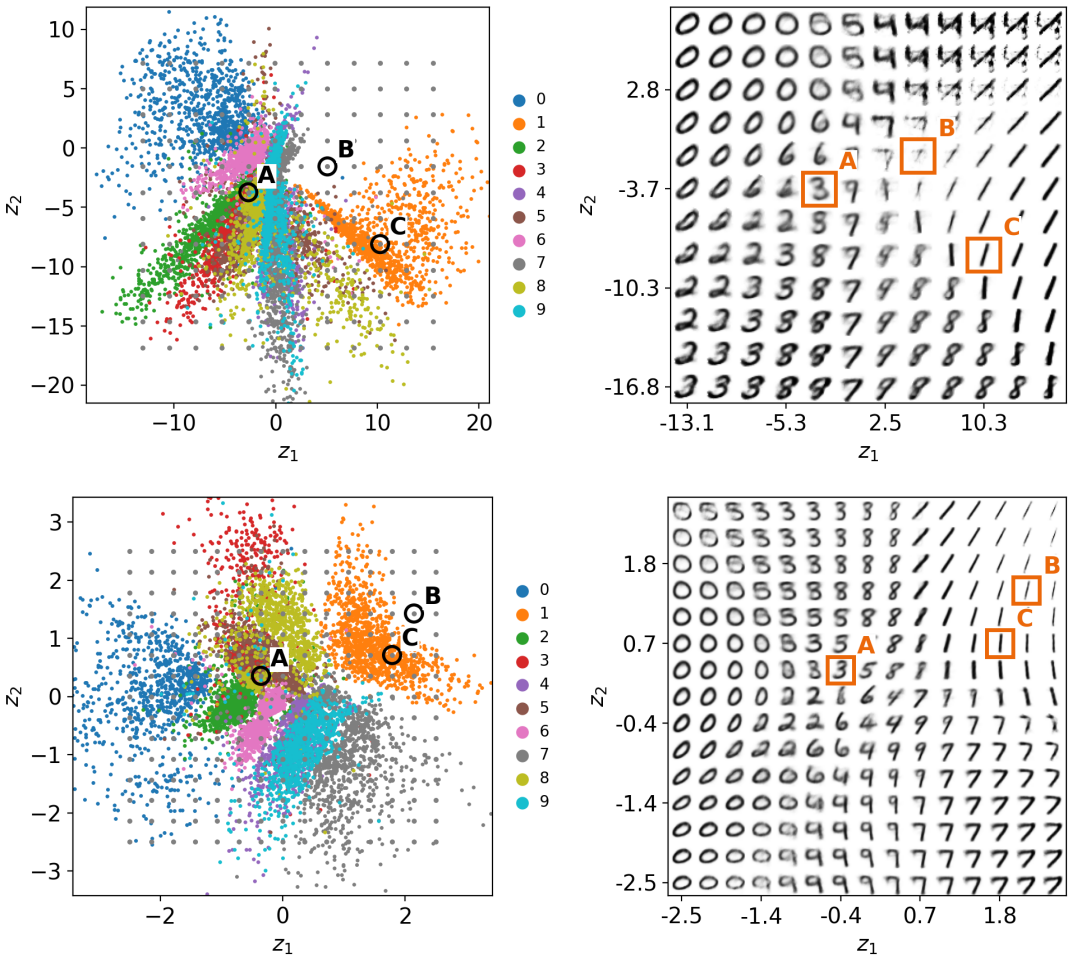


Figure 34: Walking through the latent plane. Top, an autoencoder with $d = 2$, bottom, a variational autoencoder with $d = 2$, both trained on MNIST. Left panels, the codes of the 10 000 test digits coloured by digit, with the grid points in grey. Right panels, the image decoded at every grid point, column by z_1 and row by z_2. Points A and C have many test codes nearby, point B none. In the autoencoder B decodes to a blur. In the variational autoencoder it decodes to a digit, and the whole sheet consists of digits that change smoothly.

which gives a speckled image, since many generated pixel probabilities lie far from 0 and 1 (Table 10). Since $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$ cannot be computed, the loss itself measures the quality of the model.

Table 10 collects the measured quantities. The larger latent space rebuilds the digits better and pays a larger divergence for the information it carries. Five of its 16 coordinates have collapsed onto the prior, and the decoder ignores them. The generated digits are new, since their distance in pixel space to the nearest training image is comparable to that of real test digits, which the network never saw.

The model with $d = 2$ allows the whole latent space to be looked at. Figure 34 decodes a regular grid of codes. For the autoencoder the grid has 12×12 points and spans the codes of the test digits, from the 1st to the 99th percentile of each coordinate. For the variational autoencoder it has 15×15 points and spans the square $[-2.5, 2.5]^2$, where the prior places almost all its probability. Three grid points are marked in each case, two with many test codes nearby (A and C) and one between the classes with no test code within 0.6 grid spacings (B). In the autoencoder, B lies in a hole and decodes to a faint image that is no digit. In the

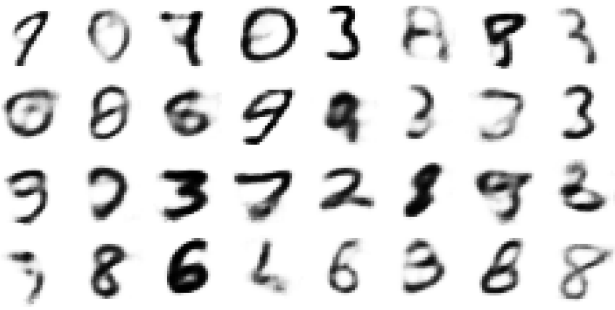


Figure 35: Thirty-two new digits from the variational autoencoder with $d = 16$, each decoded from a code drawn from $\mathcal{N}(0, I)$. The grey levels are the pixel probabilities $\text{sigm}(g_{\boldsymbol{\theta}}(\mathbf{z}))$. None of the images is a copy of a training image.

Table 11: The three distributions in the latent space of a variational autoencoder.

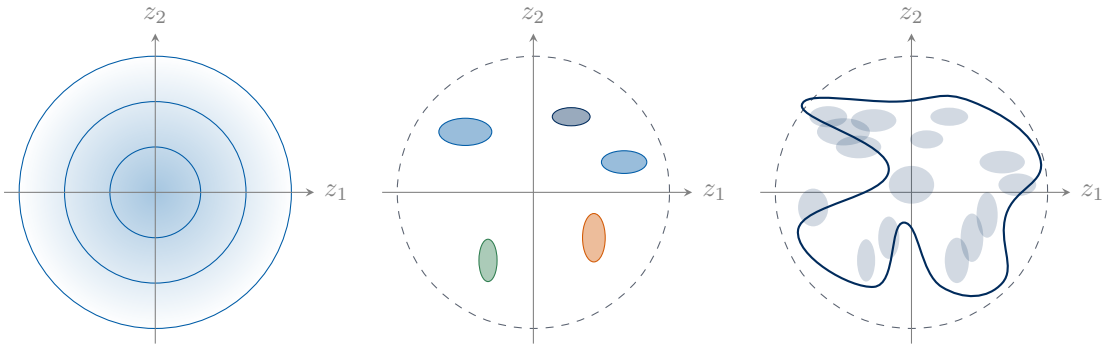
distribution	form	Gaussian	covariance
prior $p(\mathbf{z})$	$\mathcal{N}(0, I)$, chosen before training	yes	identity
encoder cloud $q_{\boldsymbol{\phi}}(\mathbf{z}\|\mathbf{x})$	$\mathcal{N}\big(\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}), \text{diag}\,\boldsymbol{\sigma}^2_{\boldsymbol{\phi}}(\mathbf{x})\big)$, one per input	yes, by construction	diagonal
true posterior $p_{\boldsymbol{\theta}}(\mathbf{z}\|\mathbf{x})$	Bayes' theorem with the decoder	in general no	in general not diagonal
aggregate posterior $q(\mathbf{z})$	$\frac{1}{n}\sum_i q_{\boldsymbol{\phi}}(\mathbf{z}\|\mathbf{x}_i)$	no, a mixture	anything

variational autoencoder B also has no test code nearby, and it decodes to a plausible 1. Neighbouring grid points give similar digits everywhere in the square, and the digits change continuously from one class into the next, which is what a latent space without holes looks like.

The lower part of Table 10 quantifies the difference visible in Figure 22. Random codes lie about as far from the nearest training code in both models. The difference lies in the clouds, each of which covers several neighbouring codes, so that the clouds overlap into the continuous density of the middle panel and almost all draws from the prior land where the decoder was trained (right panel of Figure 22).

8 Three distributions in the latent space

Three distributions describe the codes of a trained variational autoencoder, the prior from which new codes are drawn, the encoder cloud of each sample and the aggregate posterior of all samples together, and a fourth, the true posterior, is the target that each cloud approximates (Table 11). The prior is Gaussian because it is chosen so, and the encoder clouds are Gaussian because the encoder is built to output a Gaussian, while the true and the aggregate posterior are in general not Gaussian. Keeping them apart also settles whether a disentangled representation means a diagonal covariance. Figure 36 draws them, and Table 11 lists their properties.



(1) prior $p(\mathbf{z}) = \mathcal{N}(0, I)$
fixed, circular

(2) encoder clouds $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x}_i)$
one per sample, axis-aligned

(3) aggregate posterior $q(\mathbf{z})$
mixture of all clouds, lumpy

Figure 36: The three distributions in the latent space. (1) The prior is fixed, has identity covariance and independent coordinates, and generation draws from it. (2) Each sample has its own encoder cloud, a Gaussian with diagonal covariance whose ellipses are aligned with the axes. (3) The average of all clouds over the data set is a mixture of many ellipses with a lumpy shape, and the KL term pushes it only approximately towards the prior (dashed circle).

8.1 The prior

The prior $p(\mathbf{z}) = \mathcal{N}(0, I)$ is the Gaussian of case (a) in Figure 5. It is fixed before training and is not learned. Its coordinates are independent and have unit variance, and generation draws codes from it.

8.2 The encoder clouds

Each input $\mathbf{x}$ has its own encoder cloud $q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$, a Gaussian with a diagonal covariance, case (b) of Figure 5. The diagonal form keeps the encoder output at $2d$ numbers and gives the divergence the closed form (7.4). Replacing a distribution by a product of independent factors is called a mean-field approximation in statistical physics, and the diagonal cloud is the Gaussian mean-field approximation.

The cloud approximates the true posterior $p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})$, which is in general neither Gaussian nor diagonal, since it is the prior reshaped by a nonlinear decoder. The gap (6.5) measures how good the approximation is, and in a model with $d = 2$ the true posterior can be computed directly on a fine grid of codes by Bayes' theorem. This was done for a variational autoencoder trained on 20 000 images of 28×28 pixels, each showing a white bar of length 16 and width 3 pixels on a black background, with an angle drawn uniformly from $[0, \pi)$ and a horizontal shift drawn uniformly from -6 to 6 pixels. The model has two latent coordinates. Clear bars like those of the training are compared with three kinds of images the network never saw, a short bar of length 6, a faint bar with grey values multiplied by 0.3, and a bar buried in strong pixel noise. Table 12 and Figure 37 show the result. The width of a cloud is an uncertainty about the code under the fitted model. It is not directly an uncertainty about a physical quantity such as the angle of the bar, which would require a map from codes to angles, and it says nothing about whether the model itself is right.

For clear bars, the images the encoder was trained on, the cloud and the exact posterior agree. For the short bar the exact posterior is a thin streak, since a short bar fixes one combination of the latent coordinates sharply and leaves another open. A diagonal Gaussian cannot represent

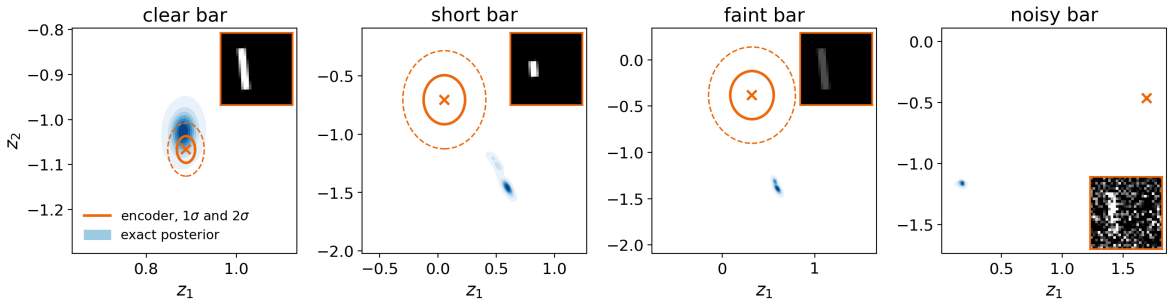


Figure 37: Exact posterior and encoder cloud for one image of each kind, in a variational autoencoder with $d = 2$ trained on clear bars. The exact posterior (blue, filled contours) is computed by Bayes' theorem on a fine grid of codes. The encoder cloud is drawn by its centre (cross) and its one- and two-standard-deviation ellipses (orange, solid and dashed). The inset shows the image. Each panel is centred between the two distributions and scaled to show both. For the clear bar the two coincide. For the short and faint bars the posterior is a thin tilted streak, and the encoder places a wider cloud elsewhere. For the noisy bar the encoder cloud is narrow and far from the posterior, and its ellipses are smaller than the cross that marks its centre.

Table 12: Width of the encoder cloud and of the exact posterior for images of a bar, geometric mean of the two widths, median over the test images of each kind.

image	encoder width	exact posterior width
clear bar	0.027	0.026
short bar	0.183	0.026
faint bar	0.257	0.032
noisy bar	0.0052	0.016

such an elongated tilted shape, and the encoder, extrapolating, places a much wider cloud at another location. For the noisy bar the encoder cloud is about three times narrower than the posterior and far from it. The encoder width measures the uncertainty of the code only for inputs like those of the training and only as far as the decoder describes the data.

8.3 The aggregate posterior

The third distribution describes where the codes of the whole data set lie. It is the average of all encoder clouds,

$$q(\mathbf{z}) \coloneqq \frac{1}{n} \sum_{i=1}^{n} q_{\boldsymbol{\phi}}(\mathbf{z} \,|\, \mathbf{x}_i), \tag{8.1}$$

a mixture (3.13) of n ellipses with equal weights, and it is not Gaussian. The divergence term pushes each cloud towards the prior, and the reconstruction term pushes the clouds apart, so the mixture fills the prior only approximately. For the digits, the middle panel of Figure 22 shows the aggregate posterior of the model with $d = 2$, a continuous density with dark wedges. A scatter plot of the cloud centres is a different object from the aggregate posterior, which also contains the width of every cloud. The centres of the test digits (Figure 38) gather by digit although no label was used, and their distribution differs from the prior, with more centres than the prior predicts within radius 1 and fewer within radii 2 and 3 (Table 13).

The density of the data, $p_{\boldsymbol{\theta}}(\mathbf{x})$, is not Gaussian either. The decoder bends the circular prior

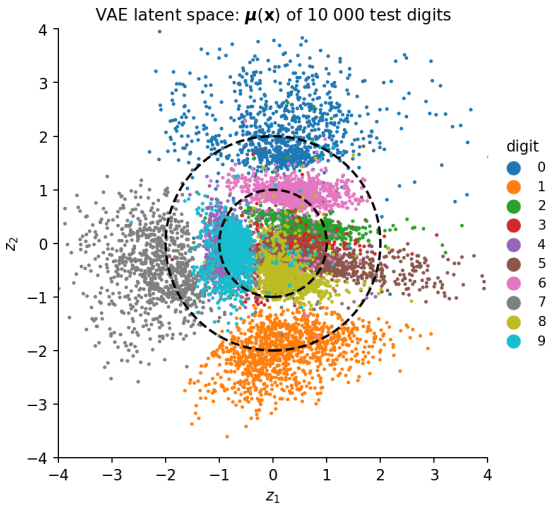


Figure 38: Centres $\boldsymbol{\mu}(\mathbf{x})$ of the encoder clouds of the MNIST test digits in a variational autoencoder with $d = 2$, coloured by digit, with the circles of radius 1 and 2 of the prior (dashed). The labels were not used in training. The clouds are narrower than the plotted dots, with a mean width of 0.041, but wider than the spacing of neighbouring codes (Table 10), and their mixture is the aggregate posterior.

Table 13: Fraction of the encoder centres of the MNIST test digits within a given radius of the origin, for the model of Figure 38, compared with the fraction of the probability of $\mathcal{N}(0, I)$ in two dimensions.

radius	1	2	3
encoder centres	48.8%	79.7%	96.5%
prior $\mathcal{N}(0, I)$	39.3%	86.5%	98.9%

into the curved data manifold, as Figure 33 shows for the sine curve, and adds a small amount of noise around it.

Diagonal clouds are not disentanglement

Every standard variational autoencoder uses encoder clouds with diagonal covariance, whether or not its coordinates have any meaning. The diagonal form is a computational choice for the approximate posterior. Disentanglement is a property of the aggregate posterior $q(\mathbf{z})$ of equation (8.1). It requires that the coordinates of the codes of the data be statistically independent, $q(\mathbf{z}) \approx \prod_k q(z_k)$, which is stronger than uncorrelated, and that each coordinate follow one generative factor of the data. Section 11.4 explains how the training can favour this property.

9 The conditional variational autoencoder

9.1 Generating what is asked for

A variational autoencoder trained on the digits produces digits of every class, in proportions set by the data, and the user cannot ask for a 7. The class of a generated digit is decided by where the drawn code happens to fall in the latent space (Figure 34). The remedy is

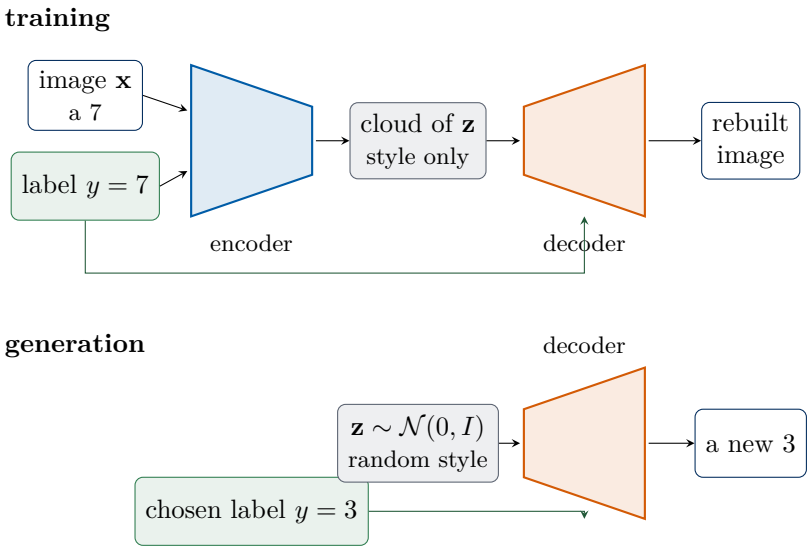


Figure 39: The conditional variational autoencoder. Top, training. The encoder receives the image and its label and returns the cloud of the code, the decoder receives a code from the cloud and the label and rebuilds the image. Since the label reaches the decoder directly, the code needs to hold only what the label does not say, the style of the handwriting. Bottom, generation without the encoder. The label is chosen, the style is drawn from the prior, and the decoder produces a new digit of the chosen class.

simple in its idea. If the label is given to the networks as an extra input, during training and during generation, the decoder learns to draw the digit it is told to draw, and the random code is left to decide only how the digit is written. Figure 39 shows the arrangement. Often the samples carry a label y that is known for every training sample and that the user would like to choose at generation time, the digit, the type of a blood cell or the temperature of a spin configuration. The object to be learned is then the conditional density $p(\mathbf{x}\,|\,y)$, the distribution of samples of a given label. The conditional variational autoencoder [26] learns it by giving the label to both networks. The label enters as a vector $\mathbf{c}$, the one-hot vector $\mathbf{e}_y$ for a class, with a 1 at position y and zeros elsewhere, or a rescaled number for a continuous label such as a temperature. The model is

$$p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,y) = \int p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z}, y)\, p(\mathbf{z})\, \mathrm{d}\mathbf{z}, \tag{9.1}$$

with the same fixed prior $p(\mathbf{z}) = \mathcal{N}(0, I)$, a decoder $p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z}, y)$ that receives the code and the label, and an encoder $q_{\boldsymbol{\phi}}(\mathbf{z}\,|\,\mathbf{x}, y)$ that receives the sample and its label.

A one-hot vector is used for classes because the label of a class is a name. Entering the digit as the number y would tell the network that 8 is closer to 9 than to 1, an order that the classes do not have. A physical parameter such as the temperature has a meaningful order and scale and enters as a number.

9.2 The conditional evidence lower bound

The derivation of Section 6 carries over when every distribution is conditioned on y. Bayes' theorem at fixed y gives, for every code, $p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,y) = p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z}, y)\, p(\mathbf{z}) / p_{\boldsymbol{\theta}}(\mathbf{z}\,|\,\mathbf{x}, y)$, and averaging

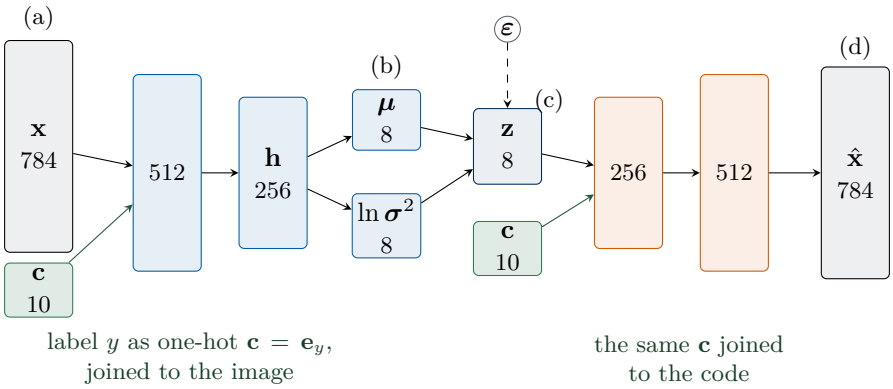


Figure 40: The conditional variational autoencoder for digits, equations (9.4). The one-hot label $\mathbf{c}$ (green) is joined to the image at the input of the encoder (a) and to the code at the input of the decoder (d). The encoder heads (b) and the sampling step (c) are those of the plain model. Widths are those of the model trained on MNIST, with $d = 8$.

its logarithm over the encoder cloud gives, as in equation (6.4),

$$\ln p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, y) = \underbrace{\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x},y)}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, \mathbf{z}, y)\big] - \mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z} \,|\, \mathbf{x}, y) \,\|\, p(\mathbf{z})\right)}_{\mathrm{ELBO}(\mathbf{x},y)} + \mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z} \,|\, \mathbf{x}, y) \,\|\, p_{\boldsymbol{\theta}}(\mathbf{z} \,|\, \mathbf{x}, y)\right). \tag{9.2}$$

The conditional bound has the two familiar terms, and the loss of a training pair $(\mathbf{x}, y)$ is its negative,

$$\mathcal{L}(\mathbf{x}, y) \coloneqq \underbrace{\mathbb{E}_{\mathbf{z}\sim q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x},y)}\big[-\ln p_{\boldsymbol{\theta}}(\mathbf{x} \,|\, \mathbf{z}, y)\big]}_{\text{reconstruct } \mathbf{x} \text{ from its code and its label}} + \underbrace{\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z} \,|\, \mathbf{x}, y) \,\|\, \mathcal{N}(0, I)\right)}_{\text{keep the cloud close to the prior}}, \tag{9.3}$$

estimated with one code per sample and averaged over a mini-batch as in equation (7.8). The first term is the binary cross-entropy of the decoded pixel probabilities, the second the closed form (7.4). The only change from the plain model is the label at the input of both networks.

The code $\mathbf{z}$ of a conditional model carries a different content. The divergence term charges every nat of information that the code carries about the sample, equation (7.4), while the information in the label reaches the decoder free of charge. The digit class is the largest single factor of variation of an image, and a model without the label must pay for it in the code, about $\ln 10 = 2.3$ nats for ten equally frequent classes. With the label available, storing the class in the code would duplicate free information at a cost, so the loss favours a code that stores mainly what the label does not say. For digits this is the style of the handwriting, its slant, thickness, width and size. The conditional model therefore separates the sample into a chosen part, the label, and a random part, the style.

9.3 Architecture

The label is appended to the input of the encoder and to the code before the decoder (Figure 40),

$$\mathbf{h} = f_{\boldsymbol{\phi}}([\mathbf{x}, \mathbf{c}]), \tag{9.4a}$$

$$\boldsymbol{\mu} = W_{\mu}\mathbf{h} + \mathbf{b}_{\mu}, \qquad \ln \boldsymbol{\sigma}^2 = W_{\sigma}\mathbf{h} + \mathbf{b}_{\sigma}, \tag{9.4b}$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I_d), \tag{9.4c}$$

$$\hat{\mathbf{x}} = \mathrm{sigm}\left(g_{\boldsymbol{\theta}}([\mathbf{z}, \mathbf{c}])\right), \tag{9.4d}$$

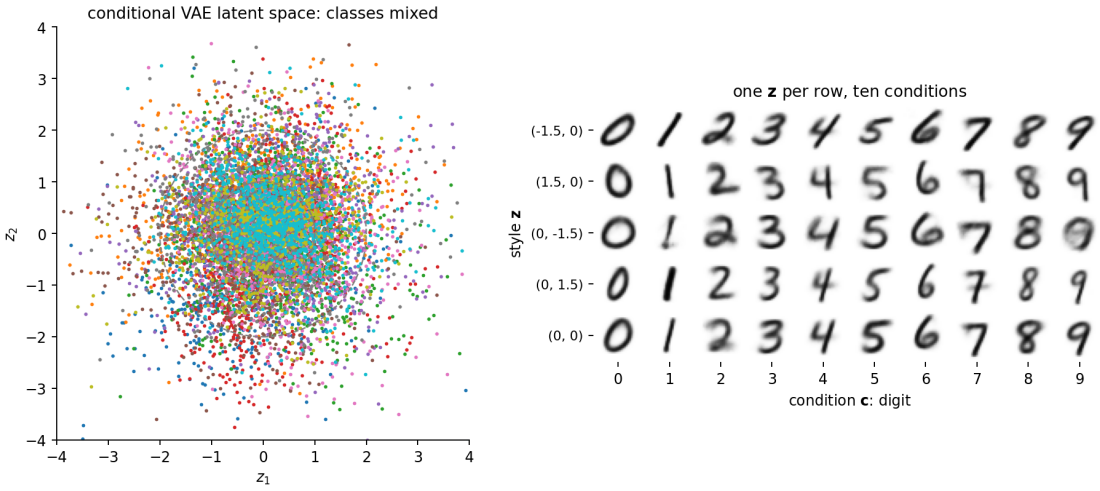


Figure 41: Latent space of a conditional variational autoencoder with $d = 2$ trained on MNIST with the digit as label. Left, the centres $\boldsymbol{\mu}(\mathbf{x}, \mathbf{c})$ of the test digits coloured by digit, mixed in one region. Right, five fixed codes (rows) decoded with the ten labels (columns). The label chooses the digit, the code chooses the style.

where $[\cdot, \cdot]$ joins two vectors into one. The encoder (9.4a) receives the image together with its label, so it knows which class it is encoding and can describe the image relative to that class. The heads (9.4b) and the sampling step (9.4c) are those of the plain model. The decoder (9.4d) receives the code and, a second time, the label, and turns them into pixel probabilities. During generation the encoder is not used. The user sets $\mathbf{c}$, draws $\mathbf{z}$ from the prior, and evaluates line (9.4d).

For images with colour channels and convolutional networks, the one-hot label enters the encoder as extra input channels. Channel k of the label is a constant image equal to 1 if the cell is of type k and 0 otherwise, so a blood-cell image with three colour channels and eight types enters as an array of $28 \times 28 \times (3+8)$ numbers. The decoder receives $[\mathbf{z}, \mathbf{c}]$ as a vector, as in the dense case. Table 28 in Appendix A lists the layers of the convolutional model used for the blood cells. A stride of two halves the width and height of the image in each convolution, and the transposed convolutions of the decoder double them.

9.4 Choosing the class and transferring the style

A trained conditional model is used in two ways. To generate a sample of a chosen class y, the label vector is fixed to $\mathbf{c} = \mathbf{e}_y$, a code is drawn from the prior, and the decoder is evaluated,

$$\mathbf{c} = \mathbf{e}_y \text{ chosen}, \qquad \mathbf{z} \sim \mathcal{N}(0, I), \qquad \hat{\mathbf{x}} = \operatorname{sigm}\left(g_{\boldsymbol{\theta}}([\mathbf{z}, \mathbf{c}])\right). \tag{9.5}$$

Many codes with one label give many samples of the same class in different styles. One code with many labels gives the same style for every class. To transfer the style of an existing sample $\mathbf{x}$ with label y to another class y', the sample is encoded with its own label, which gives the centre $\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}, \mathbf{e}_y)$ of its cloud, and this code is decoded with the new label, $\hat{\mathbf{x}}' = \operatorname{sigm}\left(g_{\boldsymbol{\theta}}([\boldsymbol{\mu}_{\boldsymbol{\phi}}(\mathbf{x}, \mathbf{e}_y), \mathbf{e}_{y'}])\right)$. The result shows the class y' written in the hand of $\mathbf{x}$, provided that the code holds only the style, as argued in Section 9.2. Section 9.6 measures how far this holds. The conditional model needs the label of every training sample. What the labels do not describe is left to the code.

Table 14: Conditional variational autoencoders on MNIST. Losses per training image in nats at the end of training. The plain model with $d = 2$ is a second training run of the network of Table 10.

quantity	value
plain VAE, $d = 2$: reconstruction + KL	138.7 = 132.0 + 6.7
conditional VAE, $d = 2$: reconstruction + KL	127.3 = 122.3 + 5.0
conditional VAE, $d = 8$: reconstruction + KL	97.6 = 83.4 + 14.2
classifier accuracy on real test digits	0.969
generated digits recognised as requested, mean over classes	0.952
generated digits recognised as requested, range over classes	0.906 to 0.990



Figure 42: Digits of a chosen class from the conditional variational autoencoder with $d = 8$. Row k uses the label $\mathbf{e}_k$ and twelve codes drawn from $\mathcal{N}(0, I)$. Within a row the digit is fixed and the style varies.

9.5 Digits of a chosen class

We first train a conditional model with $d = 2$ on the MNIST training digits and their labels, with the dense layers of Figure 40. Its loss and its divergence are lower than those of a second training run of the same network without the label (Table 14). The label supplies part of the information, and the code carries less of it. The left panel of Figure 41 shows that the codes of the different classes overlap, a sign that the code carries little information about the class. Whether it keeps the style is tested in Section 9.6. The centres of the test digits of all classes overlap in the same region of the latent plane. The right panel fixes one code per row and decodes it with all ten labels. Along a row the digit changes while the slant and the thickness stay the same, which is the style transfer of Section 9.4 applied to codes chosen in the latent plane.

For generation we train the model of Figure 40 with $d = 8$ on the 60 000 training images and their labels, with the loss (7.8) and the label appended to both inputs (Table 27). Samples are produced by equation (9.5). Figure 42 shows twelve generated digits of every class.

Whether a generated digit is the one asked for is judged by a classifier that has seen only real digits, a dense network with layers $784 \to 256 \to 10$ trained with the cross-entropy loss. It labels 1000 generated digits of each class (Figure 43 and Table 14). A harder test trains the same classifier on 60 000 generated digits only, 6000 per class with the requested class as

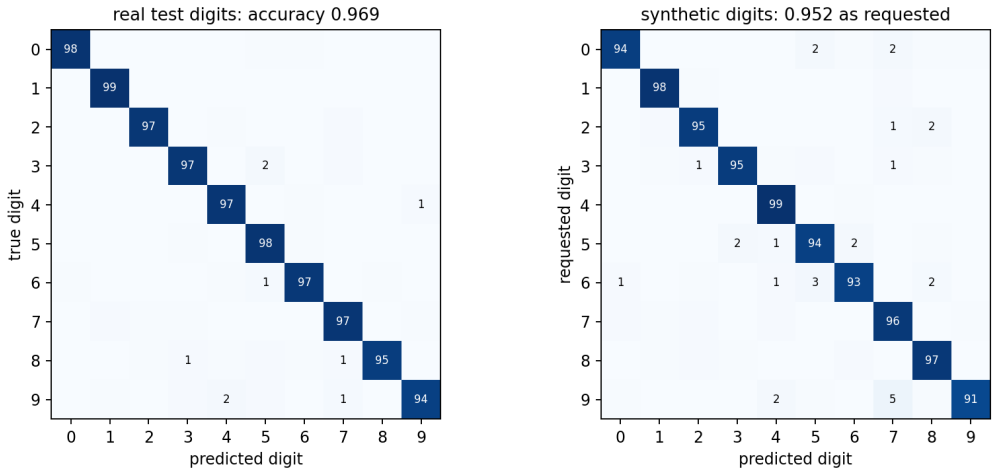


Figure 43: Confusion matrices of a classifier trained on real MNIST digits, in percent of each row. Left, real test digits, rows by true digit. Right, 1000 generated digits per class, rows by requested digit. The diagonal of the right matrix ranges from 90.6% for the digit 9 to 99.0% for the digit 4, with a mean of 95.2%.

Table 15: Accuracy of classifiers trained and tested on real or generated data. The generated test sets are drawn fresh, 1000 digits per class or 500 cells per type, independently of those of Figures 43 and 45, so the values differ from those in Tables 14 and 17 in the third decimal.

	MNIST		BloodMNIST	
	tested on real	tested on generated	tested on real	tested on generated
trained on real	0.969	0.955	0.838	0.514
trained on generated	0.918	0.986	0.637	0.864

label, and tests it on the real test digits. Table 15 collects the four combinations of real and generated data for training and testing. Trained on real and tested on generated data, the classifier measures whether the generator draws the requested class. Trained on generated and tested on real data, it measures whether the generated data carry what is needed to recognise real samples. For digits both values are high, and the gap between training on generated and on real digits measures what the generator misses of the real distribution, styles it never produces and blurred strokes.

9.6 Style transfer between digits

The transfer of Section 9.4 tests whether the code of the conditional model with $d = 8$ holds the style of a digit and leaves its class to the label. Each of the 10 000 test digits is encoded with its own label and decoded with each of the nine other labels, which gives 90 000 transferred images. The class of a transferred image is judged by the classifier trained on real digits. The style is summarised by two numbers that can be computed for any image. Let x_{ab} be the grey value of the pixel in row a and column b, and let u_{ab} and v_{ab} be its horizontal and vertical coordinates, u growing to the right and v growing upwards, both measured from the centre of mass of the ink. With the grey values as weights, the slant and the amount of ink are

$$s := \frac{\sum_{ab} x_{ab}\, u_{ab} v_{ab}}{\sum_{ab} x_{ab}\, v_{ab}^2}, \qquad \mathcal{I} := \sum_{ab} x_{ab}. \tag{9.6}$$

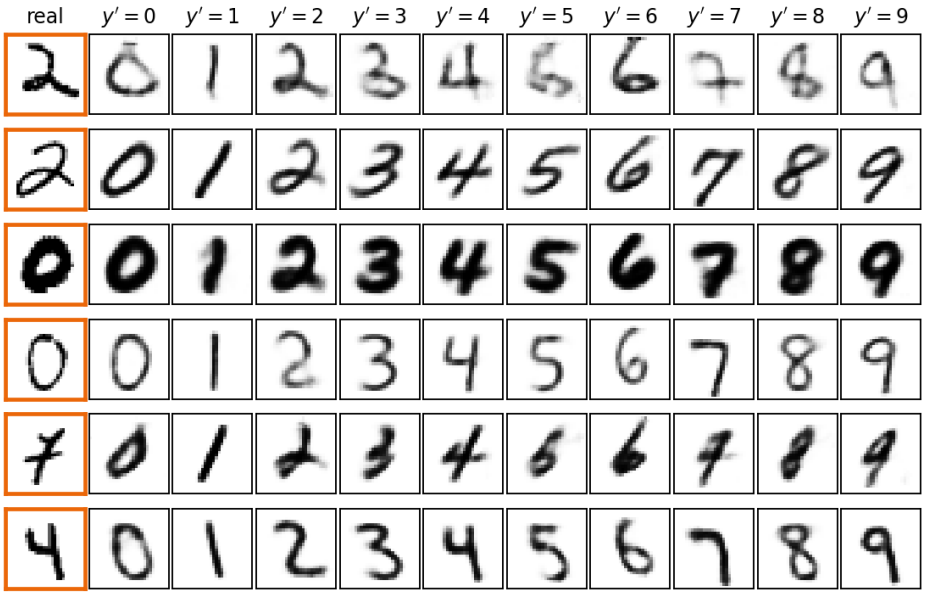


Figure 44: Style transfer. Each row starts with a real test digit (orange frame), followed by the centre of its cloud decoded with the ten labels $y' = 0, \ldots, 9$. The column with its own label is a reconstruction and is excluded from the statistics of Table 16. The six digits lie at the edges of the slant or ink range of their class, from the top a 2 leaning to the left, a 2 leaning to the right, a thick 0, a thin 0, a 7 leaning to the right and a 4 leaning to the left.

Table 16: Style transfer with the conditional variational autoencoder with $d = 8$, over the 90 000 transfers of the 10 000 test digits to the nine other labels. The slant s and the ink $\mathcal{I}$ are those of equation (9.6).

quantity	value
transferred digits read as the new label	0.959
generated digits read as the requested label (Figure 43)	0.952
correlation of s, source and transferred digit	0.85
correlation of $\mathcal{I}$, source and transferred digit	0.61

The slant s is the least-squares slope of u against v, the line $u = s\,v$ that best follows the ink, and it equals the tangent of the angle by which the stroke leans from the vertical. An upright stroke has $s = 0$, and a stroke leaning to the right has $s > 0$. The ink $\mathcal{I}$ grows with the thickness and the size of the stroke. If the code carries the style, the slant and the ink of a transferred image follow those of its source digit, and the correlation between source and transferred values, pooled over all 90 000 pairs, measures how closely.

The classifier reads 95.9% of the transferred digits as the new class, as often as the digits generated from random codes (Table 16). The slant of a transferred digit follows that of its source closely, with a correlation of 0.85. The ink follows more loosely, with a correlation of 0.61, since part of the ink is set by the class, and a 1 needs less ink than an 8 in any hand. Figure 44 shows the transfer for six digits with pronounced styles. The thick 0 gives thick digits, the thin 0 gives thin upright digits, and the 7 leaning to the right gives digits leaning to the right. The label replaces the identity of the digit, and the code keeps the hand that wrote it. The first row shows the limit of the method. A 2 leaning to the left is rare in the training data, its code lies where the decoder has seen few examples, and several of its transfers are faint.

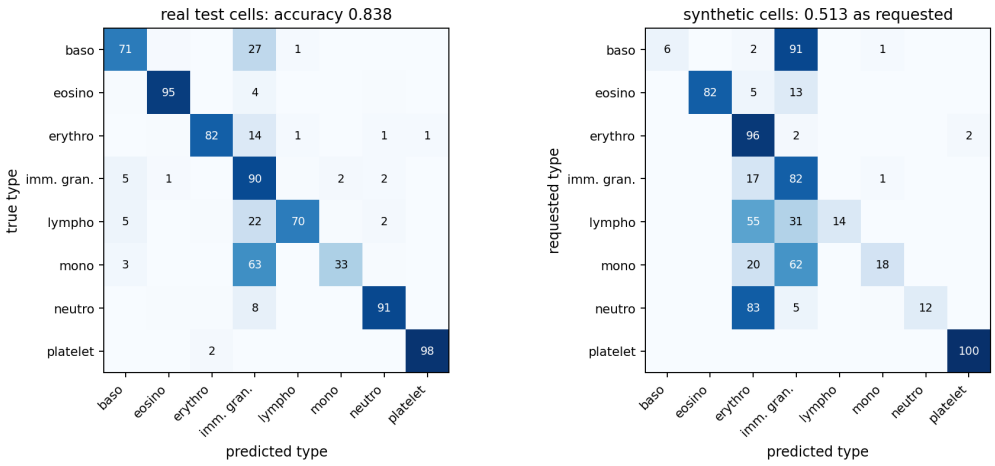


Figure 45: Confusion matrices of the classifier trained on real blood cells, in percent of each row. Left, real test cells. Right, 500 generated cells per requested type. Generated basophils and neutrophils are mostly assigned to other types, and generated lymphocytes and monocytes are also mostly taken for other types (Table 17).

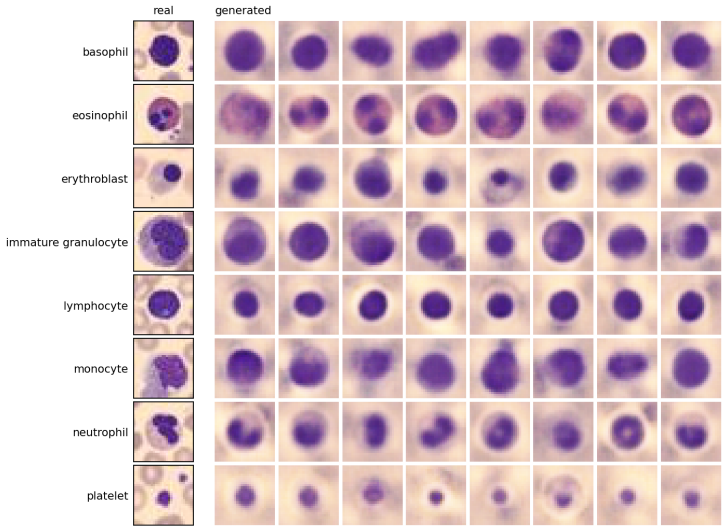


Figure 46: Blood cells of a chosen type from the convolutional conditional variational autoencoder. Each row is one type. The first column shows a real training cell of the type, the following columns cells generated from codes drawn from $\mathcal{N}(0, I)$.

9.7 Blood cells of a chosen type

The blood-cell images of BloodMNIST [2, 3], introduced in Lecture 5, are microscope images of single normal blood cells from healthy donors, reduced to 28×28 pixels in three colour channels. Each image carries one of eight types assigned by pathologists, and the 11 959 training images are unevenly divided, from 849 lymphocytes to 2330 neutrophils. The convolutional model of Section 9.3 with $d = 16$ is trained on the images and their types (Table 27). Cells of a chosen type are generated by equation (9.5) with the one-hot type (Figure 46).

The judge is a small convolutional classifier trained on real cells, with two blocks of convolution, ReLU and pooling followed by dense layers. It labels 500 generated cells of each type, and Table 17 lists the fraction recognised as the requested type. Types that differ in

Table 17: Fraction of 500 generated cells of each type that a classifier trained on real cells, with accuracy 0.838 on real test cells, assigns to the requested type, and accuracy per type of a classifier trained on 12 000 generated cells and tested on real cells.

type	generated, recognised	real, recognised after synthetic training
basophil	0.062	0.816
eosinophil	0.822	0.635
erythroblast	0.962	0.646
immature granulocyte	0.818	0.380
lymphocyte	0.142	0.827
monocyte	0.182	0.317
neutrophil	0.116	0.643
platelet	1.000	0.947
mean or overall	0.513	0.637

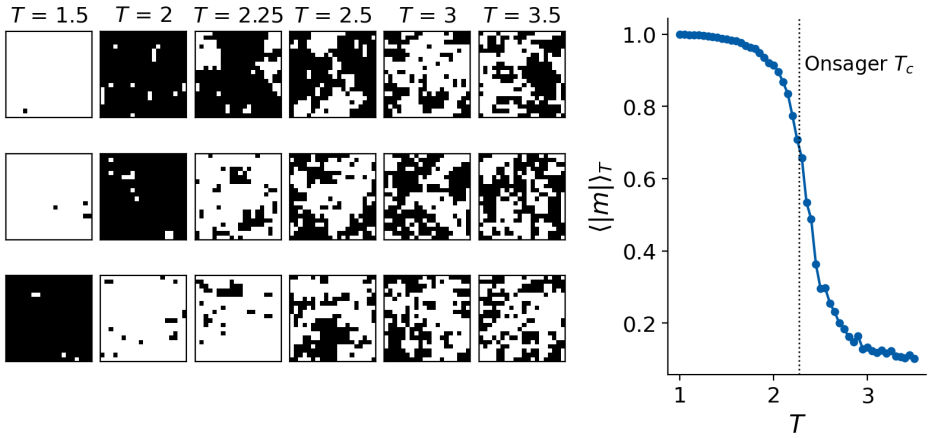


Figure 47: Monte Carlo configurations of the 20×20 Ising model at several temperatures (white, spin up, black, spin down), and the mean absolute magnetisation per spin as a function of temperature. The dotted line marks Onsager's T_c.

size and colour come out right, platelets and erythroblasts almost always. Types that differ in the shape of the nucleus do not, and generated basophils, lymphocytes, monocytes and neutrophils are mostly taken for other types (Figure 45). The blur that the variational autoencoder produces removes the detail by which the classifier tells these types apart. The four combinations of real and generated data in Table 15 complete the picture. A classifier trained on generated cells recognises generated cells well and real cells much worse, so the generated types are easy to tell apart from each other and differ from the real types.

9.8 Spin configurations at a chosen temperature

For spin configurations of the two-dimensional Ising model the data distribution is known. A configuration $\mathbf{s} = (s_1, \ldots, s_{N_\mathrm{s}})$ of $N_\mathrm{s} = 400$ spins $s_i = \pm 1$ on a 20×20 square lattice with periodic boundaries occurs at temperature T with the Boltzmann probability

$$p_{\mathrm{data}}(\mathbf{s} \,|\, T) = \frac{e^{-\mathcal{H}(\mathbf{s})/T}}{Z(T)}, \qquad \mathcal{H}(\mathbf{s}) = -\sum_{\langle ij \rangle} s_i s_j, \tag{9.7}$$

Table 18: Generated against Monte Carlo configurations of the Ising model. Differences are taken over the 51 training temperatures.

observable	largest difference	at T	mean difference
mean $\lvert m\rvert$	0.234	2.10	0.109
mean energy per spin e	0.634	2.30	0.434
temperatures up to 1.5			fraction
configurations with $\lvert m\rvert < 0.5$, Monte Carlo			0.000
configurations with $\lvert m\rvert < 0.5$, generated			0.085

where the sum runs over pairs of nearest neighbours, the coupling and the Boltzmann constant are set to one, and the partition function $Z(T)$ normalises the distribution. On the infinite lattice the order is lost at Onsager's critical temperature $T_c = 2/\ln(1+\sqrt{2}) \approx 2.269$ [29]. The distribution (9.7) is the p_{data} that the model must learn from samples, and the conditional model learns the whole family of distributions over all temperatures, with $y = T$. Since the true distribution is known, its samples can be checked with the observables of statistical physics.

The data are 10 200 configurations from Metropolis Monte Carlo sampling [30], 200 at each of 51 temperatures from 1.0 to 3.5 in steps of 0.05, those of Lecture 4. At low temperature nearly all spins are aligned, near T_c domains of all sizes appear, and at high temperature the spins are correlated only over a few lattice spacings (Figure 47). Each spin becomes a pixel $x_i = (s_i + 1)/2 \in \{0, 1\}$, so that the Bernoulli decoder and the binary cross-entropy apply, and the temperature enters both networks as the number $c = (T - 2.25)/0.75$, which runs from -1.67 to 1.67. Encoder and decoder are dense networks with $d = 8$ (Table 27), and the decoder returns 400 probabilities $\hat{x}_j$ that spin j points up. The reconstruction then costs 0.33 nats per spin, about half of the $\ln 2 = 0.69$ nats of a fair coin for every spin. The divergence, 6.3 nats or about 9 bits per configuration, is an upper bound on the information the code carries about a configuration, equation (7.5).

A configuration at a chosen temperature is generated without the encoder. The condition c is set from T, a code is drawn from $\mathcal{N}(0, I)$, the decoder returns the 400 probabilities, and each spin is drawn up with its probability. Repeating this 200 times at each training temperature gives a generated data set of the same size as the Monte Carlo one. Figure 48 compares configurations by eye, and Figure 49 and Table 18 compare the magnetisation per spin $m = \frac{1}{N_s}\sum_i s_i$ and the energy per spin $e = \mathcal{H}/N_s$. The generated magnetisation follows the shape of the Monte Carlo curve and drops near T_c, but it is too low in the ordered phase and too high in the disordered one. The energy is too high at every temperature, most near T_c. Fluctuations fail more strongly. The variance of $\lvert m\rvert$ and of e over the configurations at one temperature, which up to constant factors are the susceptibility and the heat capacity of the finite lattice, peak near T_c for Monte Carlo and grow towards low temperature for the generated data, because a minority of the generated configurations at low temperature have a small magnetisation, which no Monte Carlo configuration has (Table 18).

The spin–spin correlation $G(r)$, the average product of spins r lattice spacings apart, shows the origin of the defect (Figure 50). The generated correlations are too small at short distance and too large at long distance. At $T = 2.6$ the connected nearest-neighbour correlation, $G(1)$ minus $\langle m^2\rangle$, is 0.422 for Monte Carlo and 0.080 for generated configurations. The cause lies in the decoder. Given the code and the temperature, the spins are drawn independently, so the expected product of two different spins is the product of their means, and every correlation between spins must pass through the eight numbers of the code. A smooth probability map

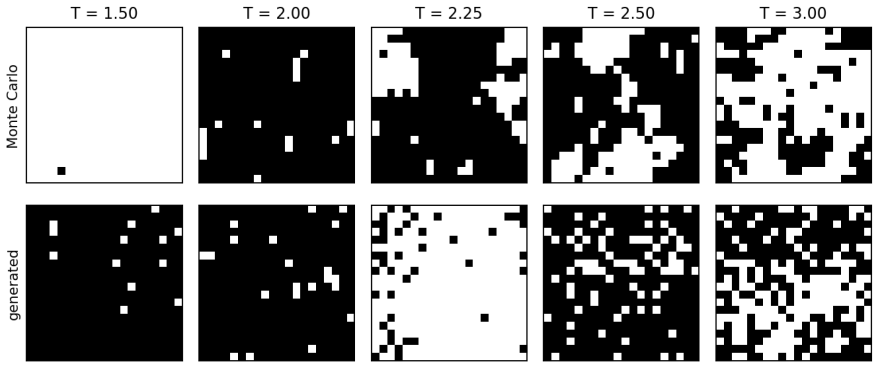


Figure 48: Monte Carlo configurations (top) and configurations generated by the conditional variational autoencoder (bottom) at five temperatures. The generated configurations carry many isolated flipped spins and lack the compact domains of Monte Carlo near and above T_c.

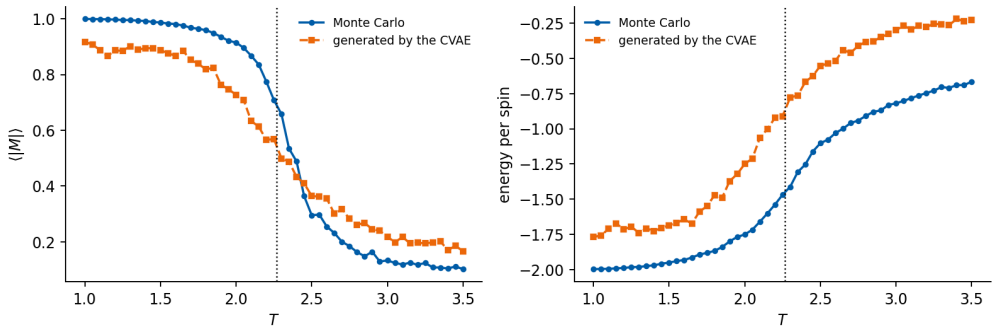


Figure 49: Mean absolute magnetisation per spin $\langle|m|\rangle$ (left) and mean energy per spin (right) against temperature, for Monte Carlo (blue) and generated (orange) configurations, 200 at each temperature. The dotted line marks T_c.

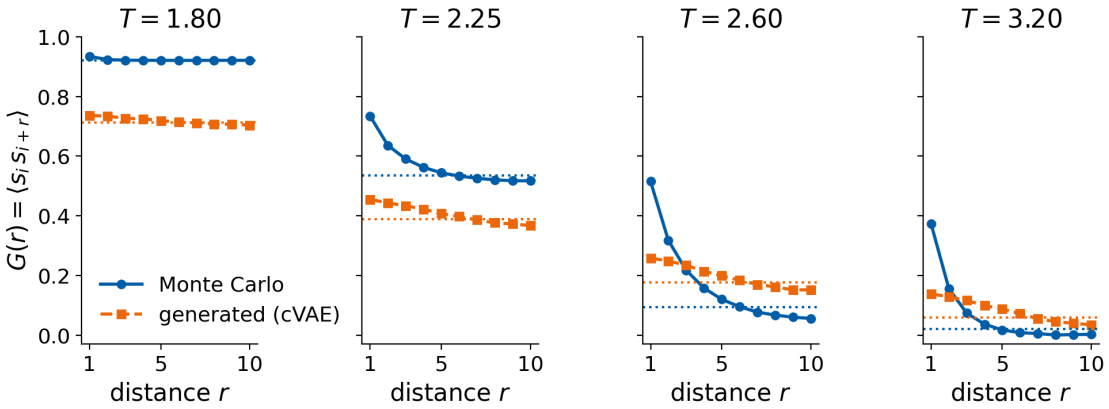


Figure 50: Spin–spin correlation $G(r)$ against distance r at four temperatures, for Monte Carlo and generated configurations, with the level $\langle m^2\rangle$ dotted. The generated correlation is too weak at short distance and too strong at long distance.

Table 19: Sampling from the trained latent space.

model	chosen	drawn	output
VAE, digits	nothing	$\mathbf{z} \sim \mathcal{N}(0, I_{16})$	pixel probabilities $\mathrm{sigm}(g_{\boldsymbol{\theta}}(\mathbf{z}))$, a digit of some class
conditional VAE, digits	class, $\mathbf{c} = \mathbf{e}_y$	$\mathbf{z} \sim \mathcal{N}(0, I_8)$	a digit of class y in a random style
conditional VAE, cells	type, $\mathbf{c} = \mathbf{e}_y$	$\mathbf{z} \sim \mathcal{N}(0, I_{16})$	a colour image of a cell of type y
conditional VAE, Ising	temperature, $c = (T - 2.25)/0.75$	$\mathbf{z} \sim \mathcal{N}(0, I_8)$, then each spin	a configuration of 400 spins at T

can place large domains, which gives the long-range part, but the short-range order, which neighbour is aligned with which, differs from configuration to configuration and does not fit through a code that carries at most about 9 bits. Every isolated flipped spin breaks four bonds and raises the energy.

Two networks trained on configurations, a phase classifier and a temperature regressor, confirm this from outside (Appendix B). The conditional model has learned how the configurations change with temperature, but not the Boltzmann distribution itself.

9.9 Generating new data from the trained latent space

Three generators have now been trained, and Table 19 sets their sampling procedures side by side. In all three the encoder is discarded after training and the prior $\mathcal{N}(0, I)$ supplies the randomness. The plain model draws everything from the code, so the class of a generated digit is decided by where the code falls in the latent plane, as the grid of Figure 34 shows. The conditional model separates what is chosen from what is drawn. For digits and cells the user chooses the class, and the code supplies the style. For the Ising model the user chooses a temperature, and the code supplies one configuration, after which the Bernoulli decoder draws the individual spins. In the Ising model the draw of the spins is part of the sample, since a configuration consists of spins, while for images the pixel probabilities themselves are usually shown.

An image is a set of pixels, and a distribution of images is a joint distribution of all its pixels together. Generation in a variational autoencoder draws the pixels one by one only after a single shared draw. The code $\mathbf{z}$ is drawn once for the whole image, and it fixes through the decoder the probability of ink of every pixel, where the stroke runs, how thick it is and which digit it forms. Given the code, the pixels are independent, and each could be drawn on its own. The model of independent pixels of Section 3.2 draws the pixels one by one without such a shared draw, and its images have no strokes (Figure 13). Every dependence between the pixels of a variational autoencoder passes through the code. Other families of models distribute the work differently. An autoregressive model draws the pixels one after another, each conditioned on all pixels drawn before it, according to the product rule $p(\mathbf{x}) = \prod_j p(x_j \mid x_1, \ldots, x_{j-1})$ [13]. The generative adversarial network of Section 10 produces all pixels of an image at once, as the output of one network.

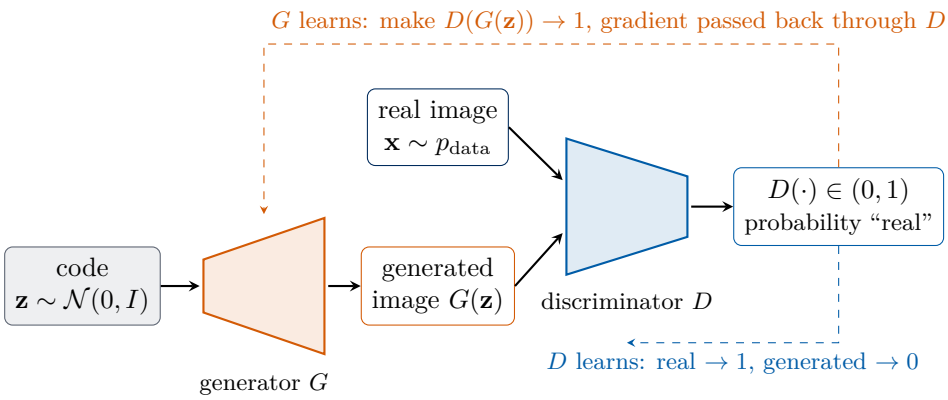


Figure 51: A generative adversarial network. The generator (orange) turns a random code into an image. The discriminator (blue) receives real images and generated images and returns the probability that its input is real. The discriminator is trained to separate the two kinds of input, and the generator is trained, through the gradient of the discriminator output, to make its images pass as real. Neither network computes a likelihood.

10 Generative adversarial networks

10.1 A generator judged by a second network

Before the chapter returns to the meaning of the latent coordinates, it compares the variational autoencoder with the other widely used way of turning random codes into samples. The comparison belongs here, after the reader has seen what a likelihood-based generator produces, and it shows which properties of the variational autoencoder come from the likelihood. Every model of the chapter so far is trained by maximum likelihood or by a bound on it. The loss compares a generated or reconstructed image with a real one pixel by pixel, and a decoder that is unsure where a stroke runs hedges by spreading grey ink over all places where it might run. This is why the digits of the variational autoencoder look blurred (Figure 35). A sharp stroke one pixel away from the true one costs more under a pixel-wise likelihood than a blurred stroke that covers both positions.

A generative adversarial network [15] drops the likelihood. It keeps the generator of a latent-variable model, a network G that turns a code $\mathbf{z} \sim \mathcal{N}(0, I)$ into an image $G(\mathbf{z})$, but it has no decoder distribution $p(\mathbf{x} \mid \mathbf{z})$ and no encoder. The generated images are judged instead by a second network, the discriminator D, which receives an image and returns a number $D(\mathbf{x}) \in (0, 1)$, its estimate of the probability that $\mathbf{x}$ is a real image of the data set and not a generated one. The discriminator is trained to tell real images from generated ones. The generator is trained to produce images that the discriminator takes for real. An analogy describes the arrangement. A forger (the generator) produces paintings, an expert (the discriminator) learns to tell them from originals, and each improves by competing with the other, until the expert can no longer tell the difference. Figure 51 shows the two networks and the flow of information between them.

10.2 The loss as a game

The discriminator solves a classification problem with two classes, real ($y = 1$) and generated ($y = 0$), and its natural loss is the binary cross-entropy (3.12) of these labels. Written as a

quantity to be maximised, the log-likelihood of the correct labels is

$$V(D,G) \coloneqq \mathbb{E}_{\mathbf{x}\sim p_{\text{data}}}\big[\ln D(\mathbf{x})\big] + \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}\Big[\ln\big(1-D(G(\mathbf{z}))\big)\Big]. \tag{10.1}$$

The first term is large when real images receive $D(\mathbf{x})$ close to one, and the second when generated images receive $D(G(\mathbf{z}))$ close to zero. The discriminator raises V. The generator appears only in the second term, and it lowers V by making $D(G(\mathbf{z}))$ large. The two networks therefore pursue opposite goals with one and the same function, and training seeks

$$\min_G \max_D V(D,G). \tag{10.2}$$

Definition 10.1 (Min-max game)**.** Two players choose parameters a and b, and a single function $V(a,b)$ is a gain for the second player and a loss for the first. The first player minimises V over a, the second maximises it over b. A solution is a saddle point (a^*,b^*), at which neither player can improve by changing only its own parameters, $V(a^*,b) \le V(a^*,b^*) \le V(a,b^*)$ for all a and b.

For a fixed generator the best discriminator can be found point by point. At a given image the integrand of (10.1) has the form $p_{\text{data}}\ln D + p_G \ln(1-D)$, where p_G is the density of the generated images, and its maximum over $D \in (0,1)$ lies where the derivative $p_{\text{data}}/D - p_G/(1-D)$ vanishes,

$$D^*(\mathbf{x}) = \frac{p_{\text{data}}(\mathbf{x})}{p_{\text{data}}(\mathbf{x}) + p_G(\mathbf{x})}. \tag{10.3}$$

The best discriminator compares the two densities. It returns one half where real and generated images are equally frequent, and values near one where only real images occur. Inserting D^* into V gives $V(D^*,G) = \mathbb{E}_{p_{\text{data}}}\big[\ln \frac{p_{\text{data}}}{p_{\text{data}}+p_G}\big] + \mathbb{E}_{p_G}\big[\ln \frac{p_G}{p_{\text{data}}+p_G}\big]$. Writing the sum in the denominators as twice the average of the two densities, $p_{\text{data}}+p_G = 2m$ with $m = \frac{1}{2}(p_{\text{data}}+p_G)$, every logarithm splits as $\ln \frac{p_{\text{data}}}{2m} = \ln \frac{p_{\text{data}}}{m} - \ln 2$, and the two terms $-\ln 2$ add up to $-\ln 4$. This gives

$$V(D^*,G) = \mathrm{KL}\left(p_{\text{data}} \,\|\, m\right) + \mathrm{KL}\left(p_G \,\|\, m\right) - \ln 4 \;\coloneqq\; 2\,\mathrm{JS}(p_{\text{data}}, p_G) - \ln 4, \tag{10.4}$$

where JS is the Jensen–Shannon divergence, the average divergence of the two densities from their mean. Like every divergence it is never negative, by Theorem 3.3, and it vanishes only when $p_G = p_{\text{data}}$. Against the best discriminator, the generator therefore minimises a divergence between the generated and the real distribution, as maximum likelihood minimises $\mathrm{KL}(p_{\text{data}} \,\|\, p_\theta)$ in Section 3.3, and the solution of the game is $p_G = p_{\text{data}}$ with $D^* = \frac{1}{2}$ everywhere [15]. The divergence is never computed. The discriminator estimates it implicitly from samples, which is why no density of the generated images is needed.

10.3 Why the game is hard to train

A variational autoencoder minimises one loss, and a small enough gradient step on a mini-batch lowers it on average. A generative adversarial network has no such loss. The two networks take gradient steps at the same time, the discriminator uphill and the generator downhill on the same function V, and every step of one player changes the landscape on which the other moves. Training then looks for a saddle point, and gradient steps do not in general converge to one. The smallest example shows why. Let the two players hold one number each, a and b, and let $V(a,b) = ab$. The saddle point is $a = b = 0$. Simultaneous gradient steps with step size η,

$$a \;\leftarrow\; a - \eta\,\frac{\partial V}{\partial a} = a - \eta\, b, \qquad b \;\leftarrow\; b + \eta\,\frac{\partial V}{\partial b} = b + \eta\, a, \tag{10.5}$$

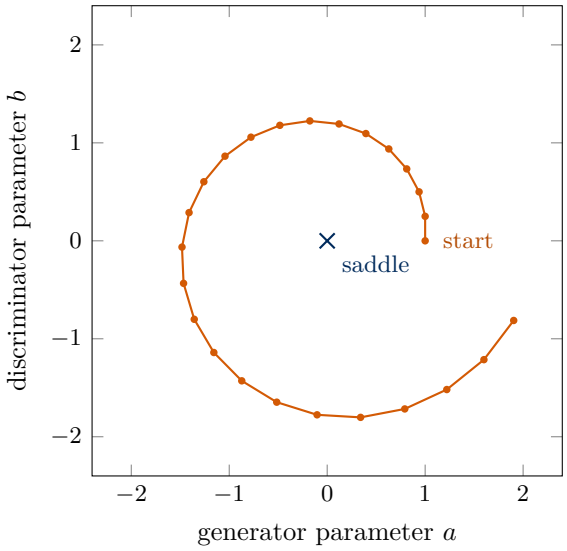


Figure 52: Simultaneous gradient steps (10.5) on the game $V(a, b) = ab$, with step size $\eta = 0.25$, starting from $(1, 0)$. The point circles the saddle at the origin, where neither player can improve, and moves away from it, after 24 steps to the distance 2.07.

rotate the point (a, b) around the saddle and multiply its distance from it by $\sqrt{1 + \eta^2}$ at every step. The players circle the solution and drift away from it (Figure 52). Each player responds to where the other is now, not to where the other will be after its own step. In a network with millions of weights the same behaviour appears as oscillating losses and as images whose quality improves and degrades again during training.

Three further difficulties are specific to the networks. The first is a vanishing gradient at the start of training. The generator begins with noise, and the discriminator rejects it confidently with $D(G(\mathbf{z})) \approx 0$. The discriminator computes its output as $D = \operatorname{sigm}(a)$ from an unbounded number a, the logit, and the gradient reaches the generator through a. The derivative of the generator term of equation (10.1) with respect to the logit is $\frac{\mathrm{d}}{\mathrm{d}a} \ln\left(1 - \operatorname{sigm}(a)\right) = -\operatorname{sigm}(a) = -D$, which vanishes as $D \to 0$, so a generator whose images are confidently rejected receives almost no gradient. The remedy, already proposed in [15], is the non-saturating generator loss

$$\mathcal{L}_G \coloneqq -\mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}\Big[\ln D\big(G(\mathbf{z})\big)\Big], \qquad \mathcal{L}_D \coloneqq -V(D, G), \tag{10.6}$$

which has the same aim, a large $D(G(\mathbf{z}))$, but is steep exactly where the generator is still bad, since $\frac{\mathrm{d}}{\mathrm{d}a}\big(-\ln \operatorname{sigm}(a)\big) = -(1 - D) \to -1$ for $D \to 0$. In a program the two networks are updated in turn, and every iteration has two steps. In the discriminator step a mini-batch of real images and a mini-batch of generated images, with the generator held fixed, give the loss $\mathcal{L}_D$, and only the discriminator weights move. In the generator step new codes are drawn, the generated images pass through the discriminator, and the gradient of $\mathcal{L}_G$ moves only the generator weights, while the discriminator is held fixed. The simultaneous steps of equation (10.5) are an idealisation of this procedure. Alternating steps also circle the solution of a game, although less violently.

The second difficulty is mode collapse. The solution of the game covers every kind of image, but only against the best discriminator (10.3). In training the discriminator is a network that has seen a finite number of examples and lags behind the generator, and against this discriminator nothing in equation (10.1) rewards the generator for producing every kind of image of the data set. A generator that produces only convincing sevens can fool a discriminator for a while, the discriminator then learns to reject sevens, and the generator moves on

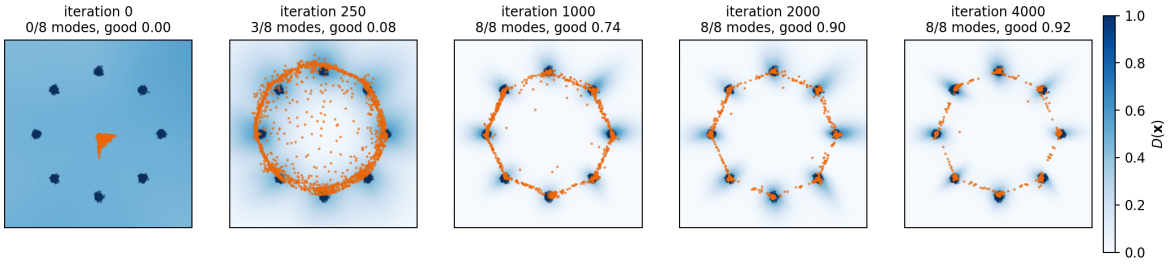


Figure 53: A generative adversarial network learning eight groups of points on a ring. Dark blue, the data. Orange, 2000 generated points. The background is the discriminator output $D(\mathbf{x})$, dark where it takes a point for real. The titles give the number of covered groups and the fraction of good generated points.

to another digit. Maximum likelihood behaves differently, since a model that gives a training image almost no probability pays a large $-\ln p_\theta(\mathbf{x})$ for it. The third difficulty is the absence of a measure of progress. The values of $\mathcal{L}_G$ and $\mathcal{L}_D$ describe the state of the game, not the quality of the images, and a GAN is judged by examining its samples, for example with a separately trained classifier. Settings found by experiment make training workable for small images, among them the Adam optimiser with step size $2 \cdot 10^{-4}$ and the momentum parameter $\beta_1 = 0.5$ [17].

10.4 Eight groups of points on a ring

A data set in the plane shows the game at work, since every generated point and the whole discriminator can be drawn. One sample is a point $\mathbf{x} \in \mathbb{R}^2$ drawn from one of eight small Gaussian groups of standard deviation 0.05, whose centres are spaced evenly on a circle of radius 2. A new batch of such points is drawn at every training step. A generated point counts as good when it lies within three standard deviations, 0.15, of some centre, and a group counts as covered when at least 1% of 2000 generated points fall near it. Points drawn from the data themselves score 8 groups and a good fraction of 0.986. Generator and discriminator are small dense networks with two hidden layers of 128 units, trained with the non-saturating loss (10.6), one step of each network per iteration and the Adam optimiser with step size 10^{-3} for both (Table 27).

Figure 53 follows the training. At the start the generator puts all its points in a small clump near the origin, and the discriminator, which has not learned anything yet, returns about one half everywhere. After 250 iterations the discriminator has marked the eight groups, and the generator has spread its points onto the circle through them, with 3 groups covered. After 1000 iterations all 8 groups are covered, and the discriminator returns high values only at the groups. At the end of 4000 iterations 92.1% of the generated points are good. The remaining points form thin bridges between neighbouring groups. The generator is a continuous map of a connected Gaussian cloud, and it cannot tear its image into eight separate pieces, so a few codes always land between the groups.

The same data with a ten times larger step size for the generator, 10^{-2}, show the instability of Section 10.3 (Figure 54). The generator now moves faster than the discriminator can follow. The number of covered groups rises to 6 and falls back, and at the end only 5 of the 8 groups are covered. Three groups receive at most one of the 2000 points, while the covered groups receive between 266 and 464. The points that are generated are good, with a good fraction of 0.93, so the samples look convincing one by one, and only a look at all of them reveals

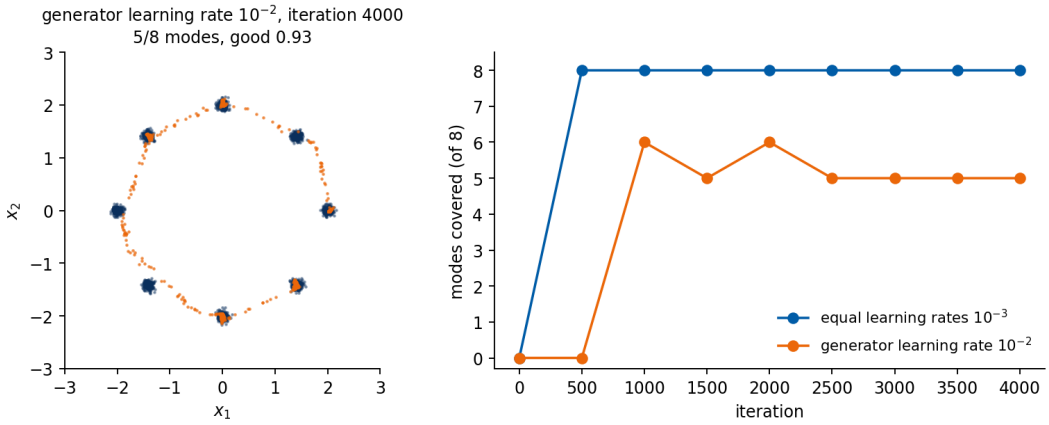


Figure 54: Partial mode collapse. Left, the data (dark blue) and 2000 points of a generator trained with a ten times larger step size (orange) after 4000 iterations. Three of the eight groups receive almost no points. Right, the number of covered groups during training for the two step sizes.

the missing groups. This is a partial mode collapse. Both runs used a single random seed, and with other seeds the run with equal step sizes ended at 7 of 8 groups instead of 8, so the outcome of a GAN training varies from run to run.

10.5 Digits from an adversarial network

The same game on the MNIST digits uses a generator that maps a code $\mathbf{z} \in \mathbb{R}^{64}$ through dense layers of 256 and 512 units to 784 pixel values, with a hyperbolic tangent at the output, so that the pixel values lie between -1 and 1, the range to which the grey values are rescaled. The discriminator maps the 784 values through 512 and 256 units to one probability. Both use the leaky rectifier $\max(0.2u, u)$, which passes a small slope for negative inputs so that no unit stops passing gradients, and they have 550 416 and 533 505 parameters. Training uses the non-saturating loss with the settings of [17] for 30 passes over the 60 000 training digits, about four minutes on a processor.

Figure 55 shows the images of eight fixed codes during training and 64 images at the end. After one pass they are grey noise with a faint digit-shaped shadow, after ten passes black strokes appear among scattered dark pixels, and after thirty passes most images are digits with sharp, black strokes. They differ visibly from the grey, blurred digits of the variational autoencoder (Figure 35). Some images are not digits at all, a dark blot in the first row, and many carry isolated black pixels in the background. The generator is never asked to explain every pixel of a training image, and stray pixels cost it nothing as long as the discriminator does not use them to recognise fakes.

The losses in Figure 56 show the game, not the quality. At the solution of the game the discriminator returns one half for every image, which gives $\mathcal{L}_D = -\ln\frac{1}{2} - \ln\frac{1}{2} = 2\ln 2$ and $\mathcal{L}_G = \ln 2$. For the first ten passes the discriminator wins, its loss falls and the generator loss rises, while the images improve from noise to strokes. Afterwards the generator catches up, and the mean discriminator output for generated images rises from 0.18 to 0.36, while that for real images falls from 0.83 to 0.64. After thirty passes the discriminator still separates the two kinds of images on average, and the game has not reached its solution. Neither loss falls steadily, and neither tells how good the digits are.

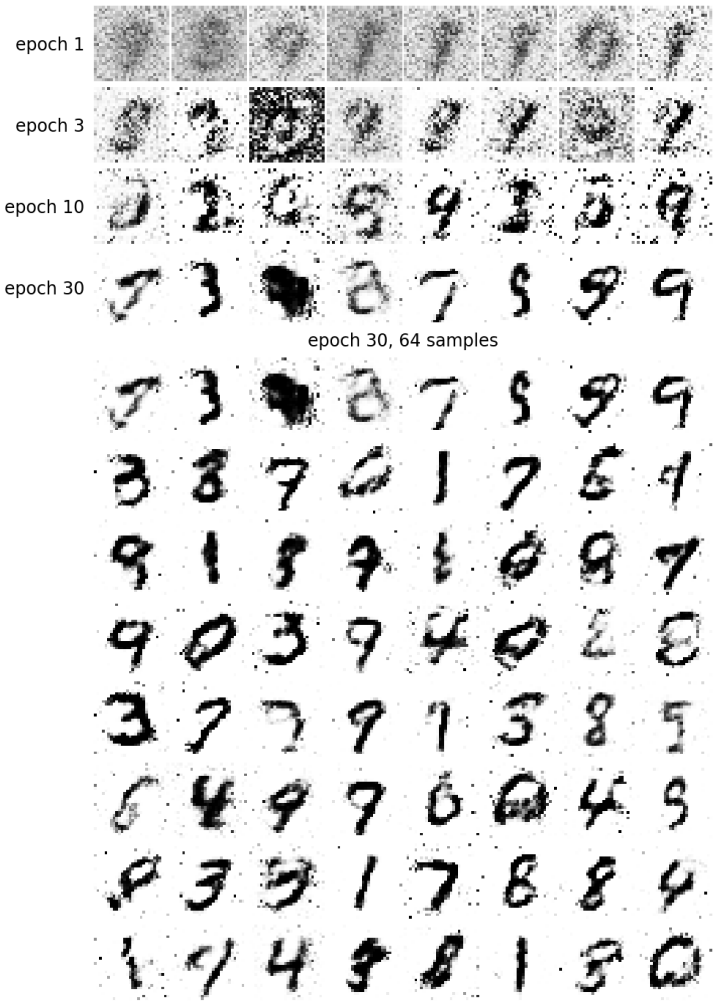


Figure 55: Digits from the generative adversarial network. Top, the images of eight fixed codes after 1, 3, 10 and 30 passes over the data. Bottom, 64 images after 30 passes.

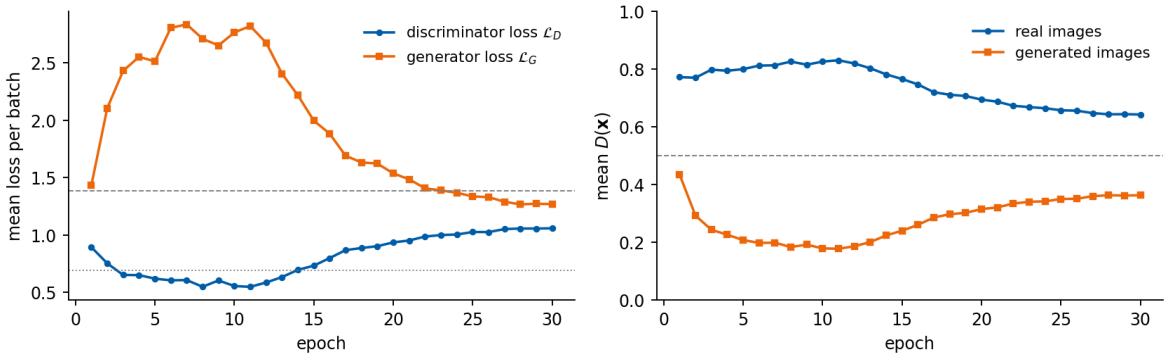


Figure 56: Training of the adversarial network on digits. Left, the mean discriminator loss $\mathcal{L}_D$ and generator loss $\mathcal{L}_G$ of equation (10.6) per pass, with their values at the solution of the game, $2\ln 2$ (dashed) and $\ln 2$ (dotted). Right, the mean discriminator output for real and for generated images, with the value one half of the solution (dashed).

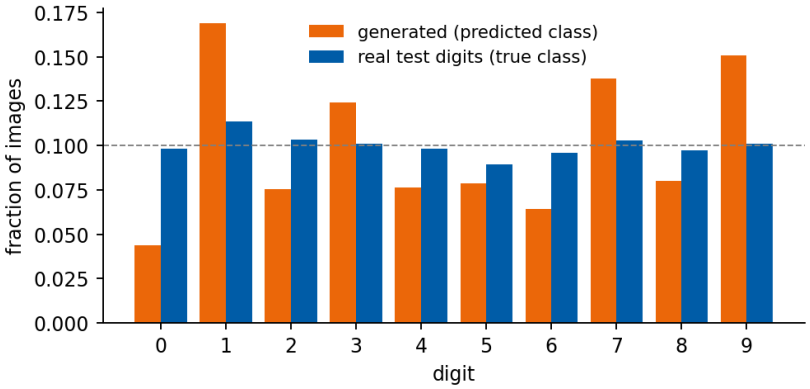


Figure 57: Classes of 10 000 generated digits, as read by a classifier trained on real digits, against the true classes of the 10 000 real test digits. The dashed line marks equal frequencies.

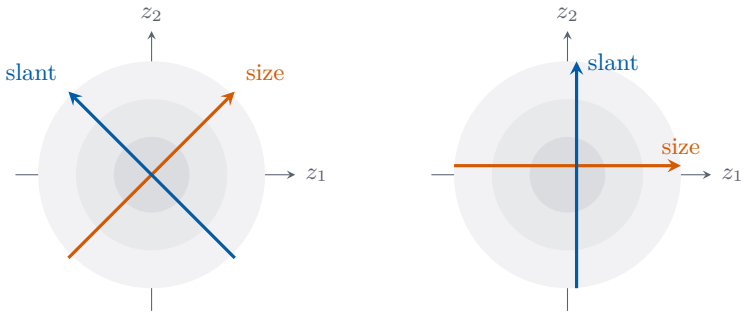
Table 20: Variational autoencoder and generative adversarial network.

	variational autoencoder	generative adversarial network
generation	$\mathbf{z} \sim \mathcal{N}(0, I)$, decoder	$\mathbf{z} \sim \mathcal{N}(0, I)$, generator
training signal	reconstruction of each training image and divergence of its cloud from the prior	judgement of a discriminator that sees real and generated images
loss	one loss, the negative evidence lower bound, minimised	a min-max game of two networks
likelihood	lower bound on $\ln p_{\boldsymbol{\theta}}(\mathbf{x})$ for every sample	none
encoder	yes, a cloud of codes for every sample	none
samples	every training image is penalised if poorly explained, which pushes towards covering the data; blurred	sharp; nothing penalises missing kinds of images, coverage can be uneven or partial
training	one loss, usually well behaved, progress visible in the loss	a game, sensitive to step sizes and seeds, progress judged from the samples

The quality is therefore judged from outside, as for the conditional model in Section 9.5. A dense classifier trained on the real training digits reaches an accuracy of 0.971 on the real test digits. It reads 10 000 generated digits with a mean confidence, its largest class probability, of 0.85, against 0.96 for real test digits, and it is more than 90% sure for 58% of the generated digits against 90% of the real ones. The generated classes are uneven (Figure 57). Ones make up 16.9% of the generated digits and nines 15.1%, while zeros make up only 4.4%, against about 10% of every class among the real digits. The generator has not lost any class, but it favours the easy thin strokes of ones and nines and produces few zeros, a mild form of the uneven coverage of the ring. The generated digits are new images and not copies. The Euclidean distance of their grey values to the nearest training image has a mean of 4.80, against 4.24 for real test digits that the networks never saw.

10.6 Adversarial networks and variational autoencoders compared

Table 20 sets the two models side by side. Both turn a code drawn from $\mathcal{N}(0, I)$ into a sample, and both are trained on samples alone. They differ in how a generated sample is judged. The variational autoencoder judges it pixel by pixel against the training image it should rebuild,



entangled: moving along z_1 changes size and slant

disentangled: each coordinate follows one factor

Figure 58: Two latent spaces of the same data with the same prior (grey). The arrows show the directions in which the decoded image changes its size and its slant. Left, the factors lie along diagonals, and every coordinate mixes them. Right, each factor has its own coordinate, and moving along z_1 changes the size alone. The density of the codes is the same round Gaussian in both cases, so the likelihood alone cannot prefer the right picture.

which gives one loss, a bound on the likelihood, an encoder and a pressure to explain every training image, at the price of blurred samples. The adversarial network judges it by a second network that looks at the whole image, which gives sharp samples, at the price of an unstable game, uneven coverage and no likelihood. Later generative models combine the two ideas or replace the game, and the diffusion models [16] now used for images, molecules and weather fields are trained by a stable loss related to the likelihood.

11 The β-weighted autoencoder and disentanglement

11.1 A price for information

The conditional model of Section 9 needs the label of every sample, and the adversarial network of Section 10 gives no handle on the meaning of its latent coordinates at all. Often the factors of interest are not labelled, as for galaxies, whose size, inclination and colour nobody has recorded for each image, or for oscillators, whose frequency and damping are what the analysis should find. The question is then whether the latent coordinates of a model trained without labels can align with such factors by themselves. The latent grid of Figure 34 shows that in a plain variational autoencoder they do not. Moving along z_1 changes the digit, its slant and its width at the same time.

The aim has a name. A latent space is disentangled when each coordinate follows one factor of the data and ignores the others (Figure 58). A rotated set of coordinates describes the data equally well, since the prior $\mathcal{N}(0, I)$ is round and looks the same in every rotated frame, so the reconstruction and the likelihood do not prefer the disentangled frame. Something else in the loss must prefer it, and the divergence term can. It charges the information stored in the code, coordinate by coordinate, against a prior whose coordinates are independent. If the price of information is raised, the code can afford only the few factors that improve the reconstruction most. A factor stored in a coordinate of its own is then cheaper than the same factor smeared over several coordinates together with other factors. Mixing factors whose distributions are not Gaussian makes the coordinates statistically dependent, and dependence costs extra nats against the independent prior. The encoder clouds, which are Gaussians with independent coordinates, add a second preference for one factor per coordinate. The weight β raises the

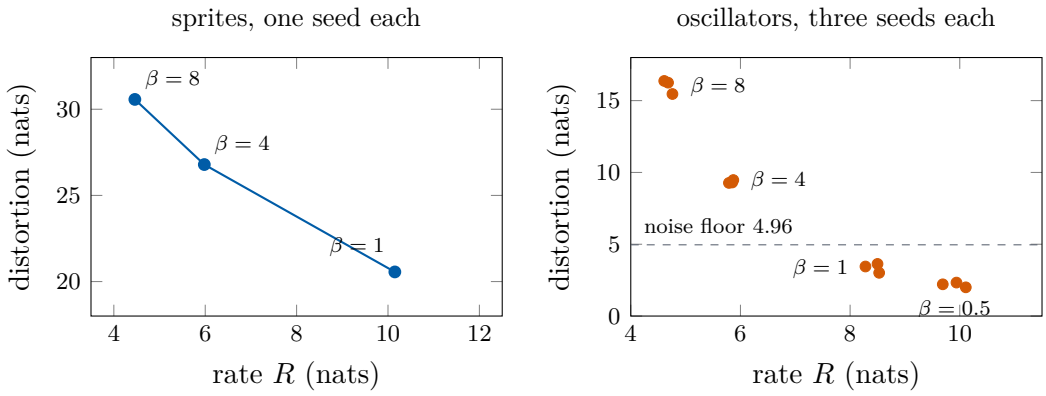


Figure 59: Rate–distortion trade-off, measured on test data. Left, β-VAEs on dSprites (Section 11.2). Right, β-VAEs on noisy oscillator trajectories (Section 12), three random seeds per value of β, with the distortion of the exact mean trajectory as the noise floor. A larger β moves a model to a lower rate and a higher distortion.

price of all information equally. It does not single out the cost of dependence, but it forces the code to be economical, and only an economical code has to exploit the cheaper arrangement. Whether this produces disentangled coordinates depends on the data. Section 11.4 makes both arguments precise, the sprites show a clear gain, and the oscillators of Section 12 show that the gain is not guaranteed [25]. The price of disentanglement is a poorer reconstruction, since the code carries less.

The β-weighted variational autoencoder, or β-VAE [19], raises the price of information by changing one number in the loss. The divergence term is multiplied by a weight β,

$$\mathcal{L}_\beta(\mathbf{x}) \coloneqq \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\big[-\ln p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\big] + \beta\ \mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}\,|\,\mathbf{x})\,\|\,p(\mathbf{z})\right). \tag{11.1}$$

For $\beta = 1$ this is the negative evidence lower bound. For $\beta > 1$ every nat of information carried by the code costs more, the code carries less, and the reconstructions become blurrier. For $\beta < 1$ the code is cheaper, the reconstructions sharpen, and the clouds drift away from the prior, as on the sine curve of Table 9.

The weight has a direct interpretation. Higgins et al. start from a constrained problem, the best reconstruction under the condition that the divergence, averaged over the data, stays under a budget ϵ_{KL}. A constraint of this kind can be replaced by a penalty, a price β charged for every nat of divergence, and the price is raised until the budget is respected,

$$\max_{\boldsymbol{\phi},\boldsymbol{\theta}}\ \mathbb{E}_{\mathbf{x}}\,\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\big[\ln p_{\boldsymbol{\theta}}(\mathbf{x}\,|\,\mathbf{z})\big] \quad \text{subject to} \quad \mathbb{E}_{\mathbf{x}}\Big[\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}\,|\,\mathbf{x})\,\|\,p(\mathbf{z})\right)\Big] < \epsilon_{\mathrm{KL}}, \tag{11.2}$$

whose Lagrangian with a multiplier $\beta \geq 0$, the objective plus β times the violated constraint, as in the method of Lagrange multipliers of mechanics, is, apart from the constant $\beta\epsilon_{\mathrm{KL}}$, minus equation (11.1). Choosing β is choosing the budget of information. The average divergence over the data,

$$R \coloneqq \mathbb{E}_{\mathbf{x}}\Big[\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}\,|\,\mathbf{x})\,\|\,p(\mathbf{z})\right)\Big], \tag{11.3}$$

is called the rate. By equation (7.5) it bounds from above the information, in nats, that the code carries about the sample. The average reconstruction term is called the distortion. Lowering one raises the other, and a family of models trained with different β traces a trade-off curve, the rate–distortion curve of information theory applied to the variational autoencoder [21]. Figure 59 shows the measured curves for two data sets of this chapter, sprites and oscillators, both introduced in the following sections.

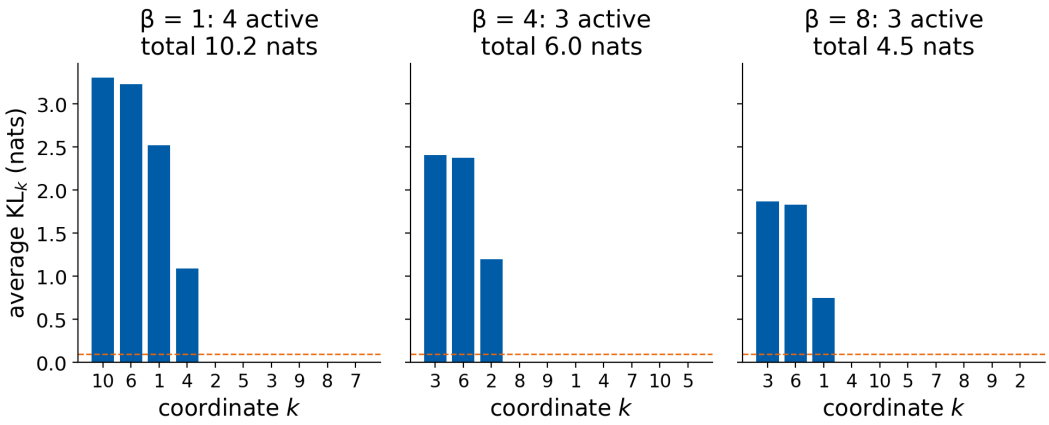


Figure 60: Average KL divergence $\overline{\mathrm{KL}}_k$ of each of the ten latent coordinates over the dSprites test images, for $\beta = 1$, 4 and 8, with the threshold of 0.1 nats for an active coordinate (dashed). Coordinates under the threshold have collapsed onto the prior.

11.2 Sprites with known factors

Whether latent coordinates align with factors can be tested only on data whose factors are known. The dSprites data set [4] was built for this test. Each image shows one white sprite on a black background and is rendered by a program from five numbers, the shape (square, ellipse or heart, 3 values), the scale (6 values from 0.5 to 1), the orientation (40 values from 0 to 2π), and the horizontal and vertical positions (32 values each). Every combination occurs once, which gives 737 280 images of 64×64 binary pixels. We use 100 000 training and 10 000 test images reduced to 32×32 pixels, on which a sprite covers on average 4.5% of the pixels. Orientation is not fully visible in the image. A square rotated by a quarter turn looks unrotated, and the angles 0 and 2π give the same image.

Following the design of Higgins et al. and Burgess et al. [19, 20], the encoder has three strided convolutions and a dense layer with $d = 10$ latent coordinates, and the decoder mirrors it with a Bernoulli output. The loss is equation (11.1), and the factors are kept for evaluation only. Three models with $\beta = 1$, 4 and 8 see the same mini-batches, so that β is the only difference between them. New sprites are generated by decoding codes drawn from $\mathcal{N}(0, I)$, and single coordinates are read by the traversals of Section 11.5.

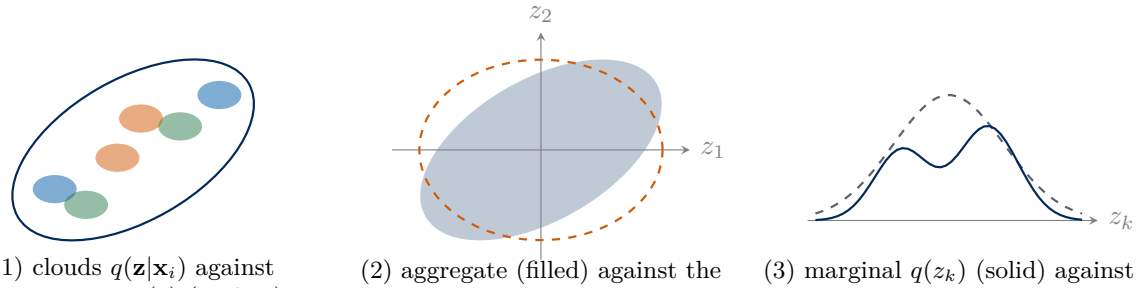
11.3 Coordinates that switch off

The divergence (7.4) is a sum over coordinates, so each coordinate pays for its own information. Its average over the test images, $\overline{\mathrm{KL}}_k$, tells which coordinates are used. A coordinate with $\overline{\mathrm{KL}}_k \approx 0$ has $\mu_k \approx 0$ and $\sigma_k \approx 1$ for every input. Its value is pure prior noise, the decoder learns to ignore it, and the coordinate is switched off. We call a coordinate active when $\overline{\mathrm{KL}}_k$ exceeds 0.1 nats.

Table 21 and Figure 60 show the result. The plain variational autoencoder uses four of its ten coordinates. With $\beta = 4$ and $\beta = 8$ three remain, two of them with nearly equal information, the other coordinates carry less than 0.01 nats, and the total information is more than halved from $\beta = 1$ to $\beta = 8$. The price is paid in reconstruction. Five factors at this resolution do not need ten coordinates, and the weight β decides how many sharp coordinates the model can afford.

Table 21: β-VAEs on dSprites, measured on the 10 000 test images. KL per active coordinate, sorted, in nats.

	$\beta = 1$	$\beta = 4$	$\beta = 8$
KL of the active coordinates	3.30, 3.23, 2.52, 1.09	2.40, 2.37, 1.20	1.87, 1.83, 0.75
active coordinates	4	3	3
total KL, rate (nats per image)	10.15	5.98	4.46
reconstruction, distortion (nats per image)	20.56	26.79	30.57
reconstruction at the centre plus KL	30.71	32.77	35.03
mutual information gap	0.150	0.295	0.301



(1) clouds $q(\mathbf{z}|\mathbf{x}_i)$ against the aggregate $q(\mathbf{z})$ (outline): information $I_q(\mathbf{x};\mathbf{z})$

(2) aggregate (filled) against the product of its marginals (dashed): total correlation

(3) marginal $q(z_k)$ (solid) against $p(z_k) = \mathcal{N}(0,1)$ (dashed): coordinate-wise mismatch

Figure 61: The three parts of the averaged divergence, equation (11.5). (1) The information carried by the codes compares each encoder cloud with the aggregate posterior. (2) The total correlation compares the aggregate posterior with the product of its own marginals and vanishes only when the coordinates are independent. (3) The last part compares each marginal with the standard Gaussian.

11.4 The mechanism of disentanglement

A representation is called disentangled when each latent coordinate responds to one generative factor of the data and is insensitive to the others, so that moving along z_k changes one property of the decoded sample. Section 8 placed this property in the aggregate posterior $q(\mathbf{z})$. Its coordinates must be statistically independent, and each must follow one factor. Two features of the loss favour such an arrangement.

The first feature is the factorised prior. $\mathcal{N}(0, I)$ has independent coordinates, and the divergence term charges each coordinate separately. Information is cheapest when the coordinates are independent and each marginal is Gaussian, which the decomposition (11.5) states, and when coordinates that carry nothing are switched off. The second feature is a decomposition of the averaged divergence that shows what it penalises [22, 23]. Let the data be the n training samples, each with probability $1/n$, and consider the joint distribution $q(\mathbf{x}, \mathbf{z}) = \frac{1}{n} q_\phi(\mathbf{z} \,|\, \mathbf{x})$ over samples and codes, whose marginal over $\mathbf{z}$ is the aggregate posterior (8.1). Let $q(z_k)$ be the marginal of coordinate k under $q(\mathbf{z})$. The prior factorises, $p(\mathbf{z}) = \prod_k p(z_k)$, and inserting $q(\mathbf{z})$ and $\prod_k q(z_k)$ into the logarithm splits the average divergence into three parts,

$$\mathbb{E}_{\mathbf{x}}\big[\,\mathrm{KL}\left(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\right)\big] = \mathbb{E}_{q(\mathbf{x},\mathbf{z})}\Big[\ln \frac{q_\phi(\mathbf{z}|\mathbf{x})}{q(\mathbf{z})} + \ln \frac{q(\mathbf{z})}{\prod_k q(z_k)} + \ln \frac{\prod_k q(z_k)}{\prod_k p(z_k)}\Big]. \tag{11.4}$$

Decomposition of the averaged divergence. *With the notation of equation* (11.4),

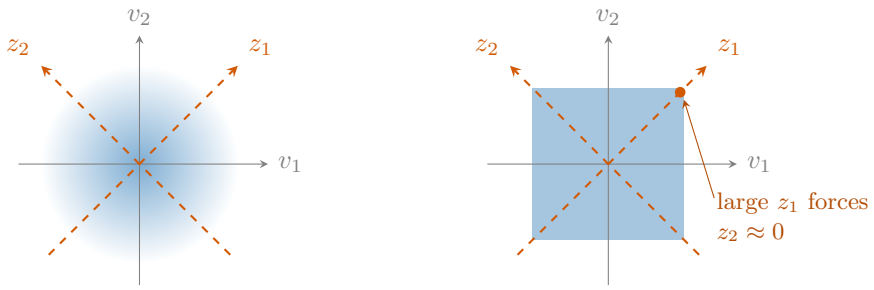
$$\mathbb{E}_{\mathbf{x}}\big[\,\mathrm{KL}\left(q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})\,\|\,p(\mathbf{z})\right)\big] = \underbrace{I_q(\mathbf{x};\mathbf{z})}_{\textit{information in the code}} + \underbrace{\mathrm{KL}\left(q(\mathbf{z})\,\Big\|\,\prod_k q(z_k)\right)}_{\textit{total correlation } \mathrm{TC}(q(\mathbf{z}))} + \underbrace{\sum_k \mathrm{KL}\left(q(z_k)\,\|\,p(z_k)\right)}_{\textit{coordinate-wise mismatch}}. \tag{11.5}$$

The first term is the mutual information of equation (7.5), and the last two terms split the divergence $\mathrm{KL}(q(\mathbf{z})\,\|\,p(\mathbf{z}))$ of that equation further. Figure 61 draws the three comparisons. The second term is the divergence of the aggregate posterior from the product of its own marginals, called the total correlation. It vanishes only when the coordinates of the codes are statistically independent over the data, which is the first requirement of disentanglement, and it captures every kind of dependence, including nonlinear ones. The third term vanishes when each coordinate, taken alone, is distributed as a standard Gaussian. Raising β raises the price of all three. The pressure on the total correlation pushes the aggregate posterior towards independent coordinates, while the pressure on the information reduces what the code carries and blurs the reconstructions.

Independence alone does not single out the axes. The prior $\mathcal{N}(0, I)$ is unchanged by a rotation $\mathbf{z} \to R\mathbf{z}$ with an orthogonal matrix R, so a decoder $g_{\boldsymbol{\theta}}(R^\top \mathbf{z})$ applied to rotated codes defines the same model of the data, with the same likelihood. Factors placed on the axes and factors placed along rotated directions are equally good for the likelihood. What breaks this symmetry is the diagonal encoder cloud. If the true posterior of a sample is narrow along one direction and wide along another, as for the short bar of Figure 37, an axis-aligned Gaussian approximates it well only when these directions are the coordinate axes. The training therefore tends to align the axes with the directions along which the data constrain the code with different precision, an effect that makes variational autoencoders resemble PCA locally [24]. This is an inductive bias of the model, an assumption built into it that favours some solutions over others, and it gives no guarantee.

The shape of the factors themselves decides whether independent coordinates can be the factors. Suppose the data are produced by two independent factors and the code holds them through some rotation. If both factors are Gaussian with equal variances, which can always be arranged by rescaling them, any rotation of them is again a pair of independent Gaussians, and no criterion of independence can tell the original axes from rotated ones. If the factors are not Gaussian, as most physical factors are, for example positions distributed uniformly or a shape that takes a few discrete values, a rotation destroys the independence (Figure 62). Two independent uniform factors fill a square, and in rotated coordinates the square becomes a diamond, in which a large value of one coordinate forces the other close to zero. Only the original axes, up to their order, their signs and a monotone change of scale of each coordinate, give independent coordinates. This is the principle of independent component analysis [27, 28], and it explains why a pressure towards independence can recover physical factors. It also explains why it can fail. Factors that are close to Gaussian leave the axes undetermined, and the decoder is nonlinear, unlike the linear mixing of independent component analysis.

The decomposition (11.5) pushes every marginal $q(z_k)$ towards a Gaussian, while a physical factor may be uniform. The two are compatible, because the decoder can apply a monotone map to each coordinate. On the sine curve the decoder turned a Gaussian z into the uniform position t through $T(z) = 4\Phi(z) - 2$, equation (7.9). In the same way a Gaussian coordinate z_k represents a uniform factor $v = \Phi(z_k)$, and since a map applied to one coordinate alone leaves the coordinates independent, the factor stays in its own coordinate (Figure 63). Recovering a factor therefore means recovering it up to such a monotone map.



Gaussian factors: rotated coordinates are again independent

uniform factors: only the original axes give independent coordinates

Figure 62: Why non-Gaussian factors fix the axes. Left, two independent Gaussian factors fill a circular cloud, and the coordinates along any rotated pair of axes (dashed) are again independent. Right, two independent uniform factors fill a square. Along the rotated axes the square is a diamond, and a large z_1 forces z_2 close to zero, so the rotated coordinates are dependent.

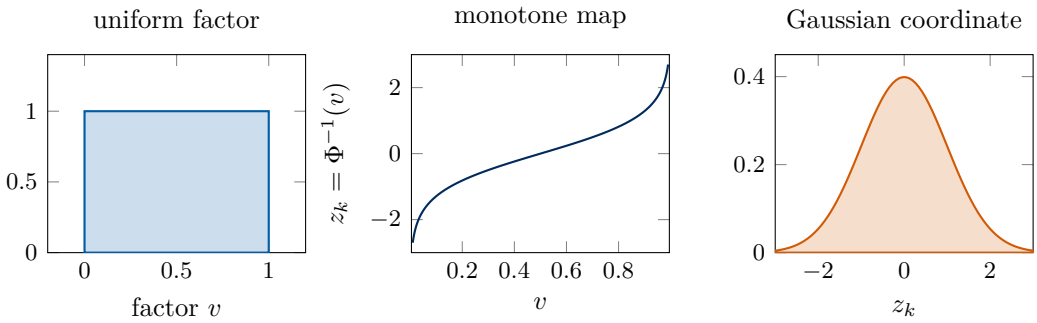
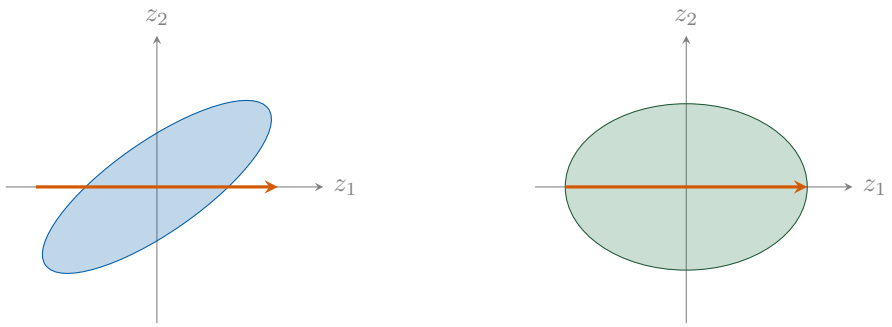


Figure 63: A uniform factor carried by a Gaussian coordinate. The monotone map Φ^{-1}, the inverse of the Gaussian distribution function, turns a uniform v into a standard Gaussian z_k, and the decoder applies the reverse map Φ, as the decoder of the sine curve does in Figure 33. The middle curve is drawn with the logistic approximation of Φ.



entangled: tilted aggregate

disentangled: axis-aligned, independent



traversal of z_1: size and position change

traversal of z_1: only the size changes

Figure 64: Entangled and disentangled aggregate posteriors, with a traversal of z_1 (orange arrow) decoded under each. Left, the codes form a tilted cloud and z_1 mixes two factors of the sprite, size and horizontal position. Right, the cloud is aligned with the axes and z_1 controls the size alone.

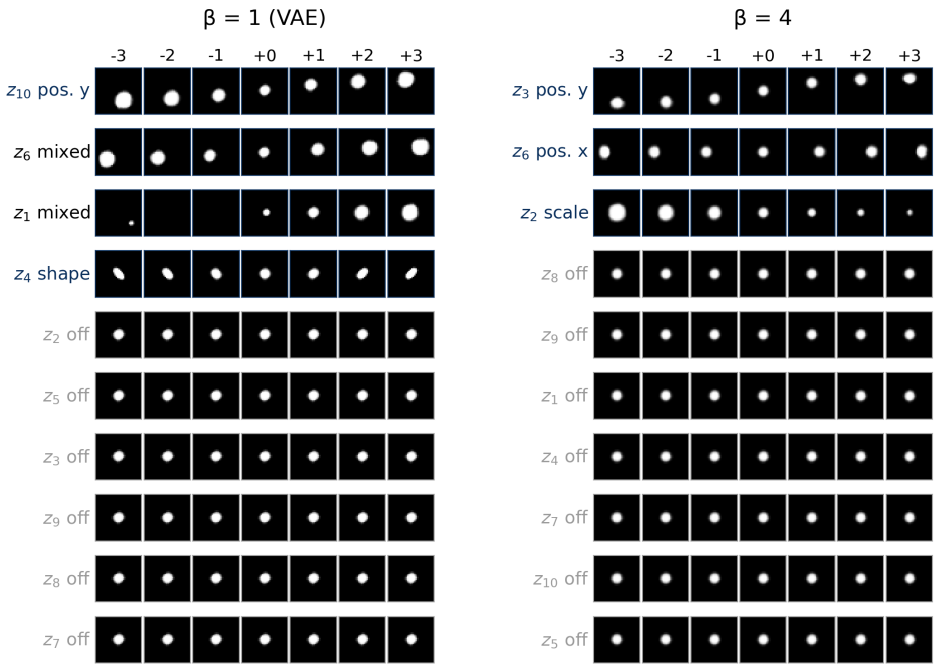


Figure 65: Latent traversals on dSprites. One test image is encoded, one coordinate is moved from -3 to $+3$ with the others fixed, and each code is decoded. Rows are coordinates sorted by their average KL and labelled by the factor they align with, "mixed" when no single factor dominates and "off" when the coordinate is switched off. Left, $\beta = 1$. Right, $\beta = 4$, with one row each for vertical position, horizontal position and scale.

Locatello et al. proved that without inductive biases on the model and on the data, unsupervised disentanglement cannot be identified from the data, since infinitely many transformations of the latent space leave the distribution of the data unchanged, and in more than 12 000 trained models they found that the outcome depends strongly on the random seed and the hyperparameters [25]. Disentanglement found by a β-VAE is therefore a hypothesis about the data to be checked against known or measured factors.

Figure 64 draws the difference. In an entangled aggregate posterior the codes form a tilted cloud, and moving along z_1 changes two factors at once, for example the size and the position of a sprite. In a disentangled one the cloud is aligned with the axes, and moving along z_1 changes only the size. The movement along one coordinate with the others held fixed is called a latent traversal, and it is the main tool for reading a latent space.

11.5 Measuring the alignment

For dSprites the factors v_j of every test image are known, and the alignment of coordinates and factors can be measured with two quantities of information theory. The entropy of a factor,

$$H(v_j) \coloneqq -\sum_{b} p(b) \ln p(b), \tag{11.6}$$

is the average surprise of its values b and equals $\ln n_v$ for n_v equally frequent values, 1.10 nats for the shape, 1.79 for the scale, 3.47 for each position and 3.69 for the orientation. The

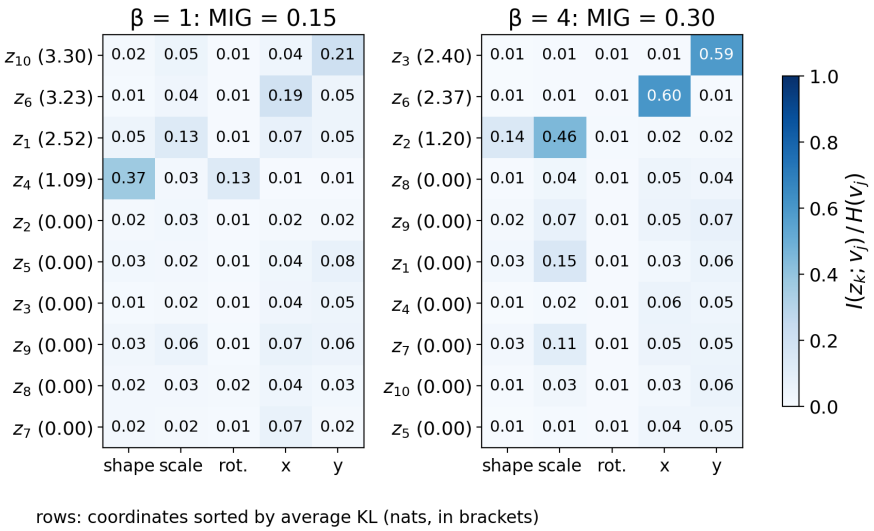


Figure 66: Normalised mutual information $I(z_k; v_j)/H(v_j)$ between latent coordinates (rows, sorted by KL) and the five factors of dSprites (columns), for the trained models. A disentangled representation has one large entry per factor, each in a different row.

mutual information of a coordinate and a factor,

$$I(z_k; v_j) := \sum_{a,b} p(a,b) \ln \frac{p(a,b)}{p(a)\,p(b)}, \tag{11.7}$$

estimated from the frequencies of 20 bins a of the encoder centre μ_k and the values b of the factor, is the reduction of the uncertainty about the factor once the coordinate is known. It vanishes when the two are independent and reaches $H(v_j)$ when the coordinate determines the factor, and it detects nonlinear relations that a correlation coefficient misses. Divided by the entropy it lies between 0 and 1. The mutual information gap [23] averages over the K factors the difference between the normalised information of the best coordinate $z_{(1)}$ and of the second best $z_{(2)}$,

$$\text{MIG} := \frac{1}{K} \sum_{j=1}^{K} \frac{I(z_{(1)}; v_j) - I(z_{(2)}; v_j)}{H(v_j)}. \tag{11.8}$$

It is large only when each factor sits in one coordinate. A factor copied into two coordinates, or spread over two, scores close to zero.

The gap doubles from 0.150 for the plain model to 0.295 for $\beta = 4$ and 0.301 for $\beta = 8$ (Table 21). For $\beta = 4$ one coordinate carries most of the information about the horizontal position (normalised mutual information 0.60), one the vertical position (0.59) and one the scale (0.46), and each carries almost nothing about the other factors (Figure 66). The traversals of Figure 65 make this visible. For $\beta = 4$ the three active rows move the sprite vertically, move it horizontally, and change its size, and the seven collapsed rows change nothing. For $\beta = 1$ the first coordinate is labelled with the vertical position, which it follows best, yet with a normalised mutual information of only 0.21, and it moves the sprite and enlarges it at the same time, while the others mix size, position and shape.

The plain model holds the same information in another arrangement. A linear regression from its ten centres to the position recovers it with a coefficient of determination R^2, the fraction of the variance of the position that the regression explains, of 0.93 and 0.94, against 0.97 and 0.97 for $\beta = 4$, yet its best single coordinate carries only 0.21 of the vertical position. No

Table 22: β-VAEs with ten coordinates on MNIST. Reconstruction in nats per test image, and Pearson correlation r of the best and second-best coordinate with three properties measured from the ink distribution.

	$\beta = 1$	$\beta = 4$
active coordinates	9	6
reconstruction (nats)	83.4	102.7
total ink, best and second r	-0.70, -0.39	-0.68, -0.52
width, best and second r	-0.82, -0.27	-0.76, -0.47
slant, best and second r	$+0.69$, $+0.51$	$+0.75$, $+0.52$

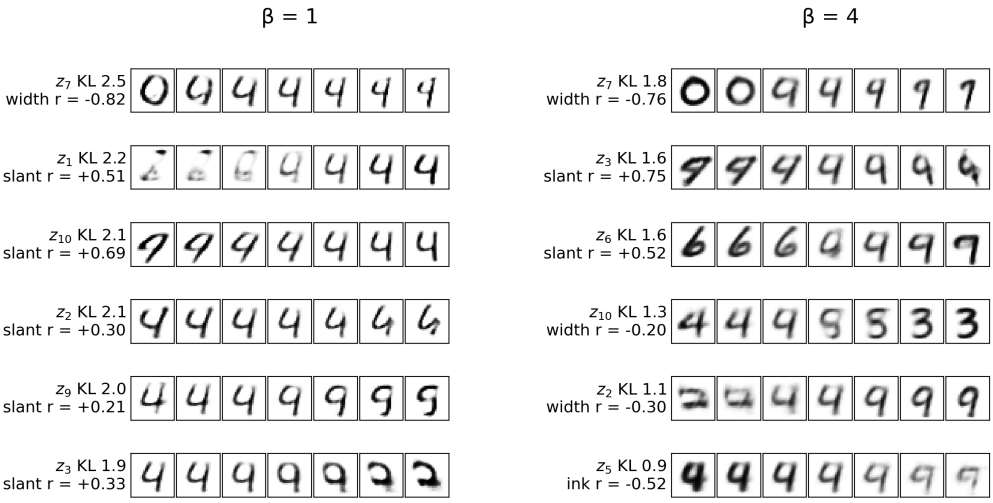


Figure 67: Latent traversals of β-VAEs on MNIST for $\beta = 1$ and $\beta = 4$. Each row moves one coordinate of an encoded test digit, and the rows shown are a subset of the active coordinates, nine for $\beta = 1$ and six for $\beta = 4$. Width, slant and thickness change together with the identity of the digit.

model learns shape or orientation well. The best shape coordinate, in the model with $\beta = 1$, reaches a normalised mutual information of 0.37 (Figure 66), and the orientation, which the image does not fully reveal, has a mutual information gap close to zero. The cost of the more interpretable axes is the reconstruction. From $\beta = 1$ to $\beta = 4$ the code loses more nats of reconstruction than it saves in information, and the negative bound rises (Table 21), so as a model of the data the β-VAE is worse.

The same experiment on the handwritten digits, with the dense network of Figure 27, ten coordinates and ten passes over the data, shows the limits of the method when the factors are not independent. Measured properties of each digit, the total ink $\mathcal{I}$ and the slant s of equation (9.6) and the width of the ink distribution, each correlate best with one coordinate (Table 22), but the second-best coordinates are not small, and the traversals of Figure 67 change the digit class together with the style. The largest factor of variation of digits is the class, which is discrete and not independent of the style, and Gaussian coordinates cannot hold it in one axis.

11.6 Galaxies

Galaxy images pose the question for real data whose factors are not listed. The Galaxy10 DECaLS data set [5] contains 17 736 colour images of galaxies from the DESI Legacy Imaging

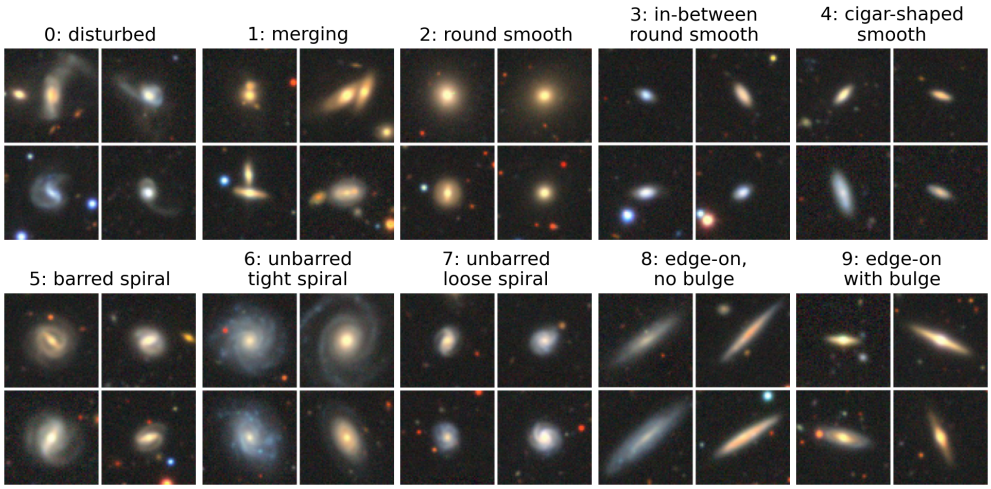


Figure 68: Examples of the ten classes of Galaxy10 DECaLS at 64×64 pixels. Red-yellow light comes from old stellar populations, blue light from star-forming regions.

Surveys [7], taken in the g, r and z bands and stored as display images of 256×256 pixels, each labelled with one of ten morphological classes from the votes of Galaxy Zoo volunteers [6]. The classes include round, in-between and cigar-shaped smooth galaxies, barred and unbarred spirals with tight or loose arms, edge-on disks with and without a bulge, and disturbed and merging systems, from 334 cigar-shaped to 2645 round smooth galaxies. We cut out the central 128×128 pixels and average blocks of 2×2, which gives images of $64 \times 64 \times 3$ values, and keep 15 076 for training and 2660 for testing (Figure 68). The labels are not used by the β-VAE.

The name of a latent coordinate, such as "size", is a hypothesis to be tested. To test it, properties of each galaxy are measured from its pixels without labels. A mask of connected pixels brighter than the sky by three standard deviations of the sky noise defines the galaxy, and from the sky-subtracted brightness $f(\mathbf{r})$ inside it we compute the apparent size (the area of the mask), the total brightness, the half-light radius r_{50}, the colour index from the red and blue channels, the concentration r_{90}/r_{50}, the offset of the light centre from the image centre, and the shape from the matrix of second moments

$$M_{ab} := \frac{\sum_{\mathbf{r}} f(\mathbf{r})(r_a - \bar{r}_a)(r_b - \bar{r}_b)}{\sum_{\mathbf{r}} f(\mathbf{r})}, \qquad \frac{b_{\mathrm{g}}}{a_{\mathrm{g}}} = \sqrt{\frac{\lambda_{\min}}{\lambda_{\max}}}, \qquad \epsilon_1 = \varepsilon_{\mathrm{g}} \cos 2\vartheta, \quad \epsilon_2 = \varepsilon_{\mathrm{g}} \sin 2\vartheta, \tag{11.9}$$

where $\lambda_{\min} \le \lambda_{\max}$ are the eigenvalues of M, $b_{\mathrm{g}}/a_{\mathrm{g}}$ is the axis ratio, ϑ the orientation of the major axis, $\varepsilon_{\mathrm{g}} = (1 - b_{\mathrm{g}}/a_{\mathrm{g}})/(1 + b_{\mathrm{g}}/a_{\mathrm{g}})$ the ellipticity, and ϵ_1, ϵ_2 its two components, along the image axes and along the diagonals. The orientation alone enters through $\cos 2\vartheta$ and $\sin 2\vartheta$, since ϑ and $\vartheta + \pi$ describe the same orientation. The proxies behave as an astronomer expects. Round smooth galaxies have a median axis ratio of 0.92 and edge-on disks without a bulge 0.23.

The β-VAE is convolutional with $d = 10$ (Table 27). Its decoder distribution is a Gaussian of width $\sigma_x = 0.1$, since the pixel values are intensities, and the loss is equation (11.1) with the squared error of equation (3.11). Three models are trained, $\beta = 1$ and $\beta = 4$ with one seed and $\beta = 4$ with a second seed. Traversals and counterfactuals are produced by decoding shifted codes of real galaxies. All three models reconstruct the test galaxies with a root-mean-square error of about 0.058 per pixel. Unlike on dSprites, all ten coordinates stay active in every model, with between 1.9 and 4.0 nats each.

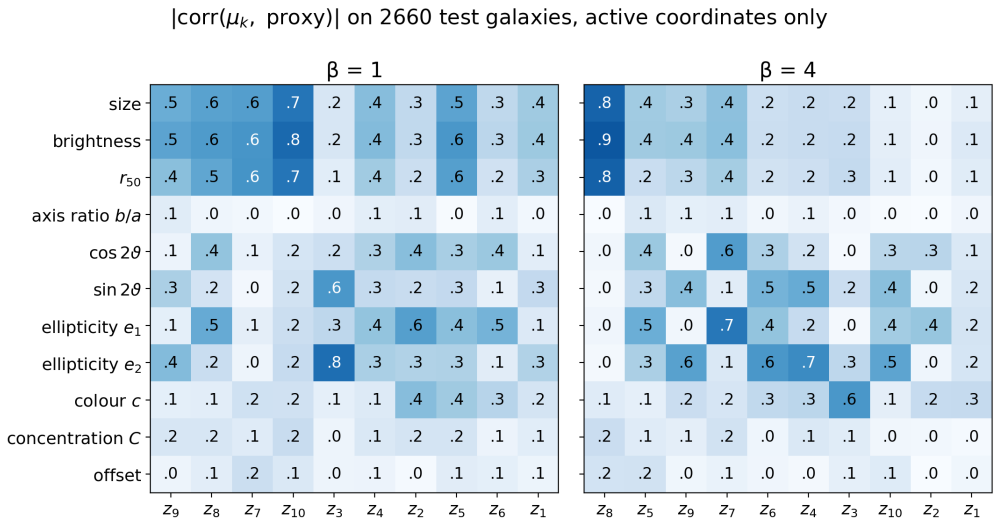


Figure 69: Absolute correlation between the encoder centre μ_k of each coordinate and each measured proxy over the test galaxies, for the β-VAEs with $\beta = 1$ and $\beta = 4$. With $\beta = 4$ the brightness and size family is concentrated in one coordinate.

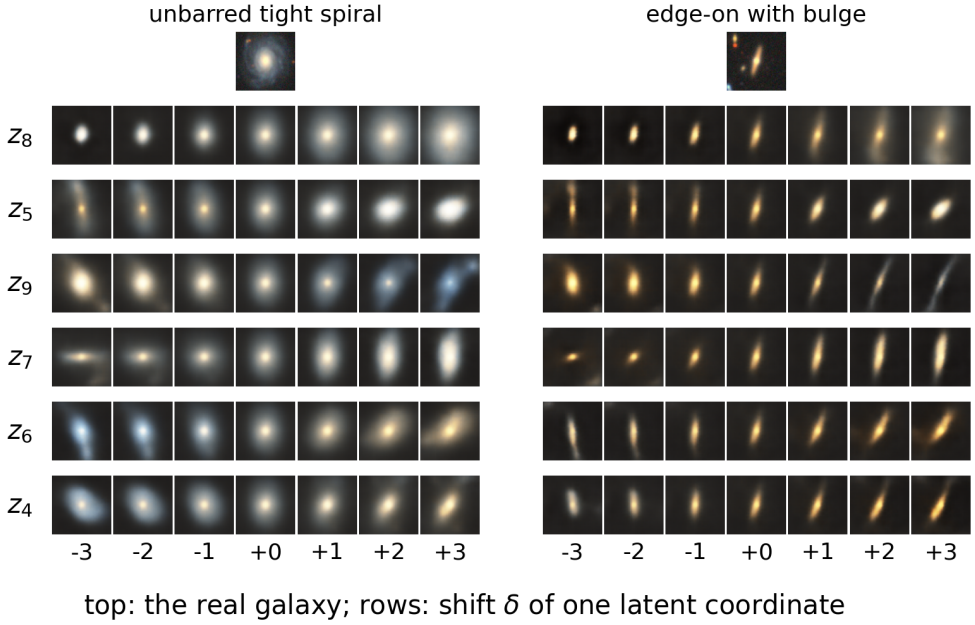


Figure 70: Traversals of the β-VAE with $\beta = 4$ on two test galaxies (real images on top). Each row shifts one coordinate by δ from -3 to $+3$ from the code of the galaxy. Along z_8 a compact core grows into a large bright disk, along z_7 the elongation turns between horizontal and vertical, along z_5 a face-on spiral becomes an elongated disk. No coordinate draws spiral arms or a bar.

Table 23: Latent coordinates of β-VAEs on galaxies and the measured proxies, test set. Correlations are absolute Pearson correlations.

quantity	value
rate, an upper bound on the information in the code, $\beta = 1$ and $\beta = 4$ (two seeds)	37.2, 26.3 and 27.2 nats
mixed coordinates, $\beta = 1$	6 of 10
$\beta = 4$, z_8: brightness, size, r_{50}	0.85, 0.80, 0.81
$\beta = 4$, z_7: elongation ϵ_1	0.72
$\beta = 4$, z_4, z_6, z_9, z_{10}: elongation ϵ_2	0.65, 0.60, 0.56, 0.54
$\beta = 4$, z_3: colour	0.60
second seed, partner of the brightness coordinate	0.97
second seed, partners of elongation and colour coordinates	0.53 to 0.79

The traversals of Figure 70 suggest names, and the correlations of each coordinate with each proxy over the 2660 test galaxies test them (Figure 69). A coordinate is named after a proxy only when its largest absolute correlation is at least 0.5 and exceeds its correlation with every other proxy by at least 0.15, size, brightness and r_{50} counting as one family. Table 23 collects the outcome. For $\beta = 4$ one coordinate follows brightness, size and half-light radius and almost nothing else, others follow the two components of the elongation and the colour, while for $\beta = 1$ most coordinates are mixed. A factor found once may be an accident of the initialisation, so the model with the second seed is compared coordinate by coordinate. Each coordinate of the first model is paired with the coordinate of the second that correlates with it most. Only brightness and size reappear as a single coordinate. Elongation and colour are present in both models with similar strength but spread differently over the coordinates.

A disentangled latent space can also be used to question a classifier, by asking how its decision changes along one coordinate (Appendix C).

On galaxies the β-VAE thus finds one robust physical factor without labels, brightness and size, and candidates for elongation and colour that do not reproduce across seeds. Fine structure such as arms and bars is not represented at 64×64 pixels.

12 Physical parameters from trajectories

12.1 Oscillators with known parameters

On galaxies the true factors are unknown, and every latent coordinate had to be compared with proxies measured from the images. Simulated physical systems remove this uncertainty, since the parameters of every sample are known by construction. A physicist summarises a mass on a spring moving in a fluid by a few numbers, the frequency of the oscillation, the rate at which it dies out, and the initial position and velocity. Iten et al. [8] asked whether a network that sees only recorded motions finds such numbers in its latent neurons, and the same question can be asked with the β-VAE and with their network.

A particle of unit mass on a spring, slowed by friction and hit at random by the molecules of the fluid, obeys the Langevin equation

$$\ddot{x} = -\omega^2 x - \gamma \dot{x} + \sqrt{2\Lambda}\,\xi(t), \qquad \langle \xi(t) \rangle = 0, \quad \langle \xi(t)\xi(t') \rangle = \delta(t - t'), \tag{12.1}$$

where ω is the angular frequency without friction, γ the friction rate, $\xi(t)$ Gaussian white noise, a random force whose values at different times are independent, and Λ its strength.

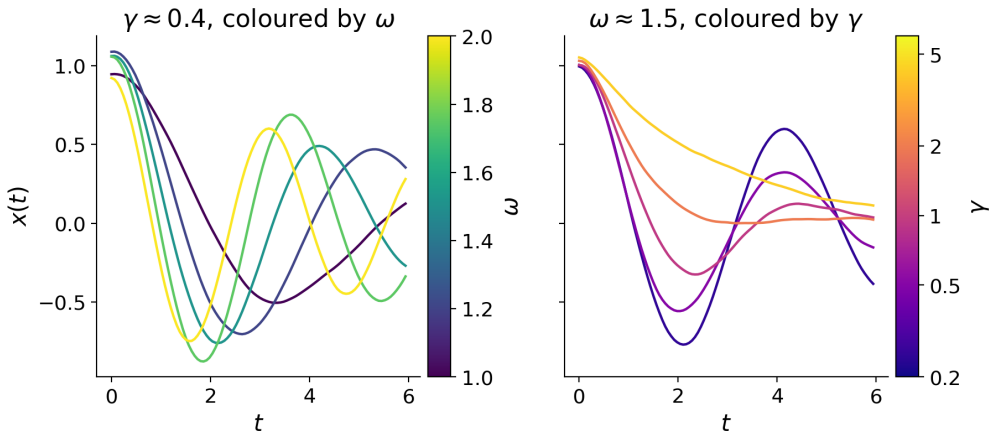


Figure 71: Test trajectories of noisy damped oscillators. Left, the friction rate is near 0.4 while the frequency varies from 1 to 2. Right, the frequency is near 1.5 while the friction rate varies from 0.25 to 5.88, from a slowly decaying oscillation to an overdamped creep. The colour gives the varied factor.

Without noise the ansatz $x \propto e^{\lambda t}$ gives the rates $\lambda_\pm = -\gamma/2 \pm \sqrt{\gamma^2/4 - \omega^2}$. For $\gamma < 2\omega$ the particle oscillates inside a decaying envelope, and for $\gamma > 2\omega$ it creeps back to the origin without oscillating. Deep in this overdamped regime the slower rate is

$$|\lambda_+| \approx \frac{\omega^2}{\gamma} + \frac{\omega^4}{\gamma^3}, \tag{12.2}$$

so that a trajectory depends on ω and γ essentially only through the combination ω^2/γ.

One sample of the data set is the trajectory of one oscillator, its positions at 100 equally spaced times,

$$\mathbf{x}_i = (x_i(t_0), \ldots, x_i(t_{99})) \in \mathbb{R}^{100}, \qquad t_j = 0.06\, j, \tag{12.3}$$

covering the window from 0 to 5.94. Each trajectory has its own four factors, drawn independently,

$$\omega \sim \mathcal{U}[1, 2], \qquad \ln\gamma \sim \mathcal{U}[\ln 0.2, \ln 6], \qquad x_0 \sim \mathcal{U}[0.5, 1.5], \qquad v_0 \sim \mathcal{U}[-0.5, 0.5], \tag{12.4}$$

where x_0 and v_0 are the initial position and velocity. Only ω and γ describe the oscillator. The initial conditions change the trajectory without describing the system and are called nuisance factors. The noise strength is $\Lambda = 0.002$ for all trajectories. The training set has 20 000 trajectories and the test set 5000, about a fifth of them overdamped. The random force moves a trajectory away from its mean by about 0.03, a small but visible perturbation compared with the typical amplitude of 0.4. The factors are stored for evaluation only and never reach the networks (Figure 71).

12.2 Two networks

Two networks with the encoder–latent–decoder structure of this chapter are applied to trajectories, and Figure 72 draws them side by side. The first is a β-VAE that reconstructs the

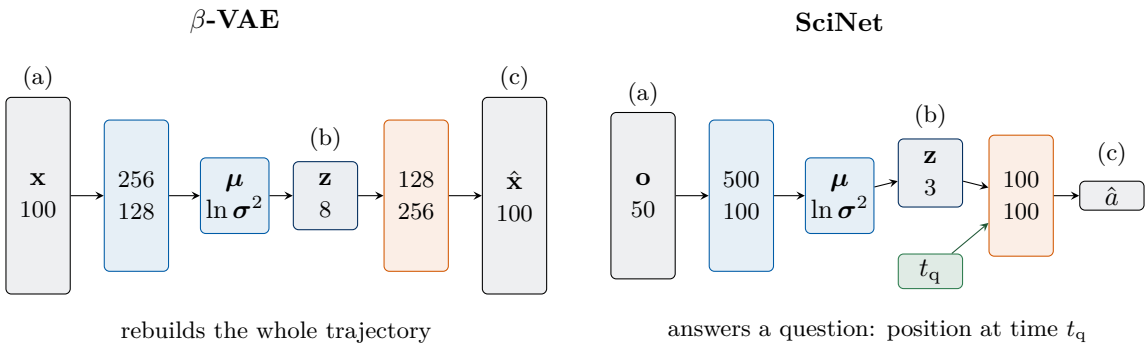


Figure 72: The two networks applied to trajectories. Left, the β-VAE of equations (12.5), which encodes a trajectory of 100 positions into 8 coordinates and rebuilds it. Right, SciNet of equations (12.6), which encodes an observed segment of 50 positions into 3 latent neurons and, given a question time t_q (green), predicts the position at that time. Labels (a) to (c) refer to the first three lines of each set of equations.

trajectory,

$$(\boldsymbol{\mu}, \ln \boldsymbol{\sigma}^2) = f_{\boldsymbol{\phi}}(\mathbf{x}), \qquad \mathbf{x} \in \mathbb{R}^{100}, \quad \boldsymbol{\mu}, \ln \boldsymbol{\sigma}^2 \in \mathbb{R}^8, \tag{12.5a}$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \tag{12.5b}$$

$$\hat{\mathbf{x}} = g_{\boldsymbol{\theta}}(\mathbf{z}) \in \mathbb{R}^{100}, \tag{12.5c}$$

$$\mathcal{L}_\beta = \frac{1}{2\sigma_x^2} \|\mathbf{x} - \hat{\mathbf{x}}\|^2 + \beta \sum_{k=1}^{8} \mathrm{KL}_k, \qquad \sigma_x = 0.1. \tag{12.5d}$$

The encoder (12.5a) has hidden layers of 256 and 128 units with ReLU, the sampling step (12.5b) is the reparametrisation of Section 7.3, and the decoder (12.5c) mirrors the encoder and returns the mean trajectory. The loss (12.5d) is the β-weighted bound with a Gaussian decoder, equation (3.11), whose width $\sigma_x = 0.1$ is about three times the deviation caused by the noise, so that storing the noise gains little.

The second network is SciNet of Iten et al. [8]. It is modelled on how a physicist works. Observations are compressed into a few parameters, and the parameters are used to answer questions about the system without returning to the raw data. The encoder maps an observation to latent neurons, a question is appended to them, and the decoder returns the answer,

$$(\boldsymbol{\mu}, \ln \boldsymbol{\sigma}^2) = f_{\boldsymbol{\phi}}(\mathbf{o}), \qquad \mathbf{o} \in \mathbb{R}^{50}, \quad \boldsymbol{\mu}, \ln \boldsymbol{\sigma}^2 \in \mathbb{R}^3, \tag{12.6a}$$

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \tag{12.6b}$$

$$\hat{a} = g_{\boldsymbol{\theta}}([\mathbf{z}, t_\mathrm{q}]) \in \mathbb{R}, \tag{12.6c}$$

$$C_\beta = \big(\hat{a} - x(t_\mathrm{q})\big)^2 + \beta \sum_{k=1}^{3} \mathrm{KL}_k, \qquad \beta = 10^{-3}. \tag{12.6d}$$

The observation $\mathbf{o}$ of equation (12.6a) is the first part of a trajectory, the question t_q in equation (12.6c) is a time, possibly beyond the observed window, and the answer $\hat{a}$ is the predicted position at that time. Encoder and decoder are small dense networks, as in the original paper. The cost (12.6d) is the β-VAE cost of Iten et al. with the reconstruction measured on the answer. An autoencoder is the special case of SciNet with the single question "repeat the observation". The latent neurons of SciNet must hold only what is needed to answer the questions, and the divergence term pushes them towards independence and economy, as in Section 11.4.

Table 24: β-VAEs on oscillator trajectories, test set. Number of active coordinates in each of three seeds, the largest absolute correlation of each factor with an active coordinate averaged over the seeds, and the mean mutual information gap. The last row gives the median visibility of each factor, the change of the trajectory in units of σ_x for a change of the factor by one standard deviation.

β	active	ω	$\ln\gamma$	x_0	v_0	MIG
0.5	4, 4, 4	0.60	0.80	0.76	0.67	0.092
1	4, 4, 4	0.60	0.82	0.73	0.68	0.098
4	4, 4, 4	0.51	0.82	0.58	0.56	0.045
8	3, 3, 3	0.50	0.83	0.56	0.24	0.043
true factors						0.916
visibility, median		16.7	23.3	11.9	5.4	

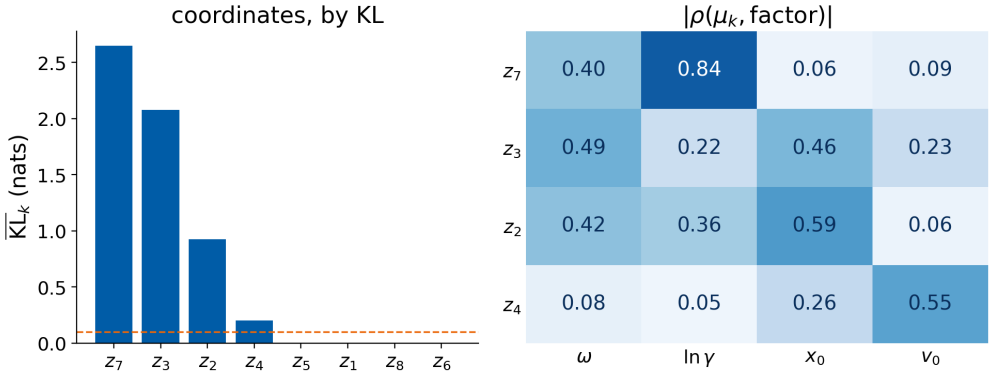


Figure 73: The β-VAE with $\beta = 4$ and seed 0 on oscillator trajectories. Left, average KL of each coordinate over the test trajectories, with the threshold of 0.1 nats (dashed). Four coordinates are active. Right, absolute correlation of the active coordinates with the four factors.

12.3 What the β-VAE stores

We train the β-VAE (12.5) on the 20 000 training trajectories (Table 27), with $\beta = 0.5$, 1, 4 and 8 and three random seeds each, twelve models in all. The rates and distortions of Figure 59 show the trade-off. For β of 0.5 and 1 the distortion is smaller than the noise floor of 4.96 nats, the distortion of the exact mean trajectory, so the code stores part of the particular realisation of the noise. For β of 4 and 8 it lies above, and the code describes smooth trajectories.

Without being told, the network uses as many coordinates as there are factors. For β of 0.5, 1 and 4 every seed has four active coordinates and four switched off. At $\beta = 8$ the weakest coordinate falls under the threshold and three remain. Table 24 compares the active coordinates with the factors through the largest absolute Pearson correlation of each factor with any active coordinate, and through the mutual information gap. The damping is the factor found best for every β, the frequency never gets a coordinate of its own, and the initial velocity is dropped at $\beta = 8$. The gap stays far from the value that the true factors would reach, and larger β does not raise it here. Figure 73 shows the model with $\beta = 4$ and seed 0, whose most informative coordinate z_7 follows $\ln\gamma$ with $|\rho| = 0.84$, while z_3 mixes the frequency (0.49) with the initial position (0.46).

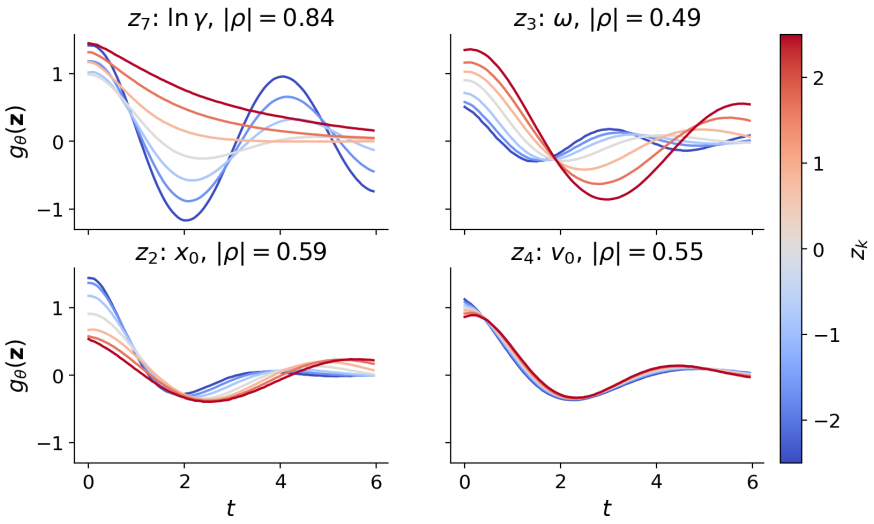


Figure 74: Traversals of the four active coordinates of the model of Figure 73, starting from a test trajectory with $\omega = 1.48$, $\gamma = 1.06$, $x_0 = 1.01$ and $v_0 = -0.01$. Each panel shows the decoded mean trajectory while one coordinate moves from -2.5 to 2.5. Along z_7 a damped oscillation turns into an overdamped creep, along z_3 the period changes together with the starting value.

The physics explains the outcome. Among the 1068 overdamped test trajectories the damping coordinate hardly follows $\ln\gamma$, with a correlation of 0.12, and it follows the combination $\ln(\omega^2/\gamma)$, with 0.66, as equation (12.2) predicts. The network stores the combination that the data contain. How well a factor can be stored at all follows from how strongly the trajectory responds to it. The idea is simple. Change one parameter of an oscillator slightly, keep the others, and subtract the old trajectory from the new one. The difference is a curve over the 100 recorded times, the sensitivity of the trajectory to that parameter. A large difference means that the parameter is easy to read from the data. Two parameters whose difference curves have the same shape are hard to tell apart, since a change of one can be imitated by a change of the other, and the cosine between the two difference curves, viewed as vectors of 100 numbers, measures this similarity. For the Gaussian decoder this response is measured by the Fisher information matrix, which sums the products of the sensitivities of the mean trajectory to two factors over the recorded times and grows when small changes of the factors produce large changes of the trajectory. For the mean trajectory $\langle\mathbf{x}\rangle(\mathbf{p})$ with respect to the factors $\mathbf{p} = (\omega, \ln\gamma, x_0, v_0)$ is

$$F_{ab} := \frac{1}{\sigma_x^2} \sum_j \frac{\partial\langle x(t_j)\rangle}{\partial p_a} \frac{\partial\langle x(t_j)\rangle}{\partial p_b}. \tag{12.7}$$

The sum runs over the 100 recorded times. A diagonal element F_{aa} is large when factor a moves the trajectory by many decoder widths. The visibility $\sqrt{F_{aa}}$ times the spread of factor a in the data is the change of the trajectory, in units of σ_x, caused by a change of the factor by one standard deviation. Figure 75 draws the four sensitivity curves for one oscillator, and Table 24 gives the median visibilities over the test set. The damping changes the trajectory most. The initial velocity changes it least, and it is the factor dropped at large β, since its storage costs more nats than it saves in reconstruction. Over the test set the sensitivity directions of the frequency and of the initial velocity are the most alike, with a median cosine of 0.79 between them, so a change of one is partly mimicked by the other, and the frequency is the factor that never gets a coordinate of its own.

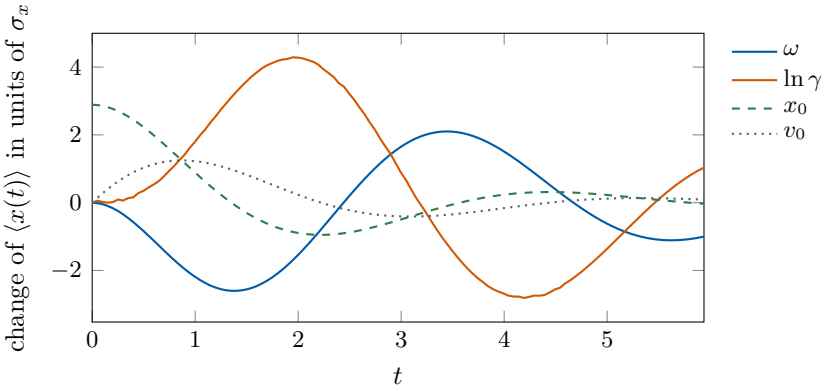


Figure 75: Sensitivity of the mean trajectory to the four factors, for one oscillator with $\omega = 1.5$, $\gamma = 1$, $x_0 = 1$ and $v_0 = 0$. Each curve is the derivative of the noise-free position with respect to one factor, multiplied by the standard deviation of that factor in the data and divided by σ_x, so it shows how many decoder widths the position moves when the factor changes by one standard deviation. The curves are computed from the closed-form solution of the oscillator at this parameter point. The similarity of the frequency and initial-velocity directions quoted in the text is a median over the test set.

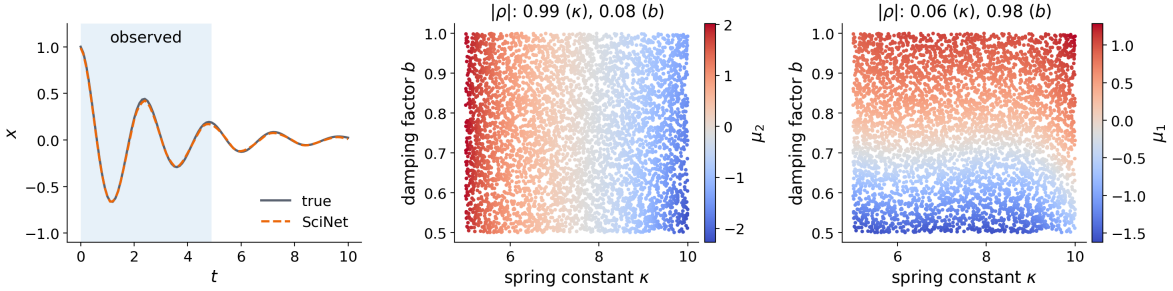


Figure 76: SciNet after 1000 passes on the noise-free pendulum. Left, a test trajectory with $\kappa = 6.83$ and $b = 0.68$ and the prediction from the observed first half (shaded) up to $t = 10$. Middle and right, every test observation placed at its true (κ, b) and coloured by the centre of one active latent neuron. One neuron changes only with κ, the other only with b.

Table 25: SciNet and the β-VAE on the pendulum. KL per latent neuron in nats, best and second-best absolute correlation of the active neurons with κ and with b, and root-mean-square prediction error of SciNet over $t \in [0, 10]$.

data	network	KL per neuron	κ	b	error
noise-free	SciNet, 100 passes	1.60, 0.24, 0.00	1.00, 0.19	0.97, 0.02	0.0514
noise-free	SciNet, 1000 passes	1.65, 0.46, 0.00	0.99, 0.06	0.98, 0.08	0.0178
noise-free	β-VAE	1.65, 0.40, 0.00	0.99, 0.01	0.97, 0.08	–
with noise	SciNet	1.74, 0.48, 0.00	0.99, 0.44	0.88, 0.02	0.0609
with noise	β-VAE	1.74, 0.51, 0.00	0.98, 0.01	0.95, 0.08	–
noise, random x_0, v_0	SciNet	2.28, 1.41, 0.03, 0.03, 0.01	0.93, 0.38	0.31, 0.01	0.0725
noise, random x_0, v_0	β-VAE	1.87, 1.32, 0.00, 0.00, 0.00	0.90, 0.04	0.27, 0.11	–

12.4 SciNet on the pendulum

The first example of Iten et al. [8] is a damped pendulum without noise,

$$\ddot{x} = -\kappa x - b\dot{x}, \qquad x(0) = 1, \quad \dot{x}(0) = -b/2, \tag{12.8}$$

with unit mass, a spring constant κ drawn from 5 to 10 and a damping factor b from 0.5 to 1, and an initial condition that fixes the amplitude and the phase. The observation is the position at 50 times from 0 to 4.9, the questions are the 101 times from 0 to 10, and the answer is the position at the asked time. On the observed window the two parameters change the trajectory in nearly independent ways, with a median cosine of 0.14 between their sensitivity directions. To see what noise and nuisance factors do, two further versions of the data add the random force of strength $\Lambda = 0.002$, and the random force together with random initial conditions drawn as in equation (12.4). Each version has 20 000 training and 5000 test trajectories. SciNet is trained with 8 random question times per observation and step, for 100 passes and, in the setting of the paper, also for 1000 passes, and with five latent neurons for the version with random initial conditions. For comparison the β-VAE (12.5) with $\beta = 4$, the same number of latent coordinates and the 50 observed positions as input is trained on each version for 60 passes.

Table 25 and Figure 76 give the results. In the setting of the paper SciNet stores κ and b in two separate neurons, each correlated strongly with its own parameter and hardly with the other, switches the third neuron off, and after 1000 passes predicts the trajectory to 1.8% of the amplitude. This reproduces the finding of Iten et al. of one neuron per parameter and an error under 2%. The β-VAE finds the same two parameters after 60 passes. The two values of β are on different scales. SciNet weighs the mean squared error of its answers with 1 and the divergence with 10^{-3}. The β-VAE weighs the squared error, summed over the 50 observed positions, with $1/2\sigma_x^2 = 50$ and the divergence with 4. Expressed per mean squared error, its divergence weight is $4/(50 \cdot 50) = 0.0016$, 1.6 times that of SciNet. Weak noise changes little. Random initial conditions change much. Both networks then keep two active neurons, one for κ and one for the initial position, and the damping is lost. With nuisance factors present, both networks store the most visible parameters first, as for the oscillators of Section 12.3.

12.5 Two coupled oscillators

Two equal masses, each attached to a wall by a spring of constant k and to each other by a coupling spring k_c, move in the same fluid. In the coordinates $q_\pm = (x_1 \pm x_2)/\sqrt{2}$ the equations of motion separate into two independent damped oscillators, the normal modes, with the frequencies

$$\omega_1 = \sqrt{k}, \qquad \omega_2 = \sqrt{k + 2k_c}, \tag{12.9}$$

for unit masses, in phase and in antiphase. A trajectory of both masses therefore depends on the springs only through ω_1 and ω_2, and two descriptions of the same system compete, the spring constants (k, k_c) and the mode frequencies (ω_1, ω_2). Since k and k_c are drawn independently, $k \sim \mathcal{U}[1, 4]$ and $k_c \sim \mathcal{U}[0.2, 2]$, the two mode frequencies are correlated, with $\rho = 0.63$, and a coordinate that followed ω_2 perfectly would correlate with k_c at 0.75. One sample joins the 100 positions of both masses into $\mathbf{x} \in \mathbb{R}^{200}$, with the friction and the four initial values drawn at random. A β-VAE with $\beta = 4$ and ten coordinates, 172 764 weights, is trained with two seeds. Both use six active coordinates. Table 26 shows that the coordinates follow ω_2 more closely than k_c, the pattern expected for a coordinate that tracks the antiphase frequency. The network describes the coupling through the normal-mode frequency ω_2, and the in-phase frequency ω_1 gets no coordinate of its own.

12.6 What the data allow

The trajectories make the lesson of the galaxies quantitative. A latent coordinate becomes interpretable when its parameter is visible in the data and no other parameter changes the

Table 26: Two coupled oscillators. Largest absolute correlation of each parameter with an active coordinate of the β-VAE with $\beta = 4$, for two seeds.

parameter	seed 0	seed 1
antiphase frequency ω_2	0.80	0.79
coupling spring k_c	0.52	0.59
in-phase frequency ω_1	0.61	0.51
spring k	0.61	0.52

data in a similar way. In the runs of this chapter the visibility in equation (12.7) indicated whether a parameter was stored at all, and the overlap of the sensitivity directions whether it received a coordinate of its own. Both are local sensitivity arguments, evaluated at the parameters of the data, and the outcome of other architectures and training runs may differ. The physics determines which combination of parameters the data actually contain, ω^2/γ for an overdamped oscillator and ω_2 for coupled masses. Where the data separate the parameters, as for the noise-free pendulum, both networks recover them in single neurons. Where they do not, none of the values of β tried here separated them, and no loss can separate what the data do not distinguish, and a coordinate must be checked against known physics before it is given a name.

13 The four opening questions

The chapter opened with four requests, and the models of the chapter answer them to different degrees. The haematology laboratory asked for more images of a cell type. The conditional variational autoencoder produces colour images of every one of the eight blood-cell types on request (Section 9.7), but a classifier trained on real cells recognises several generated types poorly, so generated cells can enlarge a training set only after such a check. The statistical physicist asked for spin configurations at a chosen temperature. The conditional model produces them directly and follows the change of the magnetisation and the energy with temperature, but its spin correlations and fluctuations differ from those of the Boltzmann distribution (Section 9.8), because the decoder draws the spins independently once the code is fixed. The astronomer asked along which directions galaxy images vary. A β-VAE trained without labels finds one robust direction, brightness and size, and less stable candidates for elongation and colour (Section 11.6). The physicist with the oscillators asked for frequency and damping from the records alone. A β-VAE stores the combinations of the parameters that the trajectories make visible, and SciNet recovers the spring constant and the damping of a noise-free pendulum (Section 12).

All four answers rest on the same construction, a simple random code drawn from $\mathcal{N}(0, I)$ and turned into a sample by a trained network. The variational autoencoder trains this network through the evidence lower bound, which gives a stable loss, an encoder and a measure of how probable the data are, at the price of blurred samples. The adversarial network trains it through a second network, which gives sharp samples, at the price of an unstable game and no likelihood. A label given to both networks makes generation controllable, and a higher price for information can, for some data, make the latent coordinates readable as physical quantities.

Table 27: Training settings. d is the number of latent coordinates, β the weight of the divergence in equation (11.1), with $\beta = 1$ for the plain and the conditional models. The network column gives the hidden layers of encoder / decoder, the activation and the number of trained parameters, where known.

model	data	d	β	decoder	network	training
sine curve, §7.5	4000 points	1	1; 0.02, 20	Gaussian, $s = 0.05$	dense 2×64, tanh; 8900	300 passes
digits, §7.7	60 000 MNIST	2, 16	1	Bernoulli	dense 512, 256 / 256, 512, ReLU; 1 079 600 for $d = 16$	Adam, batch 128, 20 passes
bars, §8.2	20 000 bar images	2	1	Bernoulli	dense 256, 128	—
conditional, digits, §9.5	60 000 MNIST + labels	8	1	Bernoulli	dense, Figure 40; 1 081 120	20 passes
conditional, cells, §9.7	11 959 BloodMNIST + types	16	1	Bernoulli per colour value	convolutional, Table 28	30 passes
conditional, Ising, §9.8	10 200 configurations + T	8	1	Bernoulli	dense 256, 128 / 128, 256; 274 976	60 passes
β-VAE, sprites, §11.2	100 000 dSprites	10	1, 4, 8	Bernoulli	3 conv. 4×4, stride 2 (32, 32, 64), dense 256; mirrored; 633 045	8000 batches of 128, same seed
β-VAE, digits, §11.5	60 000 MNIST	10	1, 4	Bernoulli	dense, Figure 27	10 passes
β-VAE, galaxies, §11.6	15 076 Galaxy10	10	1, 4 (two seeds)	Gaussian, $\sigma_x = 0.1$	4 conv. 4×4, stride 2 (32, 32, 64, 64), dense 256; mirrored	15 passes, random flips
β-VAE, oscillators, §12.3	20 000 trajectories	8	0.5, 1, 4, 8 (three seeds)	Gaussian, $\sigma_x = 0.1$	dense 256, 128, ReLU; mirrored; 120 692	Adam, batch 128, 80 passes
SciNet, pendulum, §12.4	20 000 trajectories	3	as in [8]	Gaussian answer	dense 500, 100 / 100, 100, ELU; 86 907	100 and 1000 passes
β-VAE, coupled oscillators, §12.5	two masses, $\mathbf{x} \in \mathbb{R}^{200}$	10	4 (two seeds)	Gaussian	dense; 172 764	—
GAN, ring, §10.4	fresh draws	2	—	—	dense 128, 128, ReLU, both networks	Adam 10^{-3}, batch 256, 4000 steps
GAN, digits, §10.5	60 000 MNIST	64	—	—	generator 256, 512, tanh output; discriminator 512, 256; leaky ReLU; 550 416 and 533 505	Adam $2 \cdot 10^{-4}$, $\beta_1 = 0.5$, batch 128, 30 passes

A Training settings

Table 27 collects the settings of all models of the chapter. Every model is trained by minimising its loss over all weights of encoder and decoder with the Adam optimiser on mini-batches, and new samples are produced without the encoder, by decoding codes drawn from $\mathcal{N}(0, I)$. Table 28 gives the layers of the convolutional conditional model used for the blood cells.

B Network tests of the generated Ising configurations

Two networks test the generated configurations from outside. A convolutional classifier is trained to tell ordered configurations, with $T < 1.8$, from disordered ones, with $T > 2.8$, and the temperature at which its mean probability of "disordered" crosses one half estimates the critical temperature [31]. A regressor is trained to read the temperature from a configuration. Table 29 gives the results. On Monte Carlo data the classifier finds the transition a few hundredths above Onsager's value, as expected for a 20×20 lattice, and on generated data

Table 28: The convolutional conditional variational autoencoder for BloodMNIST. Shapes are height × width × channels. A 3×3 convolution from C_{in} to C_{out} channels has $9C_{\text{in}}C_{\text{out}} + C_{\text{out}}$ parameters. Total 219 811 parameters.

layer	output	parameters
encoder, input image and $\mathbf{c}$ *as channels,* $28 \times 28 \times (3 + 8)$		
convolution 3×3, stride 2, ReLU	$14 \times 14 \times 32$	3 200
convolution 3×3, stride 2, ReLU, flatten	$7 \times 7 \times 64$	18 496
dense $\to \boldsymbol{\mu}$	16	50 192
dense $\to \ln \boldsymbol{\sigma}^2$	16	50 192
decoder, input $[\mathbf{z}, \mathbf{c}]$, $16 + 8$		
dense, ReLU, reshape	$7 \times 7 \times 64$	78 400
transposed convolution 3×3, stride 2, ReLU	$14 \times 14 \times 32$	18 464
transposed convolution 3×3, stride 2, logits	$28 \times 28 \times 3$	867

Table 29: Network tests of the generated Ising configurations. Top, critical temperature from a phase classifier, mean and standard deviation over three seeds. Bottom, mean absolute error of a temperature regressor on test configurations, one training each.

experiment	estimated T_c
classifier trained on Monte Carlo, tested on Monte Carlo	2.292 ± 0.012
classifier trained on Monte Carlo, tested on generated	2.056 ± 0.012
classifier trained on generated, tested on Monte Carlo	2.501 ± 0.057
Onsager, infinite lattice	2.269
	mean $\lvert\hat{T} - T\rvert$
regressor trained on Monte Carlo, tested on Monte Carlo	0.134
regressor trained on Monte Carlo, tested on generated	0.533
regressor trained on generated, tested on Monte Carlo	0.332
regressor trained on generated, tested on generated	0.302

clearly lower. The regressor reads generated configurations as hotter than requested, by up to 0.78 at $T = 2.5$.

C Counterfactuals of a galaxy classifier

A disentangled latent space can also be used to question a classifier. A convolutional classifier is trained on the real images with their labels. A counterfactual of a galaxy along coordinate k is the decoded image $g_{\boldsymbol{\theta}}(\boldsymbol{\mu}(\mathbf{x}) + \delta\, \mathbf{e}_k)$, where $\mathbf{e}_k$ is the unit vector of coordinate k, and the classifier is asked how its decision changes with δ. The reconstructions themselves, at $\delta = 0$, are a first check, and fewer than half of them receive the class of the real image, because the blurred reconstructions lose the arms and bars that distinguish many classes. Counterfactuals are therefore studied only for test galaxies with a faithful reconstruction. Along every coordinate the predicted class changes for most of them, typically at one prior standard deviation (Table 30). Figure 78 shows one example. Along z_5 the axis ratio of a face-on unbarred tight spiral falls from 0.89 at $\delta = 0$ to 0.37 at $\delta = -3$, and at $\delta = -2$ the class "edge-on without bulge" overtakes "unbarred tight spiral". The classifier responds to the elongation as an astronomer would to a tilted disk, but the decoded images have lost the spiral arms already at $\delta = 0$, so the example shows what the classifier does with the elongation of a smooth blob.

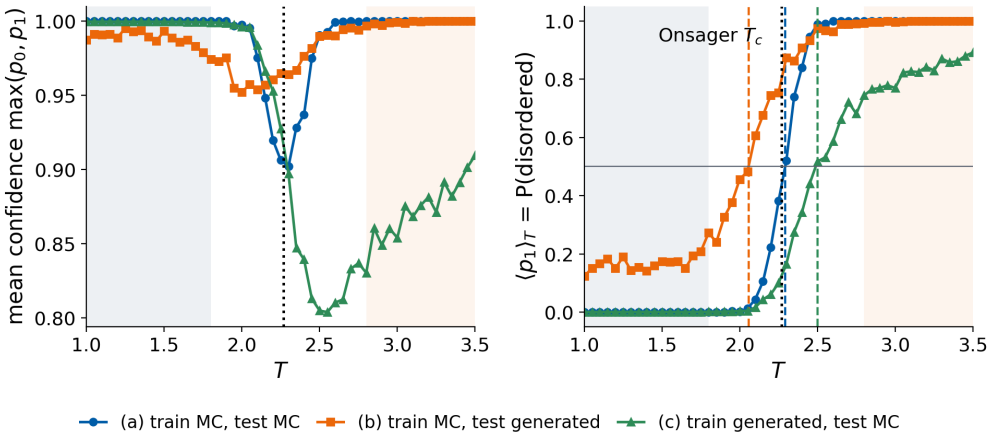


Figure 77: The phase classifier against temperature, pooled over three seeds, for the three combinations of training and test data of Table 29. Left, its mean confidence, the larger of its two class probabilities, which drops where the configurations look like neither phase. Right, its mean probability of "disordered", whose crossing of one half (dashed lines) estimates the critical temperature. The shaded ranges are the temperatures used to train the classifier.

Table 30: The galaxy classifier and its counterfactuals along the coordinates of the β-VAE with $\beta = 4$.

quantity	value
classifier test accuracy on real images	0.764
reconstructions classified like the real image	42%
faithful test galaxies, of which traversed	1129, 269
galaxies whose class flips along a coordinate	60% to 93%

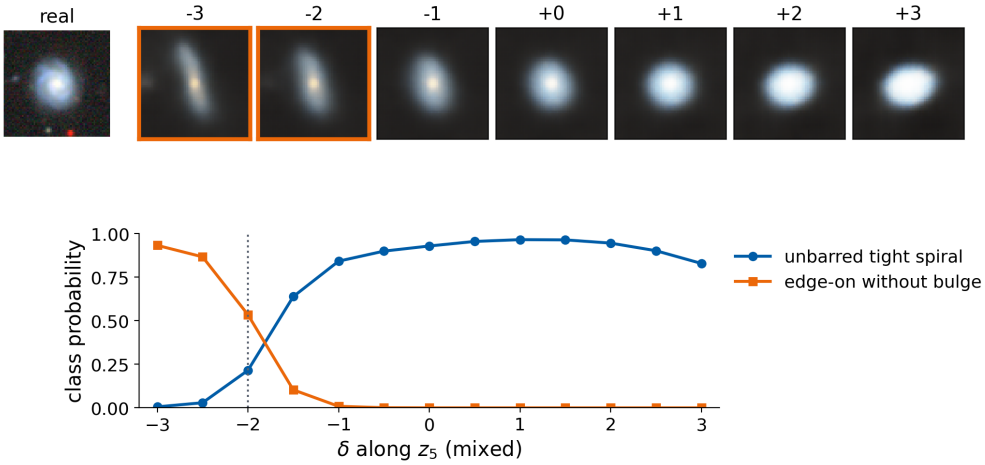


Figure 78: A counterfactual. Decoded images of one test galaxy along a traversal of z_5, with the classifier probabilities of the original class (unbarred tight spiral) and of the class it changes to (edge-on without bulge).

Symbols

The table lists the symbols of the chapter with the sections in which they are introduced. A few letters carry more than one meaning, each in its own context.

symbol	meaning
$\mathbf{x}$, $\mathbf{x}_i$, x_j	a sample, sample i, component or pixel j (throughout); outcome x of a coin (Section 2.1); position of an oscillator (Section 12)
n, D	number of training samples, number of values per sample (throughout); n also the number of coin tosses
$\mathbf{z}$, z_k, d	latent code, its coordinate k, latent dimension (throughout)
θ	bias of a coin, parameter of a model in general (Sections 2 and 3)
$\boldsymbol{\theta}$, $\boldsymbol{\phi}$	decoder and encoder weights of a network (from Section 4 on)
m	number of heads (Section 2.1); magnetisation per spin (Section 9.8)
$P(\cdot)$, $p(\cdot)$	probability of a discrete outcome, probability density
p_{data}	the unknown distribution of the data (Section 3)
p_θ, $p_{\boldsymbol{\theta}}(\mathbf{x})$	model distribution, model density of the data, equation (3.16)
$p(\mathbf{z})$	prior, $\mathcal{N}(0, I)$ (Sections 2.8 and 4)
$p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$	likelihood of the code, the decoder distribution
$p_{\boldsymbol{\theta}}(\mathbf{z} \mid \mathbf{x})$	true posterior of the code
$q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$	encoder cloud, approximate or variational posterior (Section 5.2)
$q(\mathbf{z})$, $q(z_k)$	aggregate posterior and its marginal for coordinate k, equation (8.1)
$L(\theta)$, $\hat{\theta}$	likelihood, its maximiser (Sections 2.7 and 3.2)
$\ell(\theta)$	negative log-likelihood per sample, equation (3.3)
$\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \Sigma)$	Gaussian density with mean $\boldsymbol{\mu}$ and covariance Σ
μ, σ, σ^2	mean, standard deviation, variance of a Gaussian (Section 2)
$\boldsymbol{\mu}$, $\boldsymbol{\sigma}$, μ_k, σ_k	centre and widths of an encoder cloud (from Section 5.2 on)
$\boldsymbol{\varepsilon}$, ε_k	standard Gaussian noise of the reparametrisation, equation (7.1d)
Σ, Σ_{jl}, ρ_{jl}	covariance matrix, its entries, correlation coefficient (Section 2.4)
ρ	correlation of the two coordinates in case (c) of equation (2.14); Pearson correlation of a coordinate with a factor (Sections 11 and 12)
C	sample covariance matrix of Lecture 5
$\mathbf{v}_l$, λ_l, V, Λ	eigenvectors and eigenvalues of Σ and their matrices (Section 2.5)
Λ	noise strength of the Langevin equation (Section 12)
$\mathbb{E}$, Var, Cov	expectation, variance, covariance
$\mathrm{KL}(p \,\|\, q)$, KL_k	Kullback–Leibler divergence, equation (3.9); contribution of latent coordinate k, equation (7.4)
$H(p_{\text{data}}, p_\theta)$, $H(v)$	cross-entropy, equation (3.7); entropy of a factor, equation (11.6)
$\mathrm{ELBO}(\mathbf{x})$	evidence lower bound, equation (6.6)
$w(\mathbf{z})$	importance weight, equation (6.1)
π_k, K	weights and number of components of a mixture, equation (3.13); K also the number of factors in equation (11.8)
g, $g_{\boldsymbol{\theta}}$	a monotone map (Section 3.7); decoder network
$f_{\boldsymbol{\phi}}$	encoder network
X, u, $\mathcal{U}(0,1)$	random variable, uniform draw, uniform distribution on $[0,1]$ (Section 2.3); u also the variable of Jensen's inequality, Theorem 3.3
Φ, F	standard Gaussian cumulative distribution, cumulative distribution of a density, equation (2.6)
k, k_i, λ	Poisson count, count of window i, rate (Section 2.7)
ϖ_j	probability of ink at pixel j in the model of independent pixels, equation (3.2)
s, $\mathcal{I}$, x_{ab}, u_{ab}, v_{ab}	slant, amount of ink, grey value and coordinates of pixel (a, b), equation (9.6)
$\hat{\mathbf{x}}$, $\hat{x}_j$	decoder output, pixel probabilities or Gaussian means
σ_x	width of a Gaussian decoder

Continued on the next page

symbol	meaning (continued)
t, s, $\boldsymbol{\eta}$, $T(z)$	hidden position, noise level and noise of the sine curve, horizontal decoder position (Section 4.2)
W, $\mathbf{b}$, $\mathbf{h}$	weights, biases and hidden activations of a network
B, b	mini-batch size and index (Section 7.4); damping factor b of the pendulum (Section 12.4)
β	weight of the divergence term, equation (11.1)
R	rate, equation (11.3)
$I_q(\mathbf{x};\mathbf{z})$, TC	mutual information of samples and codes, equation (7.5); total correlation, equation (11.5)
$I(z_k;v_j)$, MIG	mutual information of a coordinate and a factor, mutual information gap (Section 11.5)
v_j	generative factor j
y, $\mathbf{c}$, $\mathbf{e}_y$	label, its encoding as network input, one-hot vector (Section 9)
T, T_c, c	temperature, critical temperature, rescaled temperature given to the networks (Section 9.8)
$\mathbf{s}$, s_i, N_s	spin configuration, spin, number of spins (Section 9.8)
$\mathcal{H}$, Z, e, $G(r)$	Ising energy, partition function, energy per spin, spin–spin correlation (Section 9.8)
M_{ab}, $b_\mathrm{g}/a_\mathrm{g}$, ε_g, ϵ_1, ϵ_2, ϑ	second moments of a galaxy image, axis ratio, ellipticity and its components, orientation (Section 11.6); ϑ also the angle of a bar (Section 8.2)
δ, $\mathbf{e}_k$	shift of a latent traversal, unit vector of coordinate k
ω, γ, x_0, v_0	frequency, friction rate, initial position and velocity of an oscillator (Section 12)
$\xi(t)$	Gaussian white noise
F_{ab}	Fisher information matrix, equation (12.7)
$\mathbf{o}$, t_q, $\hat{a}$, C_β	observation, question time, answer and cost of SciNet, equation (12.6)
κ	spring constant of the pendulum (Section 12.4)
k, k_c, ω_1, ω_2	spring constants and normal-mode frequencies of coupled oscillators (Section 12.5); k also a latent index
ψ, $\hat{H}$, E_0	trial state, Hamiltonian and ground-state energy of the variational principle of quantum mechanics (Section 6.3)

References

[1] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE **86**, 2278 (1998), doi:10.1109/5.726791.

[2] J. Yang, R. Shi, D. Wei, Z. Liu, L. Zhao, B. Ke, H. Pfister and B. Ni, MedMNIST v2 — a large-scale lightweight benchmark for 2D and 3D biomedical image classification, Sci. Data **10**, 41 (2023), doi:10.1038/s41597-022-01721-8.

[3] A. Acevedo, A. Merino, S. Alférez, Á. Molina, L. Boldú and J. Rodellar, A dataset of microscopic peripheral blood cell images for development of automatic recognition systems, Data Brief **30**, 105474 (2020), doi:10.1016/j.dib.2020.105474.

[4] L. Matthey, I. Higgins, D. Hassabis and A. Lerchner, dSprites: disentanglement testing sprites dataset (2017), `https://github.com/deepmind/dsprites-dataset`.

[5] H. W. Leung and J. Bovy, Galaxy10 DECaLS, Zenodo (2024), doi:10.5281/zenodo.10845026.

[6] M. Walmsley *et al.*, Galaxy Zoo DECaLS: detailed visual morphology measurements from volunteers and deep learning for 314 000 galaxies, Mon. Not. R. Astron. Soc. **509**, 3966 (2022), doi:10.1093/mnras/stab2093.

[7] A. Dey *et al.*, Overview of the DESI Legacy Imaging Surveys, Astron. J. **157**, 168 (2019), doi:10.3847/1538-3881/ab089d.

[8] R. Iten, T. Metger, H. Wilming, L. del Rio and R. Renner, Discovering physical concepts with neural networks, Phys. Rev. Lett. **124**, 010508 (2020), doi:10.1103/PhysRevLett.124.010508.

[9] J. L. W. V. Jensen, Sur les fonctions convexes et les inégalités entre les valeurs moyennes, Acta Mathematica **30**, 175–193 (1906).

[10] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd edition, Wiley (2006).

[11] D. P. Kingma and M. Welling, Auto-encoding variational Bayes, International Conference on Learning Representations (2014), arXiv:1312.6114.

[12] D. J. Rezende, S. Mohamed and D. Wierstra, Stochastic backpropagation and approximate inference in deep generative models, Proceedings of the 31st International Conference on Machine Learning, PMLR **32**, 1278 (2014), arXiv:1401.4082.

[13] A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, Pixel recurrent neural networks, Proceedings of the 33rd International Conference on Machine Learning, PMLR **48**, 1747 (2016), arXiv:1601.06759.

[14] L. Dinh, J. Sohl-Dickstein and S. Bengio, Density estimation using Real NVP, International Conference on Learning Representations (2017), arXiv:1605.08803.

[15] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio, Generative adversarial nets, Advances in Neural Information Processing Systems **27** (2014), arXiv:1406.2661.

[16] J. Ho, A. Jain and P. Abbeel, Denoising diffusion probabilistic models, Advances in Neural Information Processing Systems **33** (2020), arXiv:2006.11239.

[17] A. Radford, L. Metz and S. Chintala, Unsupervised representation learning with deep convolutional generative adversarial networks, International Conference on Learning Representations (2016), arXiv:1511.06434.

[18] S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz and S. Bengio, Generating sentences from a continuous space, Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning, 10 (2016), doi:10.18653/v1/K16-1002.

[19] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed and A. Lerchner, β-VAE: learning basic visual concepts with a constrained variational framework, International Conference on Learning Representations (2017), `https://openreview.net/forum?id=Sy2fzU9gl`.

[20] C. P. Burgess, I. Higgins, A. Pal, L. Matthey, N. Watters, G. Desjardins and A. Lerchner, Understanding disentangling in β-VAE, arXiv:1804.03599 (2018).

[21] A. A. Alemi, B. Poole, I. Fischer, J. V. Dillon, R. A. Saurous and K. Murphy, Fixing a broken ELBO, Proceedings of the 35th International Conference on Machine Learning, PMLR **80**, 159 (2018), arXiv:1711.00464.

[22] M. D. Hoffman and M. J. Johnson, ELBO surgery: yet another way to carve up the variational evidence lower bound, NIPS 2016 Workshop on Advances in Approximate Bayesian Inference (2016).

[23] R. T. Q. Chen, X. Li, R. Grosse and D. Duvenaud, Isolating sources of disentanglement in variational autoencoders, Advances in Neural Information Processing Systems **31** (2018), arXiv:1802.04942.

[24] M. Rolinek, D. Zietlow and G. Martius, Variational autoencoders pursue PCA directions (by accident), Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 12406 (2019), doi:10.1109/CVPR.2019.01269.

[25] F. Locatello, S. Bauer, M. Lucic, G. Rätsch, S. Gelly, B. Schölkopf and O. Bachem, Challenging common assumptions in the unsupervised learning of disentangled representations, Proceedings of the 36th International Conference on Machine Learning, PMLR **97**, 4114 (2019), arXiv:1811.12359.

[26] K. Sohn, H. Lee and X. Yan, Learning structured output representation using deep conditional generative models, Advances in Neural Information Processing Systems **28** (2015), https://papers.nips.cc/paper_files/paper/2015/hash/8d55a249e6baa5c06772297520da2051-Abstract.html.

[27] P. Comon, Independent component analysis, a new concept?, Signal Process. **36**, 287 (1994), doi:10.1016/0165-1684(94)90029-9.

[28] A. Hyvärinen and E. Oja, Independent component analysis: algorithms and applications, Neural Netw. **13**, 411 (2000), doi:10.1016/S0893-6080(00)00026-5.

[29] L. Onsager, Crystal statistics. I. A two-dimensional model with an order-disorder transition, Phys. Rev. **65**, 117 (1944), doi:10.1103/PhysRev.65.117.

[30] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller and E. Teller, Equation of state calculations by fast computing machines, J. Chem. Phys. **21**, 1087 (1953), doi:10.1063/1.1699114.

[31] J. Carrasquilla and R. G. Melko, Machine learning phases of matter, Nat. Phys. **13**, 431 (2017), doi:10.1038/nphys4035.